

Apostila do Aluno

Projeto Portaria

UC12 · Curso Técnico em Informática

Projeto final · C# com Windows Forms e MySQL

14 métodos para escrever · **1 defeito** para caçar

Aluno: _____

Turma: _____ Data: ____/____/____

Sumário

PARTE 0 — Antes do primeiro método

PARTE 1 — Primeira noite: o cadastro

TODO 1 btnTestarConexao_Click

TODO 2 Usuario.Nome (o set)

TODO 3 ValidarLogin

TODO 4 LoginExiste

TODO 5 UsuarioDAO.CriarConta

TODO 6 frmCadastro.btnCadastrar_Click

★ O teste que fecha a primeira noite

PARTE 2 — Segunda noite: entrar e listar

TODO 7 MontarUsuario

TODO 8 Autenticar

TODO 9 btnEntrar_Click

TODO 10 ListarTodos

TODO 11 UltimoCadastrado

TODO 12 CarregarLista

TODO 13 ExcluirConta

TODO 14 btnExcluir_Click

PARTE 3 — A caça ao defeito

PARTE 4 — O manual de utilização

O roteiro de teste final

O que você entrega

O que é isto

Esta apostila explica, **método por método**, como construir o sistema Portaria — e, mais importante, **por que** cada linha é do jeito que é.

Ela foi escrita para você conseguir fazer sozinho. O professor circula pela sala para ajudar com o que escapar, não para ditar o código.

O sistema	Portaria — login, cadastro de conta e lista de usuários
Onde	Visual Studio 2022 · C# com Windows Forms · .NET 8
Banco	MySQL 8, banco <code>portariadb</code> , uma tabela só
O que você escreve	14 métodos , numerados TODO 1 a TODO 14
Mais	1 defeito já plantado no projeto, para você caçar
Quando	Duas noites

Como usar esta apostila

Cada capítulo tem sempre as mesmas **sete partes**, na mesma ordem:

Parte	Para que serve
O que este método faz	Em português, sem código. Leia isto primeiro
Por que ele existe assim	A decisão de projeto. É aqui que está o conteúdo
O código	O código pronto, exatamente como ele tem que ficar
Linha a linha	O porquê de cada decisão do código
O erro mais comum	O engano que a maioria comete, e como sair dele
Como saber que funcionou	Um teste concreto, que você faz sozinho
Para pensar	Uma pergunta sem resposta — e ela pode cair na arguição

Um conselho, e ele não é retórico

Leia as duas primeiras partes antes de olhar o código. O código está aqui e não vai fugir. Mas na arguição eu não vou perguntar *o que* o método faz — vou perguntar **por que ele é assim**, e essa resposta está nas partes que vêm antes do bloco de código.

Código que você não souber explicar **não conta** para o indicador, mesmo funcionando. Isso não é ameaça: é o motivo de esta apostila ter tanto texto e tão pouco código.

A regra que vale o projeto inteiro

O Regras.cs nao abre banco e nao desenha tela.

Procure dentro do Regras/Regras.cs pelo nome da biblioteca do MySQL e pelo do Windows Forms: **tem que dar zero nos dois**.

Todo método de lá **recebe** valores e **devolve** resposta. Quem fala com o banco é o UsuarioDAO; quem desenha é o frm... É isso que torna as suas regras conferíveis com o MySQL desligado.

As três regras do banco

Valem para os **cinco métodos do UsuarioDAO** que **abrem conexão** — todos menos o MontarUsuario, que só transforma uma linha já lida em objeto:

#	Regra	Por quê
1	Todo valor entra por @parametro, nunca colado com +	Texto colado pode virar comando
2	UPDATE e DELETE sempre com WHERE	Sem ele, o comando pega a tabela inteira
3	A conexão fecha no finally	Fecha mesmo quando dá erro no meio

— Um DELETE sem WHERE no código entregue reprova o indicador I5 direto. Não é rigor: é que ele esvazia a tabela toda de uma vez, e não há como desfazer.

PARTE 0 — Antes do primeiro método

Nada nesta apostila funciona com o banco fora do ar. **Faça esta parte inteira antes de escrever qualquer linha de C#.**

1. Confira o que já está instalado

#	O quê	Como conferir
1	Visual Studio 2022	Ajuda > Sobre: aparece 17.x
2	.NET SDK 8	O template "Aplicativo do Windows Forms" existe
3	Serviço MySQL80	Windows + R, digite <code>services.msc</code> , procure MySQL80 : tem que estar Em execução
4	MySQL Workbench	Abre e conecta no localhost

2. Crie o banco

- Workbench > **File** > **Open SQL Script** > `banco/CriarBanco.sql`
- Clique no **raio amarelo** (*Execute All*), não no raio com o cursor
- Atualize o painel **SCHEMAS**, na esquerda. O banco `portariadb` tem que aparecer
- Abra `portariadb` > *Tables*. A tabela `Usuario` tem que estar lá, com 5 colunas

— **O script começa com `DROP DATABASE`**. Rode uma vez só. Rodar de novo, depois de já ter cadastrado usuários, apaga os usuários.

O `conectadb` das aulas anteriores continua na sua máquina, intacto — são dois bancos diferentes.

3. Abra o projeto e ligue a conexão

- Abra `portaria-inicial/Portaria.sln`
- Aperte F5 antes de escrever qualquer coisa.** A tela de login abre
- Abra `Dados/Conexao.cs` e troque `SUA_SENHA_AQUI` pela senha do **seu** MySQL
- F5 de novo

A senha do laboratório está no quadro. Se você estiver no seu computador, é a que você escolheu quando instalou o MySQL.

4. Ache o pacote do MySQL

Em **Projeto** > **Gerenciar Pacotes NuGet** > **Instalado**, o `MySql.Data` tem que aparecer. Ele já vem declarado no `.csproj` — **este passo é para você saber onde ele fica**, porque isso cai na arguição.

5. A tabela que você vai usar o projeto inteiro

```
CREATE TABLE Usuario (  
    Id            INT AUTO_INCREMENT PRIMARY KEY,  
    Nome          VARCHAR(60) NOT NULL,  
    Login         VARCHAR(30) NOT NULL,  
    Senha         VARCHAR(50) NOT NULL,  
    DataCadastro DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
    CONSTRAINT UQ_Usuario_Login UNIQUE (Login)  
);
```

Duas coisas dessa tabela decidem código mais para a frente, e vale entender agora:

O que está escrito	O que isso significa para o seu C#
<code>Id INT AUTO_INCREMENT</code>	Você nunca manda o <code>Id</code> num <code>INSERT</code> . Quem gera é o banco
<code>DataCadastro ... DEFAULT CURRENT_TIMESTAMP</code>	Você nunca manda a data. O banco carimba sozinho — e é por isso que o <code>INSERT</code> do TODO 5 tem três parâmetros, não cinco
<code>UNIQUE (Login)</code>	O banco recusa dois logins iguais. O TODO 4 existe para dar a mensagem bonita; a garantia é esta linha

6. Onde cada coisa mora

Modelo/Usuario.cs	o molde de uma conta	TODO 2
Modelo/Sessao.cs	quem esta logado agora	(pronto)
Regras/Regras.cs	as regras, sem banco	TODO 3 e 11 + o defeito
Dados/Conexao.cs	o endereço do banco	(so a senha)
Dados/UsuarioDAO.cs	a conversa com o MySQL	TODO 4,5,7,8,10,13
Telas/frmLogin.cs	entrar	TODO 1 e 9
Telas/frmCadastro.cs	criar conta	TODO 6
Telas/frmPrincipal.cs	a lista	TODO 12 e 14

A armadilha avisada

Cada método a escrever nasce com uma linha provisória:

```
return false;    // <<< TROQUE ESTA LINHA pela sua
```

Ela existe **só para o projeto compilar antes de você começar**. Se ela ficar, o método responde sempre a mesma coisa — e você vai procurar o erro no lugar errado.

Nas telas, a provisória é um `MessageBox` dizendo que o TODO não foi escrito — com uma exceção, o TODO 12, onde ela escreve no rodapé.

PARTE 1 — Primeira noite: o cadastro

TODO 1 a 6. No fim desta noite o sistema cria contas de verdade, e elas ficam gravadas no MySQL.

#	Método	O que muda quando você escreve
1	<code>btnTestarConexao_Click</code>	A faixa acende verde. É o primeiro — sem ele nada mais funciona
2	<code>Usuario.Nome</code> (o <code>set</code>)	O nome para de entrar torto
3	<code>Regras.ValidarLogin</code>	A tela ganha como perguntar se o login serve
4	<code>UsuarioDAO.LoginExiste</code>	O sistema recusa login repetido
5	<code>UsuarioDAO.CriarConta</code>	O primeiro dado que sai do programa e fica no banco
6	<code>frmCadastro.btnCadastrar_Click</code>	O cadastro funciona. Fim da primeira noite

TODO 1 - btnTestarConexao_Click

Arquivo	Telas/frmLogin.cs
Indicador	I1 · I4
O que voce exercita	<code>if</code> / <code>else</code> , parametro <code>out</code> , chamada de metodo estatico de outra classe, <code>MessageBox.Show</code> com titulo e icone, alteracao de propriedade de controle em tempo de execucao
Depende de	Nada de codigo. Depende de o banco <code>portariadb</code> existir e de a senha estar certa no <code>Conexao.cs</code>

O que este método faz

Este metodo roda quando voce clica no botao **Testar conexao** da tela de login. Ele nao recebe nada de voce e nao devolve nada para ninguem: os dois parametros (`sender` e `e`) sao entregues pelo Windows Forms toda vez que um evento acontece, e voce nao precisa deles aqui. O que ele faz e pedir para a classe `Conexao` tentar abrir uma conversa com o MySQL e, dependendo do resultado, escrever uma frase na etiqueta `lblStatus` e abrir uma caixa de mensagem.

Ele existe para uma coisa muito pratica: enquanto ele nao acender verde, nenhum outro TODO da apostila vai funcionar. Cadastrar usuario, autenticar, listar - tudo passa pelo banco. Este e o primeiro metodo da noite justamente porque ele e o unico que voce consegue testar sem depender de mais nada. E, sem ele, cada vez que voce corrigisse a senha no `Conexao.cs` teria que fechar o programa, recompilar e abrir de novo so para descobrir se acertou. Com o botao, voce corrige, roda uma vez e clica quantas vezes precisar.

Por que ele existe assim

Repare no que este metodo **nao** faz: ele nao escreve `server=localhost`, nao cria `MySqlConnection`, nao chama `Open()`, nao tem `try/catch`. Nada disso mora aqui. Ele so pergunta `Conexao.TestarConexao(out erro)` e reage a resposta.

Essa separacao e a decisao de projeto mais importante do capitulo. O endereco do banco esta escrito **uma vez so**, dentro do `Conexao.cs`. Se voce trocasse de maquina e a senha do MySQL mudasse, voce mexeria em um lugar e o sistema inteiro passaria a usar a senha nova. Se cada tela montasse a propria string de conexao, voce teria que cacar e corrigir a senha em cinco arquivos diferentes, e no dia da apresentacao um deles ficaria para tras. A tela cuida da tela; os dados cuidam dos dados.

A segunda decisao esta na assinatura do `TestarConexao`. Ele podia devolver a mensagem de erro como `string` e deixar a string vazia significar "deu certo". Nao e o que ele faz: ele devolve `bool` pelo `return` e a mensagem por um parametro `out`. Isso e exatamente o desenho do `double.TryParse` que voce ja usou - `TryParse` devolve `true` / `false` pelo return e o numero convertido pelo `out`. A vantagem e que a pergunta "conectou?" tem uma resposta de sim ou nao, clara o bastante para caber dentro de um `if`, e o detalhe vem por fora sem atrapalhar. Voce le a primeira linha do `if` e ja sabe o que esta sendo perguntado.

A terceira decisao e mostrar o resultado em **dois lugares**: na etiqueta e na caixa de mensagem. Nao e repeticao. A caixa de mensagem interrompe voce, mostra o texto do erro (que tem mais de uma linha, com a instrucao do que fazer) e some quando voce clica em OK. A etiqueta fica. Depois que a caixa fecha, a tela continua dizendo se ha banco ou nao, e e essa informacao que voce vai olhar de canto de olho enquanto trabalha.

Por fim, este metodo e irmao do `frmLogin_Shown`, que ja vem pronto logo acima dele no arquivo. O `Shown` faz a mesma checagem automaticamente quando a janela aparece. A diferenca e que o `Shown` roda uma unica vez e nao abre caixa de mensagem nenhuma - so pinta a etiqueta. Leia o `Shown` antes de escrever o seu: ele ja e metade do desenho.

O código

```
private void btnTestarConexao_Click(object sender, EventArgs e)
{
    string erro;

    if (Conexao.TestarConexao(out erro))
    {
        lblStatus.Text = "Banco conectado: portariadb";
        lblStatus.ForeColor = System.Drawing.Color.DarkGreen;

        MessageBox.Show("Conectado ao banco portariadb.",
                        "Conexao OK",
                        MessageBoxButtons.OK, MessageBoxIcon.Information);
    }
    else
    {
        lblStatus.Text = "SEM BANCO";
        lblStatus.ForeColor = System.Drawing.Color.Firebrick;

        MessageBox.Show(erro,
                        "Nao consegui falar com o banco",
                        MessageBoxButtons.OK, MessageBoxIcon.Warning);
    }
}
```

Linha a linha

Linha	Por que ela esta ai
<pre>private void btnTestarConexao_Click(object sender, EventArgs e)</pre>	Esta assinatura ja existe no esqueleto e nao pode ser mudada. Os dois parametros, nesta ordem e com estes tipos, sao o formato que o Windows Forms exige de qualquer metodo que responde a um evento. O nome tambem tem que ficar como esta, mas por outro motivo: dentro do <code>frmLogin.Designer.cs</code> ha a linha <pre>this.btnTestarConexao.Click += new System.EventHandler(this.btnTestarConexao_Click);</pre> , e e ela que amarra o botao a este metodo. Se voce renomear o metodo, essa linha passa a chamar um nome que nao existe mais e o projeto nao compila. O <code>private</code> diz que so esta tela usa este metodo; o <code>void</code> diz que ele nao devolve nada, porque nao ha ninguem esperando resposta - quem chamou foi o clique do mouse.
<pre>string erro;</pre>	Voce declara a caixinha antes da chamada porque o <code>out</code> precisa de um lugar ja existente para depositar a mensagem. Repare que nao ha <code>= ""</code> aqui, e mesmo assim o compilador nao reclama: quem promete preencher essa variavel e o <code>out</code>. Um metodo com <code>out</code> e obrigado a atribuir valor ao parametro antes de terminar - por isso, depois da chamada, <code>erro</code> sempre tem um valor. Quando a conexao da certo, esse valor e a string vazia; quando da errado, e a mensagem do problema.
<pre>if (Conexao.TestarConexao(out erro))</pre>	<code>Conexao.</code> na frente porque <code>TestarConexao</code> e um metodo estatico: ele pertence a classe, nao a um objeto. Voce nao escreve <code>new Conexao()</code> em lugar nenhum - nao ha o que guardar dentro de um objeto <code>Conexao</code>, o endereco do banco e sempre o mesmo. Sem o <code>Conexao.</code> na frente, o compilador procura um <code>TestarConexao</code> dentro do <code>frmLogin</code>, nao acha, e acusa erro.
<pre>out erro</pre>	O <code>out</code> tem que aparecer na chamada tambem, nao so na declaracao do metodo. Ele e um aviso escrito para quem le: "esta variavel vai sair daqui diferente do que entrou". Sem a palavra <code>out</code>, o codigo nao compila.
<pre>if (...) sem == true</pre>	<code>TestarConexao</code> ja devolve um <code>bool</code>. Escrever <code>== true</code> funciona, mas e como perguntar "e verdade que e verdade?". O valor ja e a propria condicao.
<pre>lblStatus.Text = "Banco conectado: portariadb";</pre>	<code>Text</code> e a propriedade que guarda o que a etiqueta mostra. Atribuir a ela troca o texto na hora, com a janela aberta. E o mesmo <code>.Text</code> da caixa de texto e do botao - o Windows Forms usa o mesmo nome para "o texto que aparece" em quase todos os controles.
<pre>lblStatus.ForeColor = System.Drawing.Color.DarkGreen;</pre>	<code>ForeColor</code> e a cor da letra (a cor do fundo seria <code>BackColor</code>). A cor entra como verde escuro, e nao verde puro, porque letra pequena em verde vivo sobre fundo claro fica dificil de ler.

Linha	Por que ela esta ai
<code>System.Drawing.Color</code> escrito por extenso	Olhe o topo do arquivo: so ha <code>using System;</code> e <code>using System.Windows.Forms;</code> . Nao ha <code>using System.Drawing;</code> . Sem esse atalho, voce precisa dizer o caminho completo ate a classe <code>Color</code> . Se escrever so <code>Color.DarkGreen</code> , nao compila.
<code>MessageBox.Show("Conectado ao banco portariadb.", "Conexao OK", MessageBoxButtons.OK, MessageBoxIcon.Information)</code>	Quatro argumentos, nesta ordem: o texto de dentro, o titulo da barra da janelinha, quais botoes ela tem e qual icone. <code>MessageBoxButtons.OK</code> porque nao ha nada a decidir - o teste ja foi feito, voce so esta sendo informado. <code>Information</code> e o icone azul de aviso neutro.
<code>MessageBox.Show(erro, ...)</code> no <code>else</code>	Aqui entra a variavel , sem aspas. Escrever <code>MessageBox.Show("erro", ...)</code> com aspas mostraria a palavra "erro" na tela em vez da mensagem. O conteudo de <code>erro</code> foi montado la dentro do <code>Conexao.cs</code> , que traduz o numero do erro do MySQL numa frase em portugues dizendo o que fazer - por isso ela vem com mais de uma linha.
<code>"SEM BANCO"</code> sem detalhe na etiqueta	A etiqueta e estreita e o texto do erro tem duas ou tres linhas, conforme a falha. O detalhe cabe na caixa de mensagem; na etiqueta fica so o estado.
<code>MessageBoxIcon.Warning</code> no erro, e nao <code>Error</code>	Nao houve defeito no seu programa: o banco simplesmente nao respondeu, e isso e uma situacao que voce mesmo conserta e testa de novo. O icone amarelo diz "veja isso"; o vermelho, que voce vai usar no TODO 9, diz "o que voce tentou foi recusado".
<code>else</code> e nao um segundo <code>if</code>	Sao dois casos que se excluem: ou conectou, ou nao conectou. O <code>else</code> garante que exatamente um dos dois blocos roda, sem risco de os dois rodarem e a etiqueta terminar com a cor errada.

O erro mais comum

1. Deixar a linha do aviso no lugar. O esqueleto vem com `MessageBox.Show("TODO 1 ainda nao foi escrito.");` e o comentario `// <<< APAGUE esta linha`. Se voce escrever o codigo por cima e esquecer de apagar essa linha, ao clicar no botao aparecem **duas** caixas de mensagem: primeiro o aviso do TODO, depois a de verdade. Apague a linha inteira.

2. Dar duplo clique no botao dentro do Designer. Essa e a que mais custa tempo. No Designer, o duplo clique cria um metodo novo, chamado `btnTestarConexao_Click_1`, e liga o botao **nele**. Voce entao escreve o codigo no metodo certo, roda, clica, e nao acontece nada - porque o clique esta indo para o metodo vazio de nome `_1`. Neste projeto o evento **ja esta ligado**: abra o `frmLogin.cs` (com o botao direito no formulario, "VerCodigo", ou por F7) e escreva dentro do metodo que ja existe. Se voce ja criou o `_1` sem querer, conserte nesta ordem: primeiro va a janela Propriedades do botao, aba do raio (Eventos), apague o que estiver escrito no campo Click e escolha de novo `btnTestarConexao_Click` na lista; **so depois** apague do codigo o metodo `_1` vazio. Se fizer ao contrario, o projeto passa um tempo sem compilar, porque o Designer continua chamando um metodo que voce ja apagou.

3. Esquecer o `out` na chamada. Escrever `Conexao.TestarConexao(erro)` da erro de compilacao dizendo que o argumento 1 deve ser passado com a palavra-chave `out`. O programa nem chega a rodar. Acrescente o `out` e compile de novo.

4. Aparecer a caixa amarela com "Usuario ou senha do MySQL incorretos". Isso nao e defeito no seu TODO 1 - e o seu TODO 1 funcionando. Voce ainda nao trocou `SUA_SENHA_AQUI` pela senha do seu MySQL no `Conexao.cs`, na linha do `pwd`. Troque, salve, rode de novo e clique no botao.

5. Escrever `Color.DarkGreen` sem o `System.Drawing.` na frente. O compilador diz que o nome `Color` nao existe no contexto atual. E porque falta o `using System.Drawing;` no topo do arquivo. Escreva o nome completo, como esta no gabarito.

Como saber que funcionou

Faca o teste em tres etapas, nesta ordem.

Primeiro, o caso bom. Rode o projeto (F5). Assim que a tela de login aparecer, a etiqueta de baixo ja deve estar verde escrito "Banco conectado: portariadb" - isso e o `frmLogin_Shown`, que ja vinha pronto. Clique em **Testar conexao**. Tem que aparecer uma caixa com icone azul, titulo "Conexao OK" e o texto "Conectado ao banco portariadb.". Clique em OK: a etiqueta continua verde.

Segundo, o caso ruim. Feche o programa. Abra o `Conexao.cs` e troque um caractere da senha na linha do `pwd` de proposito. Rode de novo. Repare que a etiqueta ja abre vermelha, escrito "SEM BANCO - clique em Testar conexao" - isso ainda e o `frmLogin_Shown`, nao o seu metodo. Agora clique no botao: tem que vir a caixa amarela, com titulo "Nao consegui falar com o banco" e o texto "Usuario ou senha do MySQL incorretos.", com "Confira o pwd no Conexao.cs." na linha de baixo. E a etiqueta tem que **trocar de texto**, passando a mostrar so "SEM BANCO", sem o complemento. E essa troca de texto que prova que quem escreveu na etiqueta agora foi o seu `else` - a cor vermelha sozinha nao prova nada, porque ela ja estava assim antes do clique. Conserte a senha antes de seguir.

Terceiro, os tres erros do laboratorio. O esqueleto pede que voce provoque tres falhas diferentes e anote a mensagem de cada uma - e esse texto que vai para o Registro de Depuracao:

1. Pare o servico do MySQL (Windows + R, digite `services.msc`, ache **MySQL80** e pare). Clique no botao: a mensagem deve falar do servico.
2. Erre a senha no `pwd` do `Conexao.cs` (o teste que voce acabou de fazer).
3. Troque o nome do banco no `Conexao.cs`, de `portariadb` para `portariadb2`. A mensagem deve dizer que o banco nao existe.

Depois dos tres, devolva tudo ao estado original: servico rodando, senha certa, `database=portariadb`.

Conferindo pelo Workbench. Se a mensagem disser que o banco nao existe e voce jurar que ele existe, abra o MySQL Workbench e rode:

```
SHOW DATABASES;  
USE portariadb;  
SHOW TABLES;
```

A primeira consulta tem que listar `portariadb`, e a ultima tem que mostrar a tabela `Usuario`. Se `portariadb` nao aparecer, o `CriarBanco.sql` nao rodou; se o banco aparecer mas a tabela nao, ele rodou pela metade - rode de novo, inteiro.

Para pensar

O `frmLogin_Shown`, que já vem pronto, faz exatamente a mesma checagem que você escreveu, mas não abre caixa de mensagem nenhuma - só pinta a etiqueta. Por que o programa avisa com uma janela quando **você** pede o teste, e fica em silêncio quando ele mesmo testa sozinho ao abrir?

TODO 2 - Usuario.Nome (o set)

Arquivo	Modelo/Usuario.cs
Indicador	I2
O que voce exercita	campo privado + propriedade escrita a mao (com corpo, nao automatica), <code>if / else</code> , a palavra <code>value</code> , <code>Trim()</code>
Depende de	Nada. O campo <code>private string nome = ""</code> ; e o <code>get</code> ja vem prontos no esqueleto; dentro do <code>set</code> ha uma linha <code>nome = value;</code> marcada com <code>// <<< TROQUE ESTA LINHA</code> pela sua — e ela que voce substitui. Quem depende dele: TODO 6, TODO 7 e TODO 12.

O que este método faz

O `set` do `Nome` e um metodo pequeno com nome escondido. Toda vez que alguma parte do programa escreve `algumUsuario.Nome = alguma coisa`, o C# nao joga o valor direto na memoria: ele chama este bloco e entrega o que estava do lado direito do `=` dentro de uma palavra chamada `value`. O bloco entao decide o que guardar no campo `nome`, que e onde o texto de verdade fica.

A decisao dele e curta: se chegou nulo, guarda texto vazio; se chegou texto, guarda esse texto sem os espacos das pontas. Ele nao recusa nada, nao mostra MessageBox e nao interrompe o programa. Quem chama esse `set` no projeto sao duas linhas: `novo.Nome = txtNome.Text;` no TODO 6 (tela de cadastro) e `u.Nome = Convert.ToString(r["Nome"]);` no TODO 7 (quando o objeto e montado a partir do que veio do banco). Do outro lado, quem le o `Nome` e a tela principal, tanto na lista quanto na saudacao do alto.

Por que ele existe assim

As outras quatro propriedades da classe (`Id`, `Login`, `Senha`, `DataCadastro`) sao automaticas — `{ get; set; }`, sem corpo nenhum. Duas delas, `Login` e `Senha`, ainda trazem um `= ""` no fim da linha, mas isso e so o valor inicial: nao ha codigro rodando na hora da atribuicao. Uma propriedade automatica so guarda o que mandaram. Quando o valor precisa passar por algum tratamento antes de entrar, a propriedade ganha corpo — um `get` e um `set` escritos por voce — e um campo privado por tras para segurar o texto. E o encapsulamento na pratica: quem decide o que entra no objeto e o proprio objeto, nao quem esta atribuindo.

A limpeza podia estar na tela: bastava alguem escrever `novo.Nome = txtNome.Text.Trim();` no TODO 6 e o `set` continuaria de uma linha so. O problema e que existem dois caminhos de entrada, nao um. Se amanha o projeto ganhar uma tela de edicao, ou uma importacao de planilha, cada caminho novo precisa lembrar de chamar `Trim()`. Um deles vai esquecer. Colocando a limpeza no `set`, ela acontece em todos os caminhos, inclusive nos que ainda nao existem — e existe um unico lugar para consertar se a regra mudar.

Repare tambem no que o `set` **nao** faz. Ele nao recusa nome vazio, nao reclama de nome curto, nao avisa ninguem. Isso e proposital: um objeto do modelo nao conversa com o usuario — ele nao tem tela, nao tem MessageBox, nao sabe se ha alguem olhando. Quem recusa dado ruim e a tela de cadastro, na mensagem M1 ("Preencha todos os campos."), onde da para explicar o problema em portugues e parar o cadastro com um `return`. O modelo fica com a parte que ele consegue fazer sozinho: **arrumar**, nao julgar.

Por fim, o motivo pratico do `Trim()`. Sem ele, um nome digitado como `" Ana Paula"` entra no banco com dois espacos na frente e produz dois estragos visiveis. Na lista da tela principal, os nomes ocupam uma coluna

montada com `u.Nome.PadRight(24)`: os dois espaços entram na conta dos 24, então o nome daquela linha aparece deslocado para a direita, desalinhado dos nomes de todas as outras — e se os espaços empurrarem o texto para além dos 24 caracteres, o `PadRight` não tem mais o que completar e as colunas seguintes saem do lugar também. E na saudação do alto, o `Regras.PrimeiroNome` procura o primeiro espaço do texto para cortar ali; se o primeiro espaço está na posição zero, o pedaço cortado antes dele é vazio, e o rótulo mostra só `Conectado como:`, sem nome nenhum. Nenhum dos dois dá erro; os dois só ficam feios, e ninguém descobre a causa olhando a tela principal.

O código

```
private string nome = "";

public string Nome
{
    get { return nome; }
    set
    {
        if (value == null)
        {
            nome = "";
        }
        else
        {
            nome = value.Trim();
        }
    }
}
```

Linha a linha

Linha	Por que ela esta ai
<pre>private string nome = "";</pre>	O campo: e aqui que o texto realmente mora. <code>private</code> quer dizer que so codigo de dentro da classe <code>Usuario</code> enxerga esse campo — de fora, o unico caminho e a propriedade, e por isso a propriedade consegue impor a regra. O <code>= ""</code> no fim garante que um <code>new Usuario()</code> recém-criado ja tenha nome vazio; se nao houvesse, um objeto criado e nunca preenchido carregaria nulo, e a primeira linha que tentasse medir esse nome com <code>.Length</code> ou chamar <code>PadRight</code> nele quebraria. (Juntar nulo com <code>+</code> nao quebra — nesse caso o C# troca o nulo por texto vazio. Quem derruba o programa e o acesso a um membro do nulo.) Esta linha ja vem pronta no esqueleto.
<pre>public string Nome</pre>	A propriedade: o nome publico, com N maiusculo, que o resto do programa usa. Note que ela abre chaves com <code>get</code> e <code>set</code> dentro, em vez de terminar em <code>{ get; set; }</code> . Essa e a forma que se usa quando ha trabalho a fazer na entrada ou na saida.
<pre>get { return nome; }</pre>	Devolve o campo, que a esta altura ja esta limpo — quem le nunca ve o texto sujo. Repare que devolve <code>nome</code> (minusculo, o campo) e nao <code>Nome</code> . Escrever <code>return Nome;</code> aqui faria o <code>get</code> chamar o proprio <code>get</code> , sem parada.
<pre>set</pre>	Bloco chamado automaticamente a cada atribuicao. Voce nunca escreve <code>set(...)</code> com parentese e nunca declara parametro: o valor recebido ja chega com o nome fixo <code>value</code> , do tipo da propriedade (aqui, <code>string</code>).
<pre>if (value == null)</pre>	O teste do nulo vem primeiro , antes de qualquer coisa tocar no texto. Nulo nao e texto vazio: vazio e um texto de zero letra, com o qual da para trabalhar; nulo e a ausencia de texto, e chamar qualquer metodo nele derruba o programa. Como o <code>Trim()</code> e um metodo, ele so pode ser chamado depois que voce ja sabe que ha texto ali.
<pre>nome = "";</pre>	Traduz "nada" para "texto de zero letra". A partir daqui, todo o resto do projeto pode medir o <code>Nome</code> com <code>.Length</code> , juntar com <code>+</code> , comparar, chamar <code>PadRight</code> — sem uma unica checagem de nulo espalhada pelas telas. Essa e a razao de a apostila insistir em nunca deixar um <code>string</code> do modelo nulo.
<pre>else</pre>	O caminho normal, o de todo dia: chegou texto de verdade.
<pre>nome = value.Trim();</pre>	<code>Trim()</code> devolve um novo texto sem os espacos (e tabulacoes) das duas pontas; ele nao altera o texto original, porque texto em C# nao se altera depois de criado. Por isso o resultado precisa ser atribuido: escrever <code>value.Trim();</code> sozinho compila e nao faz absolutamente nada. E note o alvo da atribuicao: <code>nome</code> , o campo. Escrever <code>Nome = value.Trim();</code> aqui seria pedir ao <code>set</code> que chamasse o <code>set</code> .

Linha	Por que ela esta ai
o que <code>Trim()</code> nao faz	Ele mexe so nas pontas. <code>"Ana Paula"</code> continua com os quatro espacos do meio depois do <code>Trim()</code> . Isso e o combinado — o projeto arruma sobra de digitacao nas bordas, nao reescreve o nome da pessoa.

O erro mais comum

- Atribuir a propriedade em vez do campo.** Escrever `Nome = value.Trim();` (ou `return Nome;` no `get`). O projeto compila normalmente, sem nenhum aviso vermelho. Na hora em que a primeira atribuicao acontece, o `set` chama a si mesmo, essa chamada chama de novo, e a coisa se repete ate a memoria de chamadas acabar. O sintoma e assustador porque nao e uma `MessageBox`: o programa simplesmente morre, ou o Visual Studio para com `"StackOverflowException"` — e nenhum `try/catch` que voce escrever consegue segurar isso. A correcao e uma letra: `n` minusculo, o campo.
- Chamar `Trim()` antes de testar o nulo.** Inverter para `nome = value.Trim(); if (...)` faz o programa parar com `"NullReferenceException"` (referencia de objeto nao definida) na primeira vez que alguem atribuir nulo. A ordem do `if` nao e enfeite: e a propria protecao.
- Testar a coisa errada.** `if (value == "")` parece equivalente e nao e: texto vazio nao e nulo, e o `else` desse `if` continua chamando `Trim()` em cima de nulo. O teste tem que ser `== null`.
- Chamar `Trim()` sem guardar o resultado.** `value.Trim();` numa linha solta compila e nao muda nada: o texto sujo continua indo para o campo. O sintoma e mudo — o programa funciona, so que os nomes continuam entrando com espaco, e voce so descobre olhando o banco.
- Apagar o campo e voltar para `{ get; set; }`.** O projeto compila e roda, mas a limpeza deixou de existir. Se o `Trim()` sumiu junto, o sintoma aparece la na frente: nomes deslocados na lista e a saudacao sem nome.

Como saber que funcionou

Teste isolado, sem banco. No evento do botao "Testar conexao" da tela de login (o mesmo do TODO 1), cole temporariamente estas quatro linhas, como primeira coisa do metodo:

```
Usuario teste = new Usuario();
teste.Nome = "  Ana Paula  ";
MessageBox.Show("[ " + teste.Nome + " ]");
teste.Nome = null; MessageBox.Show("[ " + teste.Nome + " ]");
```

Rode e clique no botao. A primeira caixa tem que mostrar `[Ana Paula]`, com os colchetes colados nas letras — se aparecer `[ Ana Paula ]`, o `Trim()` nao esta sendo guardado no campo. A segunda tem que mostrar `[]` e o programa continuar de pe — se ele parar com erro, o `if (value == null)` esta faltando ou esta depois do `Trim()`. Os colchetes existem justamente para voce enxergar o espaco, que sozinho e invisivel. **Apague estas quatro linhas depois do teste;** elas nao fazem parte do projeto.

Teste completo, depois que os TODO 5 e 6 estiverem prontos. Cadastre um usuario digitando o nome com espacos de proposito: `" Ana Paula "`. Depois, no MySQL Workbench:

```
SELECT Id, CONCAT('[', Nome, ']') AS Conferencia, CHAR_LENGTH(Nome) AS Tamanho
FROM Usuario ORDER BY Id DESC LIMIT 3;
```

A coluna `Conferencia` tem que sair `[Ana Paula]` e o `Tamanho` tem que ser 9, não 13. Se vier 13, o que chegou ao banco foi o texto sujo — o `set` não limpou.

Com breakpoint. Coloque o breakpoint na linha `if (value == null)` e rode o cadastro. Quando o programa parar ali, passe o mouse sobre `value` e leia o que veio da tela; aperte F10 duas ou três vezes e veja o campo `nome` receber a versão limpa. É o jeito mais direto de provar que a atribuição da tela realmente passa por dentro deste bloco.

Para pensar

O mesmo resultado no banco seria obtido escrevendo `novo.Nome = txtNome.Text.Trim();` no TODO 6 e deixando o `set` com uma linha só. Nos dois casos o texto chega limpo ao MySQL. O que exatamente o projeto perde ao tirar a limpeza de dentro do objeto e espalha-la pelas telas que atribuem o nome?

TODO 3 - ValidarLogin

Arquivo	Regras/Regras.cs
Indicador	I2 · I3
O que voce exercita	<code>if</code> em cadeia com saída antecipada (<code>return false</code> dentro do <code>if</code>), retorno de <code>bool</code> , uso de constante no lugar de número solto, métodos de <code>string</code> (<code>IsNullOrWhiteSpace</code> , <code>Contains</code> , <code>Length</code>)
Depende de	nada. Este método não chama ninguém: só recebe um texto e responde. Pode ser escrito com o MySQL desligado e até com o banco ainda não criado

O que este método faz

Ele responde a uma única pergunta: **este texto serve como nome de usuário?** Recebe uma `string` (o que a pessoa digitou na caixa de usuário) e devolve `true` ou `false`. Nada mais. Ele não mostra mensagem, não grava, não consulta o banco, não limpa caixa de texto.

São três condições, e as três precisam valer ao mesmo tempo para a resposta ser `true`: o texto tem conteúdo de verdade (não é nulo, não é vazio, não é só espaços), não tem espaço no meio, e tem `TAMANHO_MINIMO_LOGIN` caracteres ou mais. Basta uma falhar para a resposta ser `false`.

Quem chama este método é a tela de cadastro, no `btnCadastrar_Click` (TODO 6), na segunda recusa da fila:

```
if (Regras.ValidarLogin(txtUsuario.Text) == false)
```

Você vai escrever o TODO 3 antes do TODO 6. Ou seja: por um tempo este método vai existir sem ninguém chamando ele. Isso é normal e proposital - a regra fica pronta primeiro, a tela usa depois.

Por que ele existe assim

Por que ele mora no Regras.cs e não dentro do botão. A regra "usuário não pode ter espaço e precisa de 3 caracteres" é uma decisão do *sistema*, não da *tela*. Se ela morasse dentro do `btnCadastrar_Click`, no dia em que aparecesse uma segunda tela que também cria usuário (ou uma tela de "alterar usuário"), a regra teria que ser copiada. Regra copiada é regra que vai ficar diferente de um lado para o outro - alguém corrige um dos dois e esquece o outro. Aqui ela existe em um lugar só, e as telas perguntam.

Por que ele devolve `bool` e não mostra a mensagem ele mesmo. Repare no cabeçalho do arquivo: o *Regras.cs* não abre banco e não desenha tela. Se você colocasse um `MessageBox.Show` aqui dentro, precisaria de `using System.Windows.Forms` no *Regras.cs*, e a partir daí a regra só funcionaria dentro de um programa com janelas. Devolvendo `bool`, ele funciona em qualquer lugar - inclusive no seu teste de mesa, com o MySQL desligado. E há uma segunda razão, mais prática: o **texto** da mensagem pertence à tela, porque é o texto que vai para o manual (I6). A regra decide *se* recusa; a tela decide *como* avisa.

Por que `static`. `static` quer dizer que o método pertence à classe, e não a um objeto dela. Você chama escrevendo `Regras.ValidarLogin("ana")`, direto - não precisa de `Usuario u = new Usuario()` como você faz com o `Usuario`. Faz sentido porque *Regras* não *guarda* nada: não existe "a minha regra" e "a sua regra", existe a

regra. Uma classe sem dados para guardar não precisa virar objeto. (No esqueleto o `static` já vem escrito na assinatura - você não vai digitar isso, mas vai precisar explicar na arguição.)

Por que três `if` separados e não uma condição gigante. Daria para escrever tudo em uma linha só com `||`. Não está errado, mas você perde duas coisas. A primeira: com três `if`, cada regra tem a sua própria linha para você colocar um **breakpoint** e ver, no `F10`, exatamente qual das três barrou o texto. A segunda: no dia em que o professor pedir mensagens diferentes ("tem espaço" x "é curto demais"), três `if` separados já estão no formato certo para isso; a condição gigante teria que ser desmontada antes.

O código

```
public static bool ValidarLogin(string login)
{
    if (string.IsNullOrEmpty(login))
    {
        return false;
    }

    if (login.Contains(" "))
    {
        return false;
    }

    return login.Length >= TAMANHO_MINIMO_LOGIN;
}
```

Linha a linha

Linha	Por que ela esta ai
<code>public static bool ValidarLogin(string login)</code>	O tipo de retorno <code>bool</code> e o contrato: quem chamar recebe sim ou nao, e decide o que fazer com isso. O parametro se chama <code>login</code> e nao <code>txtUsuario</code> de proposito - aqui dentro nao existe caixa de texto, existe um texto qualquer que veio de algum lugar.
<code>if (string.IsNullOrEmpty(login))</code>	Um metodo pronto do C# que responde tres perguntas de uma vez: o texto e nulo? e vazio? e so espacos? Se qualquer uma for sim, ele devolve <code>true</code> . Repare que ele responde ao contrario do que voce quer: ele diz que esta ruim. Por isso o <code>if</code> sem <code>== false</code> e o <code>return false</code> dentro.
— a ordem deste <code>if</code>	Ele tem que ser o primeiro . Se <code>login</code> for nulo (nulo e "nem texto tem", diferente de texto vazio), qualquer <code>login.Alguma coisa</code> explode na hora. Este <code>if</code> corta o nulo fora antes de as duas linhas seguintes encostarem no texto. Trocar a ordem dos tres blocos nao e questao de gosto: quebra o programa.
<code>return false;</code> (dentro do <code>if</code>)	Saida antecipada. Assim que uma regra falha, o metodo acaba - as linhas de baixo nem sao lidas. E o mesmo molde de recusa que voce vai usar na tela: descobriu que esta errado, avisa e sai.
<code>if (login.Contains(" "))</code>	<code>Contains</code> procura um pedaco dentro do texto e devolve <code>true</code> se achar. Entre as aspas ha um espaco : <code>" "</code> . Nao e <code>""</code> (vazio) nem duas aspas coladas. O objetivo e barrar <code>"joao silva"</code> como login, porque login com espaco confunde na hora de digitar e de comparar.
<code>return login.Length >= TAMANHO_MINIMO_LOGIN;</code>	Aqui nao ha <code>if</code> . A comparacao <code>>=</code> ja produz um <code>bool</code> - <code>true</code> ou <code>false</code> -, e e exatamente isso que o metodo precisa devolver. Escrever <code>if (...) return true; else return false;</code> daria no mesmo resultado com o dobro de linhas.
<code>>=</code> e nao <code>></code>	O minimo e 3, e "3 caracteres" precisa ser aceito. Com <code>></code> voce estaria exigindo 4, e o login <code>"ana"</code> seria recusado sem motivo.
<code>TAMANHO_MINIMO_LOGIN</code> e nao <code>3</code>	A constante esta declarada no alto do arquivo (<code>public const int TAMANHO_MINIMO_LOGIN = 3;</code>) e ja vem pronta. <code>const</code> e um valor que ganha nome e nunca muda durante a execucao. Usar o nome tem duas vantagens: quem le a linha entende o que o 3 significa, e a mensagem da tela de cadastro monta o texto com essa mesma constante (<code>"O usuario nao pode ter espaco e precisa de " + Regras.TAMANHO_MINIMO_LOGIN + " caracteres ou mais."</code>). Trocando o 3 por 5 no alto do arquivo, a regra e a mensagem mudam juntas. Se voce cravar o <code>3</code> aqui, no dia da mudanca a tela vai avisar 5 e o programa vai continuar aceitando 3.

Linha	Por que ela esta ai
a linha <code>return true;</code> do esqueleto	Ela existe so para o projeto compilar antes de voce escrever o metodo. Ela vai embora - as suas linhas ficam no lugar dela, nao embaixo dela.

O erro mais comum

- 1. Inverter a resposta.** Escrever `return true;` dentro dos `if`. O metodo passa a dizer "serve" justamente para os textos que nao servem. Sintoma: voce digita `"ab"` no cadastro e o sistema aceita numa boa; digita `"ana"` (que e valido) e leva a mensagem de recusa. Se o comportamento parecer o oposto do esperado, e quase sempre isso.
- 2. Esquecer de apagar o `return true;` do esqueleto.** Se ele ficar como primeira linha do metodo, **nada** abaixo dele roda: qualquer texto e aceito, ate espaco em branco. Isso nao e erro de compilacao - o projeto roda assim mesmo. O que o Visual Studio faz e avisar que existe *codigo inalcançavel* e deixar as suas linhas **esmaecidas** na tela: essa e a pista. Sintoma: o campo usuario aceita tudo. Confira se a primeira linha do metodo e o `if` do `IsNullOrWhiteSpace`.
- 3. Escrever `Contains("")` em vez de `Contains(" ")`.** Texto vazio esta contido em qualquer texto, entao `Contains("")` devolve `true` sempre. Sintoma: **todo** login e recusado, inclusive `"ana"`. Aumente o zoom ou clique entre as aspas e confira se o cursor anda uma casa - se ele nao andar, nao ha espaco ali.
- 4. Trocar o `IsNullOrWhiteSpace` por `login == ""`.** Parece a mesma coisa e nao e. `" "` (tres espacos) nao e igual a `""`, e tem `Length` igual a 3 - ou seja, passaria pelas outras duas condicoes se o `Contains` nao o pegasse. Alem disso, `login == ""` nao protege contra nulo. Use o metodo pronto: ele cobre os tres casos de uma vez.
- 5. Colocar as linhas na ordem errada, com o `Contains` antes do `IsNullOrWhiteSpace`.** Enquanto os textos vierem da caixa de texto da tela, isso nao aparece, porque uma caixa de texto vazia entrega `""` e nunca nulo. Mas se alguem chamar o metodo passando nulo, o programa para com a janela de execucao nao tratada (`System.NullReferenceException` - "referencia de objeto nao definida"). A guarda contra nulo vem primeiro sempre.
- 6. Colocar `MessageBox.Show` aqui dentro.** O projeto ate compila se voce adicionar o `using`, mas voce quebrou a regra do quadro e a arguicao vai bater nisso: a mesma regra passa a mostrar janela mesmo quando quem chamou nao queria janela nenhuma.

Como saber que funcionou

Nao procure no Workbench. Este metodo nao encosta no banco - nao existe consulta SQL que prove que ele esta certo. A prova e o comportamento da tela.

Enquanto o TODO 6 nao existir, a conferencia e no papel. No esqueleto, o `btnCadastrar_Click` inteiro e a linha `MessageBox.Show("TODO 6 ainda nao foi escrito.");`. Ou seja: ninguem chama o `ValidarLogin` ainda. Nao adianta por breakpoint dentro dele e clicar em Cadastrar - a execucao nao passa por ali, a bolinha nao pega, e voce vai achar que errou o codigo quando o codigo esta certo. O que da para fazer agora e o **teste de mesa**: pegue cada texto e passe pelas tres condicoes com o dedo, na ordem em que elas estao escritas.

Texto	IsNullOrWhiteSpace	Contains(" ")	Length >= 3	Resposta
" "	true → sai aqui	—	—	false
" "	true → sai aqui	—	—	false
"ab"	false	false	2 >= 3 e false	false
"jo ao"	false	true → sai aqui	—	false
"ana"	false	false	3 >= 3 e true	true

Se o seu código devolve essas cinco respostas quando você lê linha por linha, ele está certo. O resto é conferir na tela depois.

Teste completo (depois que o TODO 6 estiver escrito). Rode o projeto, abra a tela de cadastro pelo botão "Criar conta" e preencha o nome e as duas senhas com algo válido (`Maria Souza`, senha `1234` nos dois campos), mudando só o campo usuário:

Você digita no campo usuário	O que tem que acontecer
<code>ab</code>	aparece a mensagem "O usuário não pode ter espaço e precisa de 3 caracteres ou mais."
<code>jo ao</code>	a mesma mensagem (foi o espaço que barrou)
<code>ana</code>	a mensagem acima nao aparece. No lugar dela vem "Este usuário já está em uso. Escolha outro." - porque <code>ana</code> é uma das três contas que o script do banco já criou. Essa troca de mensagem é a prova de que o seu <code>ValidarLogin</code> aprovou o <code>ana</code> e passou a vez para a recusa seguinte
<code>maria</code>	nenhuma mensagem de recusa: aparece "Conta criada! Agora faça o login." (se você repetir o teste, <code>maria</code> já vai existir e a resposta passa a ser a mesma do <code>ana</code>)

Repare que deixar o campo em branco não testa o TODO 3: a recusa M1, que vem antes, pega o campo vazio e o `ValidarLogin` nem chega a ser chamado.

O mesmo teste com breakpoint, para ver por dentro. Com a linha do M2 já escrita no TODO 6, clique na barra cinza à esquerda da primeira linha do `ValidarLogin` para criar o breakpoint (a bolinha vermelha), rode com `F5`, preencha o cadastro com `jo ao` no usuário e clique em Cadastrar. Quando a execução parar na bolinha, vá apertando `F10` linha por linha. O amarelo vai mostrar por qual `if` ele passou e em qual `return` ele saiu. Passe o mouse sobre `login` para ver o texto que chegou - inclusive os espaços que você não tinha notado que digitou.

Para pensar

Se a mensagem "O usuário não pode ter espaço e precisa de 3 caracteres ou mais" existe por causa desta regra, por que o texto dela está escrito na tela de cadastro e não aqui dentro do método que descobriu o problema?

TODO 4 - LoginExiste

Arquivo	Dados/UsuarioDAO.cs
Indicador	I5
O que voce exercita	<code>try/finally</code> , <code>new MySqlCommand(sql, con)</code> com os dois argumentos, <code>@parametro</code> com <code>AddWithValue</code> , <code>ExecuteScalar</code> , <code>Convert.ToInt32</code> , comparacao que ja devolve <code>bool</code>
Depende de	Para escrever , nenhum outro TODO. Para testar pela tela , o TODO 6 - o botao Cadastrar e quem chama este metodo, e voce so escreve o 6 dois TODOs adiante. Antes de qualquer coisa, o <code>Conexao.cs</code> precisa estar com a senha certa e o banco <code>portariadb</code> precisa existir: e a faixa verde da tela de login que mostra isso. Sem banco, este metodo nao tem onde perguntar.

O que este método faz

Ele recebe um texto - o login que a pessoa acabou de digitar na tela de cadastro - e devolve `true` se esse login ja pertence a alguma conta gravada no banco, ou `false` se ele ainda esta livre. Nada mais. Ele nao grava, nao apaga, nao mostra mensagem: so responde uma pergunta de sim ou nao.

Quem chama ele e o `frmCadastro.btnCadastrar_Click` - o **TODO 6**, que voce escreve daqui a dois TODOs -, na quinta e ultima conferencia antes de gravar (a mensagem M5, "Este usuario ja esta em uso. Escolha outro."). A tela pergunta, recebe a resposta, e decide o que fazer com ela.

Este e o primeiro metodo do projeto que **manda uma consulta SQL** para o MySQL. O TODO 1 so pergunta se o banco atende; este aqui e o primeiro que pergunta alguma coisa **a ele**. Dai o indicador I5 comecar aqui.

Por que ele existe assim

Por que ele mora no `UsuarioDAO` e nao no `frmCadastro`. DAO quer dizer *Data Access Object*: a classe que sabe conversar com o banco. A separacao do sistema inteiro e essa - a tela sabe o que perguntar, o DAO sabe como perguntar. Se voce escrevesse o `SELECT COUNT` dentro do clique do botao, o dia em que outra tela precisasse da mesma pergunta voce teria duas copias do mesmo SQL, e uma delas ia ficar desatualizada. Aqui existe uma copia so.

Por que a assinatura recebe `string login` e nao le a caixa de texto. O `UsuarioDAO` nao enxerga a tela: ele nao sabe o que e um `TextBox`, nem quer saber. Quem sabe que o login esta no `txtUsuario.Text` e o `frmCadastro`, e e ele quem entrega o texto ja pronto. Isso e o que permite chamar o mesmo metodo de qualquer lugar depois.

Por que devolve `bool` e nao o `Usuario` encontrado. A tela so precisa decidir entre continuar e recusar. Devolver a conta inteira entregaria a senha de outra pessoa para uma tela que nao tem nada que fazer com ela. Cada metodo devolve o minimo que resolve o problema de quem chamou.

Por que `static`. `static` significa que o metodo pertence a classe, nao a um objeto dela: voce escreve `UsuarioDAO.LoginExiste("ana")` direto, sem precisar de `new UsuarioDAO()` antes. Faz sentido porque esta classe nao guarda nada entre uma chamada e outra - e o mesmo motivo de voce escrever `Convert.ToInt32(...)` sem nunca ter criado um objeto `Convert`.

Por que perguntar ao banco em vez de confiar so no C#. Aqui esta a parte que costuma cair na arguicao. A tabela `Usuario` tem `CONSTRAINT UQ_Usuario_Login UNIQUE (Login)`: e o banco que garante, de verdade, que nao vao existir dois logins iguais. Se este metodo nao existisse, o sistema ainda assim nao aceitaria login repetido - so que o aluno veria uma tela de erro feia do MySQL em ingles em vez de "Este usuario ja esta em uso". **●** `LoginExiste` e a mensagem bonita; o `UNIQUE` e a garantia. Sao duas coisas diferentes e as duas precisam existir.

O código

```
public static bool LoginExiste(string login)
{
    MySqlConnection con = Conexao.Obter();

    try
    {
        con.Open();

        string sql = @"SELECT COUNT(*)
                        FROM Usuario
                        WHERE Login = @login";

        MySqlCommand cmd = new MySqlCommand(sql, con);
        cmd.Parameters.AddWithValue("@login", login);

        int quantos = Convert.ToInt32(cmd.ExecuteScalar());

        return quantos > 0;
    }
    finally { con.Close(); }
}
```

Linha a linha

Linha	Por que ela esta ai
<pre>MySqlConnection con = Conexao.Obter();</pre>	Pega a conexao pronta, com endereco, banco, usuario e senha ja preenchidos. Nenhum DAO deste projeto escreve a string de conexao: se a senha do seu MySQL mudar, voce troca em um lugar so, no <code>Conexao.cs</code>. O <code>Obter()</code> devolve a conexao fechada - quem abre e quem chama.
A linha do <code>con</code> fica fora do <code>try</code>	Aqui tem uma razao que nao e estilo: uma variavel criada dentro do <code>try</code> nao existe no <code>finally</code>. Como o <code>finally</code> precisa chamar <code>con.Close()</code>, o <code>con</code> tem que nascer antes de o <code>try</code> comecar. Se voce mover essa linha para dentro das chaves, o projeto nem compila.
<code>try</code> sem <code>catch</code>	Voce viu <code>try/catch/finally</code> na aula de execucao. Aqui sao so o <code>try</code> e o <code>finally</code>, e isso e proposital: este metodo nao decide o que dizer ao usuario. Se voce colocasse um <code>catch</code> devolvendo <code>false</code>, um banco fora do ar viraria "este login esta livre" e o cadastro seguiria em frente mentindo. Quem cuida da conversa com o usuario sobre banco fora do ar e a tela de login, com o <code>Conexao.TestarConexao</code> e a faixa verde/vermelha.
<pre>con.Open();</pre>	Abre a conexao. Perceba que ela abre dentro do <code>try</code>: se falhar exatamente aqui, o <code>finally</code> ainda roda e chama <code>Close()</code> numa conexao que nunca abriu - e isso nao da erro nenhum. Fechar o que ja esta fechado e inofensivo; deixar aberto o que abriu e que da problema.
<pre>string sql = @"..."</pre>	O <code>@</code> colado na abertura das aspas cria um texto que pode ocupar varias linhas. Sem ele voce teria que espremer o <code>SELECT</code> inteiro numa linha so ou emendar pedacos com <code>+</code>. Cuidado com a coincidencia: este <code>@</code> (de texto multilinha) nao tem nada a ver com o <code>@login</code> la dentro. Sao dois usos diferentes do mesmo simbolo.
<pre>SELECT COUNT(*)</pre>	<code>COUNT(*)</code> conta as linhas que passaram pelo <code>WHERE</code> e devolve um numero so. Nao traz nome, nem senha, nem data - so a contagem. Voce nao precisa da conta, precisa saber se ela existe; entao nao ha motivo para trazer o resto pela rede.
<pre>WHERE Login = @login</pre>	O <code>@login</code> e um espaco reservado. Voce nao cola o texto digitado dentro do SQL. Se voce escrevesse <code>"... WHERE Login = '" + login + "'"</code>, alguem digitando <code>'</code> OR <code>'1'='1</code> no campo usuario faria o proprio texto virar comando, e a consulta passaria a responder outra coisa. Com <code>@login</code>, o que a pessoa digitou e tratado como valor, nunca como comando - mesmo que ela digite aspas, ponto e virgula ou um <code>DROP TABLE</code> inteiro.
<pre>MySqlCommand cmd = new MySqlCommand(sql, con);</pre>	O comando precisa saber duas coisas: o que executar (o <code>sql</code>) e por onde (a <code>con</code>). Por isso os dois entram juntos no <code>new</code>. Um comando sem conexao nao tem para onde ir.

Linha	Por que ela esta ai
<code>cmd.Parameters.AddWithValue("@login", login);</code>	Aqui o <code>@login</code> deixa de ser espaco reservado e recebe o valor. Escreva o nome exatamente como esta no SQL: se voce mandar um nome diferente, o comando chega ao banco com um buraco que ninguem preencheu e o programa para dizendo que o <code>@login</code> precisa ser definido. O segundo argumento e o parametro do metodo - o texto que a tela entregou.
<code>cmd.ExecuteScalar()</code>	Existem tres executores e cada um serve a um tipo de resultado: <code>ExecuteReader</code> para quando voltam linhas, <code>ExecuteNonQuery</code> para INSERT/UPDATE/DELETE, e <code>ExecuteScalar</code> para quando volta um unico valor - exatamente o caso do <code>COUNT</code> . Usar o <code>ExecuteReader</code> aqui funcionaria, mas seria montar um leitor de linhas para ler um numero.
<code>Convert.ToInt32(...)</code> em volta	O <code>ExecuteScalar</code> devolve <code>object</code> , o tipo mais geral que existe, porque ele nao tem como saber de antemao se o valor unico e um numero, um texto ou uma data. O <code>Convert.ToInt32</code> transforma esse <code>object</code> em <code>int</code> . Nao troque por <code>(int)cmd.ExecuteScalar()</code> - o motivo esta no erro mais comum, logo abaixo.
<code>int quantos = ...</code>	Guardar em uma variavel com nome nao muda o resultado, mas muda a depuracao: no breakpoint voce passa o mouse em <code>quantos</code> e ve <code>0</code> ou <code>1</code> . Se tudo estivesse numa linha so, voce nao teria onde olhar.
<code>return quantos > 0;</code>	<code>quantos > 0</code> ja e um <code>bool</code> - vale <code>true</code> ou <code>false</code> por si so. Nao existe motivo para escrever <code>if (quantos > 0) return true; else return false;</code> . E <code>> 0</code> em vez de <code>== 1</code> de proposito: se por acidente existirem duas linhas com o mesmo login, a resposta certa continua sendo "esta em uso".
<code>finally { con.Close(); }</code>	O <code>finally</code> roda sempre : quando o <code>try</code> termina bem, quando ele quebra no meio, e - o caso daqui - quando ele sai por um <code>return</code> . O <code>return quantos > 0;</code> primeiro calcula o valor, depois o <code>finally</code> fecha a conexao, e so entao o valor sai do metodo. E por isso que ter o <code>return</code> dentro do <code>try</code> nao deixa a conexao aberta. Sem o <code>finally</code> , cada consulta que desse errado deixaria uma conexao pendurada, e depois de algumas o MySQL para de aceitar novas.

O erro mais comum

1. Trocar `Convert.ToInt32` por um cast `(int)`. Este e o campeao. O `COUNT(*)` do MySQL nao volta como `int` de 32 bits, e sim como um numero de 64 bits (`Int64`). O cast `(int)` nao converte nada - ele so afirma "isto ja e um int", e como nao e, o programa para com:

```
System.InvalidCastException: Unable to cast object of type 'System.Int64' to type 'System.Int32'.
```

O `Convert.ToInt32` **converte de verdade**, e por isso funciona. Sintoma: o programa quebra na hora de clicar em Cadastrar, com a caixa amarela do Visual Studio apontando essa linha.

2. Montar o SQL com `+` em vez de `@parametro`. Sintoma: parece funcionar com logins normais e quebra com nomes que tem apostrofo. Alem de ser a falha de segurança que a regra 1 do arquivo existe para evitar - e a que a arguicao pergunta.

3. Escrever no `AddWithValue` um nome diferente do que esta no SQL - `@login` no SQL e `"@usuario"` no `AddWithValue`, ou um erro de digitacao como `"@login"`. Sintoma: o programa quebra na linha do `ExecuteScalar` com a mensagem do MySQL:

```
Parameter '@login' must be defined.
```

Ou seja: o comando chegou ao banco com um buraco que ninguem preencheu. Confira que o nome e o mesmo nos dois lugares, letra por letra.

4. Criar o `con` dentro do `try`. Sintoma: erro de compilacao, CS0103 - o nome 'con' nao existe no contexto atual, apontando o `finally`.

5. Esquecer o `con.Open()`. Sintoma: o programa para na linha do `ExecuteScalar` com `The connection is not open.` O `Conexao.Objter()` entrega a conexao fechada de proposito - abrir e tarefa de quem usa.

6. Inverter a resposta (`return quantos == 0;`). Sintoma cruel, porque nada quebra: o sistema passa a recusar todo login novo e a aceitar exatamente os que ja existem.

Como saber que funcionou

Este metodo ainda **nao tem quem o chame**. Quem chama e o botao **Cadastrar** da tela de cadastro, que e o **TODO 6** - dois TODOs adiante. Enquanto o 6 nao estiver escrito, esse botao so mostra a caixa "TODO 6 ainda nao foi escrito.". Entao a conferencia acontece em dois momentos.

Agora, assim que voce terminar de escrever o metodo.

1. Compile o projeto (Ctrl + Shift + B). Tem que compilar **sem nenhum erro**. Dois dos enganicos da lista acima - o `con` criado dentro do `try` e o nome de variavel trocado - sao erros de compilacao, e e este passo que pega os dois.
2. Abra o programa e confirme que a faixa da tela de login (o `lblStatus`) esta **verde**. Ela e escrita pelo `frmLogin_Shown`, que ja vem pronto no esqueleto, e prova que o `Conexao.cs` esta com a senha certa e que o `portariadb` existe. Se estiver vermelha, arrume a conexao antes de seguir - o problema nao e este metodo. O TODO 1, o botao **Testar conexao**, serve exatamente para reconferir isso sem fechar e abrir o programa.
3. Rode no Workbench a mesma pergunta que o metodo faz:

```
SELECT COUNT(*) FROM portariadb.Usuario WHERE Login = 'ana';
```

Tem que dar `1`. Troque para `'ana2'` e tem que dar `0`. (O `portariadb.` na frente da tabela evita o `Error 1046: No database selected`, que aparece quando o Workbench nao esta com o banco selecionado.)

Guarde essa resposta: se mais adiante o programa se comportar ao contrario do que o Workbench respondeu, o erro esta no `return`, nao no SQL.

Depois do TODO 6, quando a tela ja chamar este metodo. Este e o teste que fecha o TODO 4 - deixe ele anotado.

1. Clique em **Criar conta**.
2. Preencha Nome `Teste Um`, Usuario `ana`, Senha `1234`, Confirmar `1234`. O login `ana` ja veio no `CriarBanco.sql`.
3. Clique em **Cadastrar**.

Tem que aparecer "**Este usuario ja esta em uso. Escolha outro.**" e a tela tem que continuar aberta, sem gravar nada.

1. Agora troque o usuario para `ana2` e clique em Cadastrar de novo. Dessa vez a recusa **nao** pode aparecer - e e so isso que voce esta testando aqui. Se o TODO 5 ainda nao estiver escrito, o sistema vai mostrar "Conta criada! Agora faca o login." e fechar a janela **mesmo sem ter gravado nada**: o `CriarConta` do esqueleto ainda devolve zero e nao faz INSERT nenhum. Nao se assuste - a gravacao e o proximo TODO.

Com breakpoint (tambem depois do TODO 6). Coloque um breakpoint na linha do `int quantos = ...`, rode com F5 e faca o teste do passo 2. Quando o programa parar, pressione F10 uma vez e passe o mouse sobre `quantos`. Com o login `ana` ele tem que valer `1`.

Para pensar

A tabela `Usuario` ja tem `UNIQUE (Login)`, e o banco recusa sozinho um login repetido mesmo que este metodo nao exista. Entao por que o sistema faz a pergunta **antes** de tentar gravar, em vez de simplesmente tentar o INSERT e lidar com a recusa quando ela vier?

TODO 5 - UsuarioDAO.CriarConta

Arquivo	Dados/UsuarioDAO.cs
Indicador	I5
O que você exercita	<code>INSERT</code> com as colunas nomeadas, <code>LAST_INSERT_ID()</code> , três <code>@parametro</code> , <code>ExecuteScalar</code> , <code>new MySqlCommand</code> , <code>try/finally</code> , objeto passado como parâmetro de método
Depende de	O <code>CriarBanco.sql</code> já executado no Workbench e o <code>pwd</code> do seu MySQL já ajustado no <code>Conexao.cs</code> . Também do TODO 2 (a propriedade <code>Nome</code>), para o nome entrar limpo no banco. Não depende do TODO 4.

O que este método faz

Ele recebe um objeto `Usuario` já preenchido — nome, login e senha, os três vindos das caixas de texto da tela de cadastro — e grava uma linha nova na tabela `Usuario`. Em seguida pergunta ao banco qual foi o número que ele gerou para essa linha, guarda esse número dentro do próprio objeto que recebeu (em `u.Id`) e devolve o mesmo número para quem chamou.

Quem chama é o **TODO 6**, o `btnCadastrar_Click` do `frmCadastro`, e só no fim do método, depois que as cinco recusas daquela tela já deixaram passar: campos preenchidos, login válido, senha com tamanho mínimo, senhas iguais e login ainda livre. Só então a tela monta o objeto e chama o `CriarConta`; depois da chamada ela ainda mostra o aviso de conta criada e se fecha. Quando o `CriarConta` é chamado, a decisão de cadastrar **já foi tomada**. Ele não valida nada, não mostra nada na tela e não pergunta nada: ele grava.

Por que ele existe assim

Por que ele mora no `UsuarioDAO` e não no `frmCadastro`. Este é o único lugar do código C# onde a palavra `INSERT` aparece. Se a tela montasse o SQL sozinha, cada tela nova que gravasse usuário teria a sua própria cópia do comando — e no dia em que a tabela ganhar uma coluna, você teria que caçar todas. É a mesma ideia que faz o `Conexao.cs` guardar o endereço do banco num lugar só e o `Regras.cs` não abrir conexão nenhuma: **cada assunto tem um endereço**. Tela desenha, `Regras` decide, `DAO` conversa com o banco.

Por que a assinatura recebe `Usuario u` e não três strings. Um método `CriarConta(string nome, string login, string senha)` funcionaria — e seria uma armadilha. Os três são `string`. Se alguém chamasse na ordem errada, trocando login com senha, o projeto **compilaria sem um aviso sequer** e gravaria a conta invertida; você só descobriria na hora de entrar no sistema. Passando o objeto, cada valor viaja com o nome dele grudado: `u.Login` é o login, `u.Senha` é a senha, não há ordem para errar. E há um segundo ganho: se amanhã a conta ganhar um telefone, o objeto carrega o telefone junto e **a assinatura do método não muda** — quem chama continua chamando igual.

Por que ele devolve `int` e não `void`. O `Id` não existe antes da gravação. O objeto que a tela montou tem nome, login e senha, mas o `Id` dele ainda é zero — quem inventa esse número é o banco, no momento em que a linha entra na tabela. Um método `void` gravaria a conta e jogaria fora a única informação que **só ele** podia saber. Devolver o `Id` é o jeito de o método contar o que aconteceu.

Por que três colunas e não cinco. A tabela tem cinco colunas, mas você manda três. O `Id` é `AUTO_INCREMENT`: o banco escolhe o próximo número sozinho. A `DataCadastro` é `DEFAULT CURRENT_TIMESTAMP`: quando o `INSERT`

não menciona a coluna, o MySQL carimba a hora dele. Isso não é preguiça de digitar dois parâmetros — é uma decisão. **A hora que vale é a do servidor**, não a do relógio da máquina onde o programa está rodando. Um computador do laboratório com a data errada estragaria a ordenação "mais recentes primeiro" do TODO 10 se a data viesse do C#.

Por que `static`. Não há nada para guardar entre uma chamada e outra: o método recebe tudo de que precisa pelo parâmetro e devolve tudo pelo `return`. Sendo `static`, a tela escreve `UsuarioDAO.CriarConta(novo)` direto, sem precisar criar um objeto `UsuarioDAO` antes só para poder usá-lo.

O código

```
public static int CriarConta(Usuario u)
{
    MySqlConnection con = Conexao.Obter();

    try
    {
        con.Open();

        string sql = @"INSERT INTO Usuario (Nome, Login, Senha)
                        VALUES (@nome, @login, @senha);
                        SELECT LAST_INSERT_ID()";

        MySqlCommand cmd = new MySqlCommand(sql, con);
        cmd.Parameters.AddWithValue("@nome", u.Nome);
        cmd.Parameters.AddWithValue("@login", u.Login);
        cmd.Parameters.AddWithValue("@senha", u.Senha);

        u.Id = Convert.ToInt32(cmd.ExecuteScalar());
        return u.Id;
    }
    finally { con.Close(); }
}
```

Linha a linha

Linha	Por que ela está aí
<pre>public static int CriarConta(Usuario u)</pre>	<code>int</code> porque o <code>Id</code> novo precisa voltar; <code>Usuario u</code> porque um objeto não deixa trocar login com senha; <code>static</code> porque não há estado a guardar entre chamadas.
<pre>MySqlConnection con = Conexao.Obter();</pre>	Fica fora do <code>try</code> , e isso é obrigatório: a variável <code>con</code> precisa existir para o <code>finally</code> poder fechá-la. Declarada dentro do <code>try</code> , ela morreria antes de o <code>finally</code> rodar e o projeto nem compilaria.
	O <code>Obter()</code> devolve a conexão fechada — ele só monta o objeto com o endereço do banco. Quem abre é quem usa.
<pre>try</pre>	Tudo que pode dar errado começa aqui. O banco pode estar desligado, a senha do <code>pwd</code> pode estar errada, a rede pode cair no meio.
<pre>con.Open();</pre>	Primeira linha de dentro, porque é a primeira que pode falhar. Sem ela, o <code>ExecuteScalar</code> lá embaixo reclama que a conexão não está aberta.
<pre>string sql = @"...</pre>	O <code>@</code> antes das aspas é do C#: ele permite que o texto continue nas linhas de baixo. Sem esse <code>@</code> , o compilador acusa string não terminada na primeira quebra de linha. Ele não tem nada a ver com o <code>@</code> de <code>@nome</code> .
<pre>INSERT INTO Usuario (Nome, Login, Senha)</pre>	As colunas são nomeadas . Sem essa lista entre parênteses, o MySQL espera valores para as cinco colunas, na ordem exata da tabela — e qualquer coluna nova quebraria este comando.
<pre>VALUES (@nome, @login, @senha);</pre>	Aqui o <code>@</code> é do MySQL : é um marcador de lugar. O texto que o usuário digitou nunca entra no comando; ele é entregue à parte, e o banco o trata como valor , jamais como instrução.
	É por isso que não se cola com <code>+</code> . Um nome como <code>D'Avila</code> tem uma aspa simples no meio: montado com <code>+</code> , ele fecha a aspa do SQL no lugar errado e o comando quebra. E o que quebra com um apóstrofo aceita, do mesmo jeito, um <code>;</code> <code>DELETE FROM Usuario;</code> digitado de propósito. Com <code>@parametro</code> , os dois casos são só texto.
<pre>SELECT LAST_INSERT_ID();</pre>	Dois comandos num texto só, separados pelo <code>;</code> . <code>LAST_INSERT_ID()</code> é a função do MySQL que devolve o último número gerado por <code>AUTO_INCREMENT</code> nesta conexão — não no banco inteiro. Se dois colegas cadastrarem ao mesmo tempo, cada um recebe o <code>Id</code> da sua própria linha.
	Os dois comandos viajam juntos, na mesma ida ao banco e na mesma conexão. Essa é a razão de o <code>SELECT</code> estar aqui dentro e não numa consulta separada depois: numa segunda conexão, <code>LAST_INSERT_ID()</code> devolveria zero.

Linha	Por que ela está aí
<pre>MySqlCommand cmd = new MySqlCommand(sql, con);</pre>	O comando precisa das duas coisas: o que executar (<code>sql</code>) e por onde mandar (<code>con</code>).
<pre>cmd.Parameters.AddWithValue("@nome", u.Nome);</pre>	Uma chamada para cada marcador do SQL. O texto entre aspas tem que ser igual ao que está lá em cima, com o <code>@</code> incluído — é ele que liga o valor ao lugar.
<pre>cmd.Parameters.AddWithValue("@login", u.Login);</pre>	Repare que aqui entra <code>u.Login</code> , a propriedade do objeto , e não <code>txtUsuario.Text</code> . Este arquivo não conhece caixa de texto nenhuma; se conhecesse, ele deixaria de ser DAO.
<pre>cmd.Parameters.AddWithValue("@senha", u.Senha);</pre>	Três parâmetros, três marcadores. Nem um a mais: não existe <code>@data</code> porque não existe <code>@data</code> no SQL.
<pre>u.Id = Convert.ToInt32(cmd.ExecuteScalar());</pre>	<code>ExecuteScalar</code> é o que se usa quando o comando devolve um valor só — aqui, o número do <code>SELECT LAST_INSERT_ID()</code> . Para <code>INSERT</code> sozinho seria <code>ExecuteNonQuery</code> ; para um <code>SELECT</code> de várias linhas, <code>ExecuteReader</code> .
	<code>ExecuteScalar</code> devolve <code>object</code> , o tipo mais genérico que existe, porque ele não sabe de antemão se o valor é número, texto ou data. O <code>Convert.ToInt32</code> faz a tradução. Escrever <code>(int)</code> na frente não resolve: o <code>LAST_INSERT_ID()</code> volta empacotado como um número de outro tipo e a conversão direta quebra em execução.
	A atribuição a <code>u.Id</code> é o detalhe que a maioria pula. O objeto que a tela criou está em memória, na mão de quem chamou ; preenchendo o <code>Id</code> aqui, ele sai desta chamada completo, sem precisar de uma consulta ao banco para descobrir o próprio número.
<pre>return u.Id;</pre>	Devolve o mesmo número que acabou de guardar. Duas portas para a mesma informação: quem quiser o valor pega no retorno; quem preferir seguir usando o objeto já o tem preenchido.
<pre>finally { con.Close(); }</pre>	O <code>finally</code> roda sempre : deu certo, deu erro, e mesmo depois do <code>return</code> . Um <code>return</code> não pula o <code>finally</code> — o valor é calculado, o <code>finally</code> executa, e só então o método devolve.
	Sem ele, uma conexão que falhar no meio fica pendurada, aberta, ocupando lugar no servidor. Poucas dezenas dessas e o MySQL passa a recusar conexões novas — e o sintoma aparece longe daqui, numa tela que não tem culpa nenhuma.
	Não há <code>catch</code> neste método, e está certo: fechar a conexão é responsabilidade do DAO; decidir o que dizer ao usuário é da tela. Um <code>try</code> com <code>finally</code> e sem <code>catch</code> é código válido

Linha	Por que ela está aí
	em C#. E <code>Close()</code> numa conexão que nunca chegou a abrir não causa erro nenhum.

O erro mais comum

1. **Esquecer o `SELECT LAST_INSERT_ID();`** e deixar só o `INSERT`. É o campeão, porque não dá erro nenhum: a conta é gravada, a mensagem de sucesso aparece, e o `ExecuteScalar` — sem nada para devolver — retorna vazio. O `Convert.ToInt32` transforma esse vazio em **zero**, e o método devolve zero como se fosse um `Id` legítimo. Sintoma: no Workbench a linha está lá, com o `Id` certo, mas o `u.Id` do programa é 0. Confira colocando um breakpoint na linha do `ExecuteScalar`.
1. **Trocar `ExecuteScalar` por `ExecuteNonQuery`**. O `ExecuteNonQuery` devolve *quantas linhas foram afetadas*, e um `INSERT` de uma linha afeta uma linha. O método passa a devolver **1 sempre** — na primeira conta parece certo, porque o `Id` 1 realmente existe. Da segunda em diante, todo mundo "nasce" com `Id` 1.
1. **Tentar mandar a data.** Há três desfechos, e nenhum deles é o que você quer. Se você acrescentar `DataCadastro` na lista de colunas e esquecer o valor correspondente no `VALUES`, o MySQL responde que a contagem de colunas não bate com a de valores. Se acrescentar o marcador `@data` no `VALUES` e esquecer o `AddWithValue` dele, o programa quebra em execução dizendo que o parâmetro `@data` não foi definido. E se você fizer as duas coisas direito, aí o comando roda — só que a data passa a vir do relógio da máquina onde o programa está, que é exatamente o que este `INSERT` foi escrito para evitar. São **três** parâmetros porque são **três** marcadores. Sempre o mesmo número.
1. **Errar o nome do parâmetro.** Um `@login` no `AddWithValue` com `@login` no SQL derruba o programa com uma exceção dizendo que o parâmetro `@login` não foi definido. O nome é a única ligação entre os dois lugares; leia os dois lado a lado antes de rodar.
1. **Fechar a conexão dentro do `try` em vez do `finally`**. Se o `con.Close()` vier **depois** do `return`, ele nunca executa — o `return` já saiu do método — e o compilador ainda avisa que aquela linha é código inalcançável. Se vier **antes** do `return`, ele executa quando dá tudo certo, mas não executa quando o banco falha no meio, que é justamente a hora em que fechar a conexão importa. O lugar do `Close()` é o `finally`, e é exatamente por isso que o `finally` existe.

Como saber que funcionou

Este método ainda não tem botão que o chame: quem aperta o gatilho é o **TODO 6**, que você escreve logo em seguida. Então escreva os dois e teste juntos.

1. Antes de rodar, abra o Workbench e execute `SELECT Id, Nome, Login, DataCadastro FROM Usuario ORDER BY Id DESC;`. Devem aparecer as **três** contas de exemplo, com `Id` 1, 2 e 3. Anote o maior `Id`.
2. Rode o projeto com **F5**, abra a tela de cadastro e crie uma conta — por exemplo, nome `Carlos Pereira`, usuário `carlos`, senha `1234` nos dois campos.
3. **Feche o programa.** Isso importa: o teste é provar que o dado ficou no banco, não na memória.
4. Execute a mesma consulta de novo no Workbench. Tem que existir uma quarta linha, com `Id` **4**, e a `DataCadastro` preenchida com a hora de agora — hora que **você não digitou em lugar nenhum**. Se essa data está lá, o `DEFAULT CURRENT_TIMESTAMP` fez o trabalho dele e os seus três parâmetros estavam certos.

Para ver o `Id` chegando do banco, faça uma segunda passada: ponha um breakpoint (**F9**) na linha `u.Id = Convert.ToInt32(cmd.ExecuteScalar());`, cadastre outra conta, e quando o programa parar ali aperte **F10** para executar só aquela linha. Passe o mouse sobre `u.Id`: o número que aparecer tem que ser o mesmo que o Workbench mostra na linha nova. Se aparecer **0**, volte ao erro 1 da lista acima.

Não rode o `CriarBanco.sql` de novo para "limpar" a tabela entre os testes — a primeira linha dele apaga o banco inteiro. Para tirar uma conta de teste, use o `DELETE ... WHERE Id = ...` no Workbench.

Para pensar

O método guarda o número em `u.Id` e devolve esse mesmo número no `return`. Por que a mesma informação em dois lugares — o que deixaria de ser possível se ele fizesse apenas uma das duas coisas?

TODO 6 - frmCadastro.btnCadastrar_Click

Arquivo	Telas/frmCadastro.cs
Indicador	I3 · I2 · I6
O que você exercita	cinco <code>if</code> em cadeia, cada um com <code>return;</code> numa condição de três partes; comparação de texto com <code>==</code> e <code>!=</code> ; <code>.Length</code> de string; chamada de método <code>static</code> de outra classe; <code>new Usuario()</code> e atribuição de propriedades; <code>MessageBox.Show</code> com título e ícone; <code>this.Close()</code>
Depende de	TODO 2 (<code>Usuario.Nome</code>), TODO 3 (<code>Regras.ValidarLogin</code>), TODO 4 (<code>UsuarioDAO.LoginExiste</code>), TODO 5 (<code>UsuarioDAO.CriarConta</code>). Escreva os quatro antes deste.

O que este método faz

Este é o método que roda quando o usuário clica no botão **Cadastrar** da tela de criação de conta. Ele recebe dois parâmetros — `sender` e `e` — que você não vai usar em nenhuma linha: eles estão ali porque essa é a forma que o Windows Forms exige de qualquer método ligado a um evento de clique. Quem escreveu essa assinatura foi o Designer, quando o botão foi criado; você só preenche o corpo. O método não devolve nada (`void`), porque não existe ninguém esperando um valor de volta — quem chamou foi o clique do mouse.

Por dentro, ele faz duas coisas em sequência. Primeiro **confere** o que está nas quatro caixas de texto da tela (`txtNome`, `txtUsuario`, `txtSenha`, `txtConfirmar`) e, ao encontrar o primeiro problema, mostra um aviso e para ali mesmo. Se nenhum problema aparecer, ele **grava**: monta um objeto `Usuario` com o que foi digitado, entrega esse objeto ao `UsuarioDAO.CriarConta`, avisa que deu certo e fecha a janela. São cinco conferências, apelidadas de M1 a M5 — e esses cinco apelidos não são decoração: eles são exatamente as cinco primeiras linhas da tabela "Mensagens do sistema" que vai para o seu manual, na seção 6. É por isso que este método carrega o indicador I6 junto com o I3.

Por que ele existe assim

Por que a conferência mora na tela, e não no `Regras.cs` nem no `UsuarioDAO.cs`. Repare no que cada arquivo sabe fazer. O `Regras.ValidarLogin` responde uma pergunta seca: *este texto serve como login?* Ele devolve `true` ou `false` e vai embora — não sabe o que é um `MessageBox`, não sabe que existe uma tela. O `UsuarioDAO.LoginExiste` responde outra pergunta seca: *já tem alguém com esse login no banco?* Também devolve `true` ou `false` e não decide nada.

Nenhum dos dois toma a decisão de **recusar o cadastro e explicar o motivo para uma pessoa**. Essa decisão é sempre da tela, porque a tela é a única parte do sistema que conversa com o usuário. Se você colocasse um

`MessageBox` dentro do `Regras.cs`, quebraria a regra do quadro que está escrita no topo daquele arquivo: o *Regras.cs não abre banco e não desenha tela*. E perderia a coisa mais útil daquele arquivo — o fato de poder ser conferido com o MySQL desligado.

Então a divisão é esta: o `Regras` e o `DAO` **respondem**; a tela **decide e fala**.

Por que cinco `if` separados, cada um com `return`, e não um `if` gigante com `&&`. Você poderia juntar tudo numa condição só. O problema aparece na hora de escrever a mensagem: com uma condição só, você teria uma mensagem só — algo como "dados inválidos" —, e o usuário ficaria sem saber se o problema foi a senha curta ou o login com espaço. Cinco condições separadas dão cinco mensagens específicas, e é isso que o manual precisa.

Também não é um `if / else if / else` encadeado. Cada recusa é um bloco independente que termina em `return`. Esse `return` é o que substitui o `else`: quando ele executa, o método acaba naquela linha, e nada abaixo dele roda. O resultado é que **no máximo uma caixa de mensagem aparece por clique**, e o código fica reto, sem indentação crescendo a cada regra nova. Quando você precisar de uma sexta recusa amanhã, ela entra como mais um bloco de cinco linhas, sem mexer nos outros.

Por que M5 é o último. As quatro primeiras conferências olham só para o que está na tela: comparar textos e contar caracteres não custa quase nada e não depende de o MySQL estar ligado. A quinta abre uma conexão, manda um `SELECT COUNT(*)` e espera a resposta do banco. Colocá-la por último significa que um usuário que clicou em Cadastrar com a tela em branco recebe o aviso na hora, sem que o programa tenha ido ao banco à toa. **O barato primeiro, o caro depois** — e isso vale para muito além deste método.

O código

```

private void btnCadastrar_Click(object sender, EventArgs e)
{
    // M1 - nenhum campo pode ficar vazio
    if (txtNome.Text == "" || txtUsuario.Text == "" || txtSenha.Text == "")
    {
        MessageBox.Show("Preencha todos os campos.", "Atencao",
            MessageBoxButtons.OK, MessageBoxIcon.Warning);
        return;
    }

    // M2 - o login precisa servir (sem espaco, 3 caracteres ou mais)
    if (Regras.ValidarLogin(txtUsuario.Text) == false)
    {
        MessageBox.Show("O usuario nao pode ter espaco e precisa de "
            + Regras.TAMANHO_MINIMO_LOGIN + " caracteres ou mais.",
            "Atencao", MessageBoxButtons.OK, MessageBoxIcon.Warning);
        return;
    }

    // M3 - a senha precisa ter 4 caracteres ou mais
    if (txtSenha.Text.Length < Regras.TAMANHO_MINIMO_SENHA)
    {
        MessageBox.Show("A senha precisa ter "
            + Regras.TAMANHO_MINIMO_SENHA + " caracteres ou mais.",
            "Atencao", MessageBoxButtons.OK, MessageBoxIcon.Warning);
        return;
    }

    // M4 - as duas senhas precisam ser iguais
    if (txtSenha.Text != txtConfirmar.Text)
    {
        MessageBox.Show("As senhas nao sao iguais.", "Atencao",
            MessageBoxButtons.OK, MessageBoxIcon.Warning);
        return;
    }

    // M5 - o login nao pode estar em uso
    if (UsuarioDAO.LoginExiste(txtUsuario.Text) == true)
    {
        MessageBox.Show("Este usuario ja esta em uso. Escolha outro.",
            "Atencao", MessageBoxButtons.OK, MessageBoxIcon.Warning);
        return;
    }

    // Passou por tudo: o que foi digitado vira OBJETO, e o
    // objeto e que vai para o banco.
    Usuario novo = new Usuario();
    novo.Nome = txtNome.Text;
    novo.Login = txtUsuario.Text;
    novo.Senha = txtSenha.Text;

    UsuarioDAO.CriarConta(novo);

    MessageBox.Show("Conta criada! Agora faca o login.", "Pronto",
        MessageBoxButtons.OK, MessageBoxIcon.Information);

    this.Close();
}

```

Linha	Por que ela está aí						
<pre>private void btnCadastrar_Click(object sender, EventArgs e)</pre>	Assinatura obrigatória de um evento de clique no Windows Forms. <code>sender</code> seria o botão que foi clicado e <code>e</code> traria detalhes do evento; aqui nenhum dos dois é usado, mas os dois têm que existir , senão o Designer não consegue mais ligar o botão a este método. Não apague nem renomeie os parâmetros.						
<pre>// M1 - nenhum campo pode ficar vazio</pre>	O comentário não é enfeite: <code>M1</code> é o nome dessa recusa no manual. Quando o professor pedir na arguição " <i>abra a linha M4 do seu manual e me mostre onde ela está no seu código</i> ", é este comentário que responde em dois segundos.						
<pre>`txtNome.Text == "" \</pre>	<pre>\</pre>	<pre>txtUsuario.Text == "" \</pre>	<pre>\</pre>	<pre>txtSenha.Text == ""^</pre>	Três testes ligados por \	<pre>\</pre>	<pre>&&` (e)</pre> inverte o sentido: só recusaria quando os três estivessem vazios ao mesmo tempo — ou seja, aceitaria um cadastro com nome em branco.
<pre>txtConfirmar</pre> não aparece no M1	De propósito. Se a confirmação ficar vazia e a senha não, os dois textos são diferentes, e quem recusa é o M4, com						

Linha	Por que ela está aí
	uma mensagem mais precisa ("As senhas nao sao iguais") do que "Preencha todos os campos". Uma recusa a menos, sem buraco na conferência.
<pre>MessageBox.Show("...", "Atencao", MessageBoxButtons.OK, MessageBoxIcon.Warning);</pre>	Quatro argumentos: o texto, o título da janelinha, quais botões ela mostra e qual ícone. <code>MessageBoxButtons.OK</code> põe um botão só, porque não há nada a escolher — é um aviso, não uma pergunta. <code>MessageBoxIcon.Warning</code> põe o triângulo amarelo, que o usuário lê como "algo deu errado" antes mesmo de ler a frase.
<pre>return; (dentro de cada if)</pre>	A linha mais importante do método. Ela encerra o <code>btnCadastrar_Click</code> na hora. Sem ela, o programa mostra o aviso, fecha a caixinha e continua descendo — mostra o próximo aviso, e o próximo, e no fim grava a conta que você acabou de recusar. Como o método é <code>void</code>, o <code>return</code> vem sozinho, sem valor depois dele.

Linha	Por que ela está aí
<pre>Regras.ValidarLogin(txtUsuario.Text)</pre>	<code>Regras.</code> na frente porque <code>ValidarLogin</code> é <code>static</code>: pertence à classe, não a um objeto. Você não cria um <code>new Regras()</code> — chama direto pelo nome da classe. Aqui a tela pergunta e a regra responde; a tela não repete a lógica dos espaços e do tamanho mínimo.
<pre>== false (e, no M5, == true)</pre>	<code>ValidarLogin</code> já devolve <code>true</code> ou <code>false</code>; escrever <code>== false</code> é a forma longa de dizer "se não valeu". Existe a forma curta com <code>!</code> na frente, mas o <code>!</code> some no meio da linha quando você está cansado, e um <code>!</code> esquecido inverte a regra sem erro de compilação. A forma longa está aqui para ser lida em voz alta na arguição.
<pre>+ Regras.TAMANHO_MINIMO_LOGIN +</pre> dentro da mensagem	O número 3 não é digitado na frase. Ele vem da constante, que mora em um lugar só. Se amanhã a regra virar 5 caracteres, você muda o <code>const</code> no <code>Regras.cs</code> e a mensagem da tela — e, junto com ela, a linha M2 do manual — passa a dizer 5 sozinha. Número solto dentro de texto é a forma mais silenciosa de um manual ficar mentindo.
<pre>txtSenha.Text.Length < Regras.TAMANHO_MINIMO_SENHA</pre>	<code>.Length</code> conta quantos caracteres tem o texto. É <code><</code> e não <code><=</code>: com <code>TAMANHO_MINIMO_SENHA</code> valendo 4, uma senha de

Linha	Por que ela está aí
	exatamente 4 caracteres passa, porque 4 não é menor que 4. Trocar por <code><=</code> exigiria 5 caracteres e faria a mensagem mentir.
<pre>txtSenha.Text != txtConfirmar.Text</pre>	<code>!=</code> é "diferente de". Comparar duas <code>string</code> com <code>==</code> e <code>!=</code> compara o conteúdo dos textos, letra por letra — e a comparação diferencia maiúscula de minúscula, que é exatamente o que se quer numa senha.
<pre>UsuarioDAO.LoginExiste(txtUsuario.Text)</pre>	Única linha do método que encosta no banco, e por isso a última das cinco. Também é <code>static</code>, chamada pelo nome da classe.
<pre>Usuario novo = new Usuario();</pre>	<code>new</code> constrói um objeto novo na memória, com <code>Id</code> em 0, <code>Nome</code>, <code>Login</code> e <code>Senha</code> como texto vazio. <code>novo</code> é o apelido pelo qual você fala com ele daqui para baixo. Este é o ponto em que texto de tela vira objeto: até aqui existiam quatro caixas soltas; a partir daqui existe <i>um usuário</i>.
<pre>novo.Nome = txtNome.Text;</pre>	Atribuição de propriedade — e a única das três que faz mais do que copiar. O <code>set</code> do <code>Nome</code> (seu TODO 2) apara os espaços das pontas antes de guardar. Por isso um nome digitado como <code>" Ana Silva "</code> entra no banco como <code>"Ana Silva"</code>, sem você fazer nada aqui. Os dois espaços antes do <code>=</code> são só alinhamento visual

Linha	Por que ela está aí
	com as linhas de baixo; o compilador ignora.
<pre> novo.Login, novo.Senha </pre>	Propriedades automáticas, sem tratamento: guardam o que receberam. A senha vai em texto puro, como combinado no projeto. <code>DataCadastro</code> não é preenchida aqui de propósito — quem carimba a data é o banco, pelo <code>DEFAULT CURRENT_TIMESTAMP</code> da tabela.
<pre> UsuarioDAO.CriarConta(novo); </pre>	Você entrega o objeto inteiro, não três textos separados. Se um dia a conta ganhar um campo novo, a assinatura de <code>CriarConta</code> não muda — só o objeto fica maior. E repare: <code>CriarConta</code> devolve um <code>int</code> (o Id gerado), mas aqui esse retorno é ignorado. Você pode ignorar o retorno de um método quando não precisa dele; nesta tela, o Id novo não serve para nada, porque a janela vai fechar.
<pre> MessageBoxIcon.Information </pre> na mensagem final	Muda o ícone de triângulo amarelo para o "i" azul. É o sinal visual de que desta vez não houve problema — o mesmo tipo de detalhe que faz a diferença entre um sistema que parece pronto e um que parece rascunho.
<pre> this.Close(); </pre>	Fecha esta janela (<code>this</code> é o próprio <code>frmCadastro</code>), e não o programa. A tela de cadastro foi aberta pelo

Linha	Por que ela está aí
	login com <code>ShowDialog()</code> , que fica parado esperando; quando ela fecha, o login volta a responder e a pessoa já pode entrar com a conta recém-criada. Se aqui estivesse <code>Application.Exit()</code> , o programa inteiro se fecharia depois de cada cadastro.

O erro mais comum

1. Esquecer o `return` dentro de algum `if`. Sintoma: você clica em Cadastrar com a tela em branco e aparecem **duas, três ou quatro** caixas de mensagem uma atrás da outra — e, depois da última, a conta é criada assim mesmo (ou o programa quebra com um erro do MySQL sobre `Login` duplicado, que é o `UNIQUE` da tabela recusando o que o seu `if` deixou passar). Como sair: confira que cada um dos cinco blocos tem `return;` como **última linha antes da chave que fecha o `if`**. Cinco `if`, cinco `return`.

2. Escrever o TODO 6 antes dos TODO 3, 4 e 5. Este é o engano que mais confunde, porque **o projeto compila e roda**. Os esqueletos dos outros métodos têm respostas provisórias: `ValidarLogin` devolve `true` sempre, `LoginExiste` devolve `false` sempre e `CriarConta` devolve `0` sem gravar nada. Sintoma: o cadastro aceita `ab` como login e aceita o mesmo login duas vezes, e a mensagem "Conta criada!" aparece — mas o `SELECT * FROM Usuario` no Workbench continua com as três linhas que o script `CriarBanco.sql` gravou, e nada da conta nova. Não é o seu TODO 6 que está errado. Como sair: termine os TODO 2, 3, 4 e 5 e teste este de novo, sem mudar nada aqui.

3. Trocar `||` por `&&` no M1. Sintoma: a tela aceita cadastro com o nome em branco, e só reclama quando os três campos estão vazios ao mesmo tempo. Como sair: leia a condição em voz alta trocando `||` por "ou" — "se o nome está vazio ou o usuário está vazio ou a senha está vazia, recuse". É essa frase que tem que fazer sentido.

4. Deixar a linha do esqueleto no lugar. Sintoma: a caixa "TODO 6 ainda nao foi escrito." aparece antes de tudo o mais. Como sair: apague a linha marcada com `<<< APAGUE esta linha`.

5. Mudar a redação das mensagens depois de escrever o manual. Não produz erro nenhum na tela — produz um manual errado. As cinco frases deste método são copiadas letra por letra para a tabela de mensagens do PDF. Decida a redação agora e não mexa mais.

Como saber que funcionou

Antes de começar, uma coisa que muda o resultado dos testes: **o banco não nasce vazio**. O script `CriarBanco.sql` que você rodou no começo já gravou três contas de exemplo — `ana`, `bruno` e `admin`. Os logins que você usar para testar o cadastro **novo** precisam ser diferentes desses três, senão quem responde é o M5, e não a mensagem que você espera.

Rode o programa (**F5**), confirme que a faixa do login está verde e clique em **Criar conta**. Faça os sete testes abaixo na ordem, um clique em Cadastrar para cada um:

#	O que digitar	O que tem que acontecer
1	Nada, tudo em branco	"Preencha todos os campos." — uma caixa só
2	Nome <code>Carla Nunes</code> , usuário <code>ca</code> , senha <code>1234</code> , repetir <code>1234</code>	"O usuario nao pode ter espaco e precisa de 3 caracteres ou mais."
3	Usuário <code>carla nunes</code> (com espaço), o resto igual	a mesma mensagem do teste 2
4	Usuário <code>carla</code> , senha <code>123</code> , repetir <code>123</code>	"A senha precisa ter 4 caracteres ou mais."
5	Usuário <code>carla</code> , senha <code>1234</code> , repetir <code>4321</code>	"As senhas nao sao iguais."
6	Usuário <code>ana</code> , senha <code>1234</code> , repetir <code>1234</code>	"Este usuario ja esta em uso. Escolha outro." — o <code>ana</code> veio do script, e é o M5 recusando
7	Nome <code>Carla Nunes</code> , usuário <code>carla</code> , senha <code>1234</code> , repetir <code>1234</code>	"Conta criada! Agora faça o login." e a janela de cadastro fecha sozinha

Depois do teste 7, clique em **Criar conta** de novo e repita exatamente os mesmos dados da Carla. Agora tem que aparecer "Este usuario ja esta em uso. Escolha outro." — é o M5 recusando um login que não existia dez segundos atrás, e é a prova de que o teste 7 realmente gravou.

Confira no Workbench. Feche o programa e rode no MySQL Workbench:

```
SELECT Id, Nome, Login, Senha, DataCadastro FROM Usuario ORDER BY Id;
```

A linha da Carla tem que estar lá, embaixo das três que o script criou (`ana`, `bruno` e `admin`), com `Id` preenchido pelo banco e `DataCadastro` com a data e a hora de agora — nenhum dos dois foi digitado por você. Este é o teste que fecha a noite 1: **criar uma conta pelo sistema, fechar o programa, e ela aparecer no SELECT**.

Um teste a mais, se quiser ver o TODO 2 trabalhando: cadastre mais uma conta — com um login ainda não usado, `diego` por exemplo — digitando o nome com um espaço sobrando no começo e no fim, e olhe o `Nome` no Workbench. Ele tem que estar sem os espaços.

Para pensar

O M1 recusa o cadastro quando `txtNome.Text == ""` — texto vazio. Mas um nome digitado como três espaços (`" "`) **não** é texto vazio, então ele passa pelo M1, chega ao `novo.Nome`, e o `Trim()` do seu TODO 2 o transforma em `""` antes de guardar. O resultado é uma conta gravada no banco com o nome em branco. Por que o M2 não deixa isso acontecer com o **login**, e o que precisaria mudar — na tela ou no `Regras.cs` — para o nome ficar igualmente protegido?

★ O teste que fecha a primeira noite

Você terminou os TODO 1 a 6. Antes de ir embora, faça isto:

1. Rode o programa (F5) e clique em **Criar conta**
2. Cadastre alguém — nome completo, um usuário e uma senha de 4 ou mais
3. **Feche o programa**
4. Abra o MySQL Workbench e rode:

```
SELECT * FROM portariadb.Usuario;
```

A conta que você criou está lá?

Então o dado **saiu do programa e ficou no banco**. Isso é o que o indicador I5 pede, e é a coisa mais importante que acontece nesta noite: até hoje, tudo o que você programou nesta UC evaporava quando o programa fechava.

Se a conta não estiver lá, o problema está no TODO 5 ou no TODO 6 — e o capítulo de cada um tem uma seção "O erro mais comum" que provavelmente descreve o seu.

PARTE 2 — Segunda noite: entrar e listar

TODO 7 a 14. No fim desta noite o sistema está inteiro.

#	Método	O que muda quando você escreve
7	UsuarioDAO.MontarUsuario	Nada sozinho — mas o 8 e o 10 dependem dele. Escreva primeiro
8	UsuarioDAO.Autenticar	O banco passa a saber dizer quem é quem
9	frmLogin.btnEntrar_Click	Você entra no sistema
10	UsuarioDAO.ListarTodos	A lista sai do banco, nas duas ordens
11	Regras.UltimoCadastrado	O rodapé diz quem foi o último
12	frmPrincipal.CarregarLista	A lista aparece na tela
13	UsuarioDAO.ExcluirConta	Dá para tirar alguém do sistema
14	frmPrincipal.btnExcluir_Click	O botão Excluir funciona

TODO 7 - MontarUsuario

Arquivo	Dados/UsuarioDAO.cs
Indicador	I5 · I2
O que voce exercita	<code>new</code> de objeto, leitura de coluna pelo nome (<code>r["Coluna"]</code>), conversao com <code>Convert</code> , metodo privado com retorno
Depende de	TODO 2 (a propriedade <code>Nome</code> , porque a linha <code>u.Nome = ...</code> passa pelo <code>set</code> dela). Nada mais.

O que este método faz

Ele recebe um leitor de banco ja posicionado numa linha da tabela `Usuario` e devolve essa linha transformada em um objeto `Usuario` do C#. Entra uma linha do MySQL, sai um objeto com as cinco propriedades preenchidas: `Id`, `Nome`, `Login`, `Senha` e `DataCadastro`.

Quem chama ele nao e nenhuma tela. Sao os dois metodos que vem logo depois neste mesmo arquivo: o `Autenticar` (TODO 8), que le uma linha so, e o `ListarTodos` (TODO 10), que le todas. Por isso a ordem de escrita importa: escreva o TODO 7 primeiro, senao os dois seguintes nao tem o que chamar e voce vai acabar copiando as mesmas cinco atribuicoes duas vezes.

Repare no que ele **nao** faz: ele nao abre conexao, nao fecha conexao, nao monta SQL e nao avanca o leitor para a proxima linha. Ele so olha para a linha que ja esta debaixo do leitor e copia os cinco valores. Essa lista de coisas que ele nao faz e o assunto da secao seguinte.

Por que ele existe assim

Por que existe. Sem ele, as cinco atribuicoes apareceriam duas vezes: uma dentro do `Autenticar` e outra dentro do `ListarTodos`. Codigo repetido nao e so feio, e caro de manter: no dia em que a tabela ganhar uma coluna `Telefone`, voce teria que lembrar de mexer nos dois lugares. Esquecer um deles produz um defeito estranho, do tipo "o telefone aparece na lista mas some depois que eu faco login". Com o metodo aqui, a resposta e outra: mudou uma coluna? Muda aqui, num lugar so, e os dois metodos passam a trazer a coluna nova de graca.

Por que neste arquivo e nao no `Usuario.cs`. A tentacao e colocar esse metodo dentro da propria classe `Usuario`, no Modelo. Nao coloque. Olhe o topo do `Modelo/Usuario.cs`: ele tem `using System;` e mais nada. A classe `Usuario` nao sabe que existe MySQL, nao sabe o nome das colunas, nao sabe se os dados vieram de um banco, de um arquivo texto ou digitados na tela. Ela e so o formato do dado. No momento em que voce escrevesse `MySqlDataReader` la dentro, o Modelo passaria a precisar da biblioteca do MySQL para compilar, e trocar de banco um dia significaria mexer no Modelo tambem. Conhecimento sobre colunas e sobre `MySqlDataReader` e conhecimento de banco, e conhecimento de banco mora no DAO.

Por que `private`. As telas nunca vao chamar este metodo — elas nem teriam como. Para chamar o `MontarUsuario` voce precisa ter em maos um `MySqlDataReader` aberto, e o leitor so funciona enquanto a conexao esta aberta. Como a conexao abre e fecha dentro do proprio DAO (no `try/finally`), fora daqui nao existe leitor vivo. Um metodo que so faz sentido dentro deste arquivo se declara `private`: assim o Visual Studio nem oferece ele no `IntelliSense` das telas, e ninguem tenta usar o que nao da para usar.

Por que ele foge do "esqueleto de sempre". Todos os outros metodos do `UsuarioDAO` seguem o mesmo roteiro: pega a conexao, `try`, `con.Open()`, monta o SQL, executa, `finally { con.Close(); }`. Este e a excecao, e a excecao tem motivo: ele nao abre nada, entao nao tem nada para fechar. A linha que ele esta lendo pertence a uma conexao que **outra pessoa** abriu e que **outra pessoa** vai fechar. Se voce abrisse uma segunda conexao aqui dentro, ela seria aberta e fechada uma vez para cada usuario da lista, sem servir para nada — o valor que ele copia ja esta no leitor que chegou pelo parametro.

Por que devolve um `Usuario` em vez de receber um por parametro. Ele podia ter sido escrito como `MontarUsuario(MySqlDataReader r, Usuario u)`, preenchendo um objeto que o chamador criou. Seria mais linha para o chamador escrever e uma chance a mais de ele mandar um objeto nulo. Como so ha uma coisa para devolver, o `return` da conta: quem chama escreve `lista.Add(MontarUsuario(r));` numa linha so.

O código

```
private static Usuario MontarUsuario(MySqlDataReader r)
{
    Usuario u = new Usuario();

    u.Id          = Convert.ToInt32(r["Id"]);
    u.Nome        = Convert.ToString(r["Nome"]);
    u.Login       = Convert.ToString(r["Login"]);
    u.Senha       = Convert.ToString(r["Senha"]);
    u.DataCadastro = Convert.ToDateTime(r["DataCadastro"]);

    return u;
}
```

Linha a linha

Linha	Por que ela esta ai
<code>private</code>	So o proprio <code>UsuarioDAO</code> chama este metodo. As telas nao poderiam chamar mesmo: elas nao tem um leitor aberto na mao. Marcar como <code>private</code> deixa isso explicito e evita que alguem tente.
<code>static</code>	Os metodos que chamam ele (<code>Autenticar</code> e <code>ListarTodos</code>) sao <code>static</code> , e um metodo <code>static</code> so consegue chamar outro <code>static</code> sem ter um objeto. Alem disso, ninguem escreve <code>new UsuarioDAO()</code> neste projeto — a classe e uma caixa de ferramentas, nao um objeto com estado.
<code>Usuario</code> (o tipo antes do nome)	E o que sai do metodo. Como o retorno e <code>Usuario</code> , quem chama pode escrever direto <code>Usuario achado = MontarUsuario(r);</code> sem converter nada.
<code>(MySqlDataReader r)</code>	O tipo do parametro e exatamente o que o <code>cmd.ExecuteReader()</code> devolve — por isso o <code>Autenticar</code> e o <code>ListarTodos</code> conseguem passar o leitor deles sem conversao nenhuma. O nome curto <code>r</code> se justifica porque ele aparece cinco vezes em cinco linhas seguidas; um nome longo so encheria a linha.
<code>Usuario u = new Usuario();</code>	O <code>new</code> reserva a memoria e cria o objeto vazio. Vazio nao quer dizer nulo: gracias ao <code>= ""</code> nas propriedades e ao <code>set</code> do <code>Nome</code> , o objeto ja nasce com textos vazios em vez de nulos. As cinco linhas seguintes so trocam esses valores iniciais pelos que vieram do banco.
<code>r["Id"]</code>	O leitor entrega o valor da coluna chamada <code>Id</code> da linha atual . Usar o nome da coluna, e nao a posicao dela, e uma escolha: se um dia alguem trocar a ordem das colunas do <code>SELECT</code> , o nome continua achando o valor certo. Cuidado: o que esta entre aspas o compilador nao confere. Escrever <code>r["Idd"]</code> compila sem reclamar e so quebra quando o programa roda.
<code>Convert.ToInt32(...)</code>	O leitor nao sabe, na hora de compilar, que tipo tem cada coluna, entao ele devolve tudo como <code>object</code> — o tipo mais geral do C#, aquele em que cabe qualquer valor. <code>object</code> nao serve para atribuir a um <code>int</code> . O <code>Convert.ToInt32</code> pega esse valor e devolve o numero. Nao use <code>TryParse</code> aqui: o <code>TryParse</code> e para texto digitado , que pode nao ser numero; este valor veio do banco, de uma coluna <code>INT</code> , e ja e numero.
Por que <code>Convert</code> e nao <code>(int)</code> na frente	Colocar <code>(int)</code> na frente exige que o valor por tras seja exatamente um <code>int</code> ; se o banco devolver outro tipo numerico, o programa para com <code>InvalidCastException</code> . O <code>Convert.ToInt32</code> aceita qualquer um deles e entrega o <code>int</code> . Ele custa um pouco mais e nao quebra — troca boa.

Linha	Por que ela esta ai
<code>Convert.ToString(...)</code>	Mesma historia das colunas de texto: o que chega e <code>object</code> , e <code>object</code> nao entra numa propriedade <code>string</code> sem conversao.
<code>Convert.ToDateTime(...)</code>	A coluna <code>DataCadastro</code> e <code>DATETIME</code> no MySQL e <code>DateTime</code> no C#. Convertida aqui, ela chega na tela principal como data de verdade, e por isso o <code>frmPrincipal</code> consegue escrever <code>u.DataCadastro.ToString("dd/MM/yyyy")</code> . Se voce guardasse a data como texto, esse <code>ToString</code> com formato nao existiria.
<code>u.Nome = ...</code>	Esta linha nao guarda o valor direto no objeto: ela passa pelo <code>set</code> que voce escreveu no TODO 2, que tira os espacos das pontas e troca nulo por texto vazio. E o encapsulamento funcionando sem voce pedir — o objeto se defende sozinho, venha o dado da tela ou do banco.
Os <code>=</code> alinhados em coluna	Puro conforto de leitura: com os sinais de igual um embaixo do outro, o olho compara os cinco lados direitos de uma vez e enxerga rapido se alguma coluna ficou de fora. O compilador ignora esses espacos.
As cinco linhas, e nao quatro	A <code>Senha</code> e lida mesmo sem aparecer em tela nenhuma. Ela vem porque o <code>SELECT</code> do TODO 8 e do TODO 10 traz as cinco colunas, e este metodo precisa combinar com aquele <code>SELECT</code> . Combinar os dois e responsabilidade sua — o compilador nao verifica isso.
Nao ha <code>r.Read()</code> aqui dentro	Quem avanca o leitor e quem chama: o <code>Autenticar</code> com <code>if (r.Read())</code> , o <code>ListarTodos</code> com <code>while (r.Read())</code> . E justamente por nao avancar nada que este mesmo metodo serve aos dois.
Nao ha <code>try / finally</code> nem <code>con</code>	A conexao pertence a quem chamou. Aqui nao se abre nada, entao nao ha nada para fechar.
<code>return u;</code>	Devolve o objeto pronto. Sem esta linha o metodo nem compila — o compilador exige que todo caminho de um metodo com retorno termine devolvendo alguma coisa.

O erro mais comum

1. Deixar a linha do esqueleto no lugar. A linha `return new Usuario();` vem escrita no arquivo, e muita gente escreve o codigo novo **embaixo** dela. O metodo sai no primeiro `return`, e o resto nunca roda. O Visual Studio avisa com um aviso amarelo (`CS0162 - codigo inacessivel detectado`), que e facil de ignorar porque o projeto compila assim mesmo. O sintoma aparece na tela principal: a lista mostra as linhas certas em quantidade, mas com o nome em branco, o login vazio entre parenteses e a data `01/01/0001`. Apague a linha do esqueleto, nao escreva em volta dela.

2. O `SELECT` que nao traz as cinco colunas. No `CriarBanco.sql` existe uma consulta de conferencia que traz `Id, Nome, Login, DataCadastro` — sem a `Senha`. Copiar aquela consulta para o TODO 10 faz o `MontarUsuario` parar na linha `u.Senha = Convert.ToString(r["Senha"]);`, porque a coluna nao veio no resultado. Na tela voce ve

a caixa "Não consegui ler a lista." com um recado dizendo que a coluna `Senha` não foi encontrada. O conserto não é aqui: é no `SELECT` do método que chamou.

3. Um nome de coluna errado dentro das aspas. `r["Data_Cadastro"]`, `r["nome_usuario"]`, `r["Usuario.Id"]` — tudo isso compila. O compilador não lê o que está entre aspas, ele só vê um texto. O erro só aparece quando o programa roda, e sempre com a mesma cara: uma reclamação de que a coluna não foi achada no resultado. Confira os nomes contra o `DESCRIBE Usuario;` do Workbench, letra por letra.

4. Esquecer o `Convert`. Escrever `u.Id = r["Id"];` produz erro de compilação (`CS0266 - não é possível converter implicitamente 'object' em 'int'`). Este erro é o único bom da lista: ele aparece antes de rodar, com a linha sublinhada, e o conserto é envolver o valor no `Convert` certo.

5. Chamar `r.Read()` aqui dentro. Parece que falta, já que o método "lê" o leitor. Não falta: chamar `Read()` aqui faz o leitor pular para a próxima linha antes de copiar os valores. No `ListarTodos` o leitor passa a andar duas casas por volta — o método copia o usuário errado e, quando o `Read()` de dentro não acha mais linha, estoura com um recado de tentativa de leitura sem linha disponível. Como o estouro acontece lá dentro do DAO, a tela nem chega a redesenhar a lista: você vê só a caixa "Não consegui ler a lista.". No `Autenticar`, o login de um usuário que existe passa a dar erro. Deixe o avanço do leitor com quem chama.

Como saber que funcionou

Este método é `private`, então você não consegue chamá-lo de uma tela para testar. O teste honesto é com **breakpoint**, e ele funciona assim que o TODO 8 (`Autenticar`) estiver escrito:

1. Clique na margem cinza à esquerda da linha `return u;` para pintar a bolinha vermelha.
2. F5 para rodar. Na tela de login digite `ana` e `1234` e clique em Entrar.
3. A execução para na linha do `return`. Passe o mouse em cima do `u` e clique na setinha para abrir o objeto.
4. Você tem que ver, exatamente: `Id = 1`, `Nome = "Ana Souza"`, `Login = "ana"`, `Senha = "1234"` e `DataCadastro` com a data e a hora em que você rodou o `CriarBanco.sql`.

Se algum desses campos estiver com `0`, `""` ou `01/01/0001`, a linha correspondente não rodou ou leu a coluna errada.

Depois que o TODO 10 estiver pronto, o teste fica visual: entre no sistema e compare a tela principal com esta consulta no Workbench:

```
SELECT Id, Nome, Login, Senha, DataCadastro
FROM Usuario
ORDER BY Nome;
```

São as mesmas cinco colunas que o TODO 10 pede. A tela tem que mostrar os três nomes na mesma ordem, com o mesmo login entre parênteses e a mesma data. Um detalhe que vale conferir: os `Id` da tela não aparecem, mas eles estão lá por trás. Se o `u.Id` não tiver sido preenchido, todos os usuários ficam com `Id = 0` — inclusive o que está logado — e o botão **Excluir o selecionado** passa a recusar **qualquer** linha com a mensagem "Você não pode excluir a sua própria conta enquanto está conectado". Se isso acontecer com você na segunda noite, o defeito não está no botão: está na primeira das cinco atribuições daqui.

Para pensar

Este mesmo método serve ao `Autenticar`, que lê uma linha só, e ao `ListarTodos`, que lê todas — sem uma vírgula de diferença entre os dois usos. O que exatamente ele **deixou de fazer** para que isso fosse possível, e o

que aconteceria com a lista da tela principal se ele fizesse essa coisa a mais?

TODO 8 - Autenticar

Arquivo	Dados/UsuarioDAO.cs
Indicador	I5 · I3
O que você exercita	<code>SELECT</code> com dois critérios ligados por <code>AND</code> ; dois <code>@parametro</code> no mesmo comando; <code>ExecuteReader</code> ; <code>if (r.Read())</code> ; devolver <code>null</code> ; <code>try/finally</code>
Depende de	TODO 7 (MontarUsuario) — é ele quem preenche o objeto. O projeto até compila sem o TODO 7, porque o esqueleto já traz esse método com um <code>return new Usuario();</code> dentro; só que aí o login aprova a dupla certa e devolve uma conta em branco . Escreva o TODO 7 primeiro. Para <i>testar</i> , você não precisa cadastrar nada: o <code>CriarBanco.sql</code> já gravou três contas — <code>ana / 1234</code> , <code>bruno / 1234</code> e <code>admin / admin</code>

O que este método faz

Ele recebe dois textos — o login e a senha que a pessoa digitou na tela de entrada — e vai ao banco perguntar uma coisa só: *existe alguma linha da tabela `Usuario` em que o `Login` seja exatamente este e a `Senha` seja exatamente esta?*

Se existir, ele monta o objeto `Usuario` com os dados dessa linha e devolve o objeto. Se não existir, ele devolve `null` — a palavra que em C# significa "nenhum objeto", "não há nada aqui".

Quem chama é o `btnEntrar_Click` da tela de login (o TODO 9). Aquele método não sabe nada de SQL: ele pega o que veio de volta e faz uma pergunta só, `if (achado == null)`. Se veio `null`, mostra a mensagem M7 e não deixa entrar. Se veio um objeto, guarda em `Sessao.UsuarioLogado` e abre a tela principal. Ou seja: **este método não decide nada na tela — ele produz a informação com a qual a tela decide.**

Por que ele existe assim

Por que ele mora no `UsuarioDAO.cs` e não na tela de login. A tela é o lugar de perguntar e avisar; o DAO é o lugar de falar com o banco. Se o `SqlCommand` estivesse dentro do `btnEntrar_Click`, o SQL da autenticação ficaria trancado num evento de botão: nenhum outro pedaço do sistema conseguiria reaproveitar, e trocar o banco de lugar exigiria mexer numa tela. Neste projeto, todo comando SQL mora num arquivo só, o `UsuarioDAO.cs` — em nenhum outro. O `Conexao.cs`, que fica ao lado dele, guarda apenas o endereço do banco e a abertura da conexão: não tem uma linha de SQL dentro.

Por que ele devolve `Usuario` e não `bool`. Seria possível escrever um `AutenticarOk` que devolvesse só `true` ou `false`. Só que a tela, logo depois de aprovar a entrada, precisa do **nome** e do **Id** de quem entrou, para escrever "Conectado como: Ana" e para saber de quem é a sessão. Com um `bool`, a tela teria que ir ao banco uma segunda vez buscar exatamente a linha que este método acabou de ter na mão. Devolver o objeto resolve as duas coisas numa ida só. E `null` já é uma resposta de "não" perfeitamente clara — não faz falta um `bool` separado.

Por que os dois critérios vão no mesmo `SELECT`. Existe um caminho tentador que não é o daqui: buscar a linha só pelo login e depois, no C#, comparar `u.Senha == senha` com um `if`. Funciona, mas espalha a decisão

em dois lugares — meia regra no banco, meia regra no código —, e no dia em que alguém alterar um dos dois lados a autenticação passa a mentir. Aqui a pergunta é uma só, e a resposta do banco já é a resposta final: **veio linha = a dupla confere; não veio linha = não confere**. Repare que o método também não tem como saber *qual* dos dois estava errado — e isso é bom: a tela mostra uma frase única, "Usuario ou senha incorretos", e quem estiver tentando adivinhar uma senha não descobre por essa mensagem se pelo menos acertou o login.

Por que a assinatura é `static`. Como todos os métodos do DAO. `static` quer dizer que o método pertence à classe, não a um objeto dela: você chama `UsuarioDAO.Autenticar(...)` direto, sem escrever `new UsuarioDAO()` em lugar nenhum. Um DAO não guarda estado — não tem nada dentro dele para valer a pena criar um objeto.

O código

```
public static Usuario Autenticar(string login, string senha)
{
    Usuario achado = null;

    MySqlConnection con = Conexao.Obter();

    try
    {
        con.Open();

        string sql = @"SELECT Id, Nome, Login, Senha, DataCadastro
                        FROM Usuario
                        WHERE Login = @login
                        AND Senha = @senha";

        MySqlCommand cmd = new MySqlCommand(sql, con);
        cmd.Parameters.AddWithValue("@login", login);
        cmd.Parameters.AddWithValue("@senha", senha);

        MySqlDataReader r = cmd.ExecuteReader();

        if (r.Read())
        {
            achado = MontarUsuario(r);
        }
    }
    finally { con.Close(); }

    return achado;
}
```

Linha a linha

Linha	Por que ela está aí
<pre>public static Usuario Autenticar(string login, string senha)</pre>	O tipo de retorno é <code>Usuario</code> , não <code>bool</code> : a tela precisa do nome e do Id de quem entrou, e não só de um "sim/não". Entram dois <code>string</code> porque são dois textos digitados — o método não recebe um objeto <code>Usuario</code> , já que nesse momento ainda não existe usuário nenhum, só o que a pessoa escreveu nas duas caixas.
<pre>Usuario achado = null;</pre>	Declarada antes do <code>try</code> , e não dentro dele, por um motivo prático: uma variável declarada dentro de um bloco <code>{ }</code> deixa de existir quando o bloco termina. Se ela nascesse dentro do <code>try</code> , o <code>return achado;</code> lá embaixo estaria falando de uma variável que já não existe, e o projeto não compilaria.
<pre>= null</pre> (o valor inicial)	Este é o valor de fracasso , e ele já está posto antes de o banco ser consultado. Se o <code>SELECT</code> não trouxer linha nenhuma, ninguém precisa fazer nada: <code>achado</code> continua <code>null</code> e é isso que sai. O código só mexe nessa variável no caso em que <i>achou</i> .
<pre>MySqlConnection con = Conexao.Obter();</pre>	O <code>Obter()</code> devolve uma conexão fechada , já com o endereço do banco preenchido. Nenhum DAO escreve a string de conexão: se a senha do MySQL mudar, muda em um lugar só, no <code>Conexao.cs</code> .
<pre>try { ... finally { con.Close(); }</pre>	O <code>finally</code> roda de qualquer jeito: se tudo der certo, se der erro no meio, e até quando o método sai por um <code>return</code> escrito dentro do <code>try</code> — é o caso do TODO 4, onde o <code>return quantos > 0;</code> fica lá dentro e mesmo assim a conexão fecha. É a garantia de que a conexão fecha. Sem ela, cada login mal-sucedido deixaria uma conexão aberta pendurada no servidor, até o MySQL recusar novas. Repare que não há <code>catch</code> aqui : este método não sabe o que fazer com um erro de banco e não finge que sabe; ele só se responsabiliza por fechar o que abriu.
<pre>con.Open();</pre>	O <code>Obter()</code> entrega fechada de propósito, para a conexão ficar aberta o menor tempo possível. Quem abre é quem vai usar, e abre já dentro do <code>try</code> — assim, se a abertura falhar, o <code>finally</code> ainda assume o fechamento.
<pre>string sql = @"..."</pre>	O <code>@</code> antes das aspas permite que o texto ocupe várias linhas sem quebrar. É só apresentação: com o SQL escrito em quatro linhas alinhadas, você lê <code>SELECT / FROM / WHERE / AND</code> de relance e enxerga que existem dois critérios.
<pre>SELECT Id, Nome, Login, Senha, DataCadastro</pre>	As cinco colunas estão escritas com o nome, e não com <code>*</code> , porque quem lê essa linha é o <code>MontarUsuario</code> , que procura cada coluna pelo nome. Escrevendo a lista, o <code>SELECT</code> e o <code>MontarUsuario</code> combinam entre si de forma visível: dá para conferir um contra o outro sem abrir a tabela. São as mesmas

Linha	Por que ela está aí
	cinco colunas do <code>ListarTodos</code> — e é por isso que os dois métodos conseguem usar o mesmo <code>MontarUsuario</code> .
<pre>WHERE Login = @login
 AND Senha = @senha</pre>	O <code>AND</code> é o método inteiro. Ele exige que as duas condições sejam verdadeiras na mesma linha. Com <code>OR</code> no lugar, bastaria uma delas bater — e qualquer senha entraria em qualquer conta cujo login existisse.
<pre>@login e @senha</pre>	São <i>marcadores</i> : buracos no comando que o conector preenche depois, avisando ao MySQL "isto aqui é um valor, não é comando". É por isso que o valor nunca é colado com <code>+</code> . Se o SQL fosse montado com <code>+</code> , alguém digitando no campo de senha um texto como <code>' OR '1'='1</code> faria a condição virar sempre verdadeira, e entraria sem saber senha alguma. Com <code>@parametro</code> , esse mesmo texto é procurado como senha literal — e, claro, não encontra ninguém.
<pre>new MySqlCommand(sql, con)</pre>	O comando precisa de duas coisas para existir: o texto do que fazer (<code>sql</code>) e por onde falar (<code>con</code>). Por isso os dois vão no <code>new</code> .
<pre>cmd.Parameters.AddWithValue("@login", login);</pre>	Aqui o buraco é preenchido. O nome entre aspas tem que ser exatamente o que está no SQL, com o <code>@</code> incluído. São duas chamadas porque são dois marcadores: um parâmetro esquecido faz o comando falhar na hora de executar, não na hora de compilar.
<pre>MySqlDataReader r = cmd.ExecuteReader();</pre>	<code>ExecuteReader</code> é o que se usa quando a resposta vem em linhas com colunas . Não é <code>ExecuteScalar</code> (que devolve um valor só, como no <code>COUNT</code> do TODO 4) nem <code>ExecuteNonQuery</code> (que devolve uma contagem, como no <code>DELETE</code> do TODO 13). Aqui você quer cinco colunas de volta.
<pre>if (r.Read())</pre>	O leitor começa posicionado antes da primeira linha — por isso não dá para ler coluna nenhuma sem chamar <code>Read()</code> primeiro. O <code>Read()</code> avança uma linha e devolve <code>true</code> se havia linha para onde avançar. É <code>if</code> e não <code>while</code> porque a coluna <code>Login</code> é <code>UNIQUE</code> na tabela: existe no máximo uma linha possível. Um <code>while</code> funcionaria, mas diria a quem lesse o código "aqui podem vir várias", o que é falso. O código também comunica intenção.
<pre>achado = MontarUsuario(r);</pre>	Só nesta linha <code>achado</code> deixa de ser <code>null</code> . É a única saída "positiva" do método. E é a mesma chamada que o <code>ListarTodos</code> faz: as cinco atribuições de coluna existem uma vez só, no TODO 7 — mudou uma coluna, muda lá, e os dois métodos acompanham.
<pre>return achado;</pre>	Fica depois do <code>finally</code> , fora do <code>try</code> . Assim você lê o método na ordem em que as coisas acontecem: consulta, fecha a conexão, devolve. O que sai daqui é o objeto, se o <code>if</code>

Linha	Por que ela está aí
	entrou, ou o <code>null</code> do começo, se não entrou. Nunca um <code>Usuario</code> vazio.

O erro mais comum

- 1. Devolver `new Usuario()` em vez de `null` quando não acha.** É o erro grave deste método, e o pior tipo: o projeto compila, roda e *parece* funcionar. O sintoma aparece só quando você testa com senha errada — e a tela principal abre assim mesmo, com o cabeçalho "Conectado como:" vazio. A causa é que a tela de login pergunta `if (achado == null)`; um `Usuario` vazio **não é** `null`, é um objeto de verdade, com Id 0 e nome em branco. O `if` não entra, e qualquer senha passa. Corrija deixando `Usuario achado = null;` no topo e não atribuindo nada a ela fora do `if (r.Read())`.
- 2. Declarar `Usuario achado = null;` dentro do `try`.** O projeto não compila e o Visual Studio aponta a linha do `return`: "O nome 'achado' não existe no contexto atual" (CS0103). Não é implicância do compilador: a variável realmente termina no `}` do `try`. Suba a declaração para antes do `try`.
- 3. Trocar o `AND` por `OR`, ou esquecer o segundo critério.** Não dá erro nenhum — dá um sistema que deixa entrar com a senha errada. Este é o teste que você tem obrigação de fazer: digitar o login certo com uma senha errada e conferir que **não** entra.
- 4. Montar o SQL com `+` em vez de `@parametro`.** Costuma funcionar nos primeiros testes e quebra depois: um login com apóstrofo (`D'Avila`) produz "You have an error in your SQL syntax", porque o apóstrofo digitado fecha o texto do comando no meio. E, muito pior que o erro visível, fica o buraco de segurança descrito na tabela acima. Na correção do I5, valor colado com `+` conta como não feito.
- 5. Esquecer o `con.Open();`.** O programa quebra na linha do `ExecuteReader` com "The connection is not open". Acontece quando se copia o esqueleto do TODO 7, que não abre conexão nenhuma — o `MontarUsuario` recebe um leitor já pronto e por isso não tem `Open`, nem `try`, nem `finally`.
- 6. Deixar o TODO 7 por escrever e testar mesmo assim.** O projeto compila (o esqueleto do `MontarUsuario` devolve um `new Usuario()`), o login com a dupla certa é aprovado, e você vê "Conectado como:" sem nome nenhum. Não é defeito deste método: é o TODO 7 ainda em branco.

Como saber que funcionou

Agora, antes mesmo do TODO 9. O botão Entrar ainda não chama este método (isso é o TODO 9), então o teste de agora é do SQL. Abra o Workbench e rode **exatamente a mesma consulta que você escreveu**, trocando os `@parametro` por valores reais. Você não precisa cadastrar ninguém antes: o `CriarBanco.sql` já gravou três contas, e a `ana` (senha `1234`) é uma delas.

```
USE portariadb;

-- a dupla certa: tem que voltar 1 linha
SELECT Id, Nome, Login, Senha, DataCadastro
FROM Usuario
WHERE Login = 'ana'
      AND Senha = '1234';

-- a senha errada: tem que voltar 0 linha
SELECT Id, Nome, Login, Senha, DataCadastro
FROM Usuario
WHERE Login = 'ana'
      AND Senha = 'errada';
```

O segundo `SELECT` tem que devolver `0 row(s) returned`. Se ele devolver a linha da Ana, o seu `AND` virou `OR` ou o segundo critério não está lá.

Depois que o TODO 9 estiver escrito. Rode o programa (F5) e faça as duas passagens:

Você digita	Tem que acontecer
login e senha corretos	a tela de login some e a tela principal abre
login correto, senha errada	a caixa "Usuario ou senha incorretos", a senha some do campo e o cursor volta para ela; a tela principal não abre
login que não existe	a mesma caixa "Usuario ou senha incorretos"

Se quiser ver por dentro. Ponha um ponto de parada (breakpoint, clicando na margem cinza) na linha `if (r.Read())` e rode com F5. Ao parar ali, aperte F10 uma vez e observe: com a dupla certa, a execução entra no `if`; com a senha errada, ela pula direto para o `finally`. Passe o mouse sobre `achado` nos dois casos — na segunda vez ele mostra `null`, e é esse `null` que a tela de login vai receber.

Para pensar

O `SELECT` pede a coluna `Senha` de volta ao banco, mesmo o método já tendo recebido a senha digitada no parâmetro — e essa senha acaba guardada dentro do objeto que vira `Sessao.UsuarioLogado`. Se você tirasse `Senha` da lista de colunas do `SELECT`, deixando-a apenas no `WHERE`, o login continuaria funcionando? O que mudaria no resto do sistema?

TODO 9 - btnEntrar_Click

Arquivo	Telas/frmLogin.cs	
Indicador	I3 · I4 · I6	
O que você exercita	<code>if</code> com `	<code>, comparação com <code>null</code>, <code>return</code> como saída antecipada, uso de classe estática (<code>Sessao</code>), <code>new</code> de um formulário, <code>Hide</code> / <code>ShowDialog</code> / <code>Close</code>`</code>
Depende de	TODO 1 (o banco precisa estar verde) e TODO 8 <code>Autenticar</code> — que por sua vez depende do TODO 7 <code>MontarUsuario</code> . Além disso, precisa existir pelo menos uma conta na tabela: o <code>CriarBanco.sql</code> já gravou três (<code>ana</code> , <code>bruno</code> e <code>admin</code>), e o TODO 6 grava as que você criar pela tela de cadastro.	

O que este método faz

Este é o método que roda quando você clica no botão **Entrar** da tela de login — e também quando você aperta **Enter** dentro de um dos dois campos, porque no Designer o `btnEntrar` foi marcado como `AcceptButton` do formulário. Ele não recebe nada de você: os dois parâmetros (`sender` e `e`) são mandados pelo Windows Forms e, neste método, nenhum dos dois é usado. Ele também não devolve nada — é `void`. O que ele produz é efeito na tela: uma mensagem de recusa, ou a abertura da tela principal.

Por dentro, ele decide entre três caminhos, sempre nesta ordem. Primeiro pergunta se os campos estão vazios; se estiverem, recusa e para por ali (mensagem **M6**). Se houver texto nos dois, entrega o que foi digitado ao `UsuarioDAO.Autenticar` e guarda a resposta numa variável do tipo `Usuario`. Essa resposta pode ser um objeto de verdade (alguém com aquele login e aquela senha existe na tabela) ou pode ser `null`, que é o jeito do TODO 8 dizer "não achei ninguém". Se for `null`, recusa e para (mensagem **M7**). Só no terceiro caminho ele registra quem entrou na `Sessao`, esconde o login, abre a tela principal e espera ela ser fechada para encerrar o programa.

Por que ele existe assim

Por que ele mora na tela e não no DAO. Repare que aqui não aparece `MySqlConnection`, nem `SELECT`, nem `@parametro`. A tela pergunta, o `UsuarioDAO` responde. Isso é a mesma divisão do `Regras.cs`: quem fala com o banco não desenha tela, e quem desenha tela não escreve SQL. É essa separação que permite trocar a mensagem M7 sem abrir o arquivo do banco, e trocar a consulta sem abrir o arquivo da tela.

Por que a decisão "acesso negado" é tomada aqui e não lá dentro. Seria possível o `Autenticar` mostrar a `MessageBox` ele mesmo. Não faz. Ele devolve `null` e vai embora. A razão é que o `Autenticar` não sabe — e não deve saber — o que o programa quer fazer com a resposta: hoje é uma tela de login, amanhã poderia ser um relatório que só quer conferir se a conta existe. Quem sabe o que fazer com o `null` é quem chamou. Por isso o TODO 8 tinha que devolver `null` de verdade, e não um `new Usuario()` vazio: um objeto vazio parece uma conta encontrada, e você só descobriria a diferença olhando campo por campo.

Por que a assinatura é essa e não outra. Você não escolheu

`private void btnEntrar_Click(object sender, EventArgs e)`. Quem escolheu foi o Designer, na linha `this.btnEntrar.Click += new System.EventHandler(this.btnEntrar_Click);` do arquivo `frmLogin.Designer.cs`. O evento `Click` só aceita métodos exatamente com essa forma — dois parâmetros, nessa ordem, e sem retorno. Se você mudar o nome ou os parâmetros, o projeto para de compilar ou o botão simplesmente deixa de fazer efeito, porque ninguém mais está ligado nele.

Por que a `Sessao` é preenchida neste ponto do programa. Este método é o único lugar do sistema em que se tem certeza de que a autenticação passou. Uma linha depois já é tarde: a `frmPrincipal`, no `Load` dela, chama `Sessao.TemUsuarioLogado()` e se fecha sozinha se não houver ninguém. Como o `Load` da principal roda **dentro** do `ShowDialog`, a atribuição precisa vir antes dele.

O código

```
private void btnEntrar_Click(object sender, EventArgs e)
{
    // M6
    if (txtUsuario.Text == "" || txtSenha.Text == "")
    {
        MessageBox.Show("Informe o usuario e a senha.", "Atencao",
            MessageBoxButtons.OK, MessageBoxIcon.Warning);
        return;
    }

    Usuario achado = UsuarioDAO.Autenticar(txtUsuario.Text, txtSenha.Text);

    // O Autenticar devolve null quando nao acha ninguem.
    // Conferir isso ANTES de usar o objeto e o que impede o
    // programa de quebrar aqui.
    if (achado == null)
    {
        // M7
        MessageBox.Show("Usuario ou senha incorretos.", "Acesso negado",
            MessageBoxButtons.OK, MessageBoxIcon.Error);
        txtSenha.Text = "";
        txtSenha.Focus();
        return;
    }

    Sessao.UsuarioLogado = achado;

    // esconde o login, abre o principal, e so volta quando ele fechar
    this.Hide();

    frmPrincipal tela = new frmPrincipal();
    tela.ShowDialog();

    this.Close();
}
```

Linha a linha

Linha	Por que ela está aí			
<pre>private void btnEntrar_Click(object sender, EventArgs e)</pre>	Forma obrigatória de um método de evento. <code>sender</code> é o controle que disparou (o próprio botão) e <code>e</code> traz dados do evento; aqui nenhum dos dois é usado, mas os dois têm que estar declarados, senão o <code>Click</code> do Designer não encaixa.			
<pre>// M6</pre>	A etiqueta da mensagem no manual do sistema. O comentário existe para que, quando o professor pedir "mostre onde nasce a M6", você aponte a linha em dois segundos. A mensagem do código e a do manual têm que ser a mesma, palavra por palavra.			
<pre>`if (txtUsuario.Text == "" \</pre>	<pre>\</pre>	<pre>txtSenha.Text == ""`</pre>	<pre>\ \</pre>	ignora-se o primeiro caractere de escape de barra invertida &&`, a recusa só apareceria se os dois estivessem vazios, e o campo solitário iria parar no banco.
a mesma linha, olhando a ordem	Esta conferência vem antes de qualquer chamada ao <code>UsuarioDAO</code>. Duas razões: não faz sentido abrir conexão, montar comando e ir até o MySQL para perguntar por uma senha em branco; e, se por descuido existisse uma linha com senha vazia na tabela, o			

Linha	Por que ela está aí
	<code>Autenticar</code> diria que ela serve.
<code>MessageBox.Show("Informe o usuario e a senha.", "Atencao", ...)</code>	Primeiro argumento é o texto, segundo é o título da janelinha. Copie a frase exatamente como está: ela é uma das mensagens do manual.
<code>MessageBoxIcon.Warning</code> na M6	Ícone de aviso, não de erro. Campo em branco é um lembrete para o usuário, não uma falha do sistema. O ícone é parte da mensagem.
<code>return;</code> (dentro do primeiro <code>if</code>)	Encerra o método na hora. Sem ele, a execução continua para baixo e chama o <code>Autenticar</code> com texto vazio — a mensagem M6 aparece e, logo atrás dela, a M7.
<code>Usuario achado = UsuarioDAO.Autenticar(txtUsuario.Text, txtSenha.Text);</code>	A variável é do tipo <code>Usuario</code> porque é isso que o TODO 8 devolve. O nome <code>achado</code> descreve o que ela guarda: o que foi encontrado — ou nada. Repare que login e senha vão juntos numa consulta só: quem compara a senha é o banco, no <code>WHERE ... AND Senha = @senha</code> . A tela nunca compara senha.
<code>// O Autenticar devolve null...</code>	Comentário que vem do gabarito. Ele fica no código de propósito: é a única linha do arquivo que explica por que existe o <code>if</code> logo abaixo.
<code>if (achado == null)</code>	<code>null</code> é a ausência de objeto. Compare com

Linha	Por que ela está aí
	<code>null</code> , nunca com <code>""</code> — <code>achado</code> não é texto, é um <code>Usuario</code> (ou o lugar vazio onde um <code>Usuario</code> estaria). <code>achado == ""</code> nem compila.
<pre>// M7 + MessageBox.Show("Usuario ou senha incorretos.", "Acesso negado", ...)</pre>	A segunda mensagem do manual. Ela é propositalmente vaga: não diz <i>qual</i> dos dois está errado. Contar isso ajudaria quem está testando logins alheios.
<code>MessageBoxIcon.Error</code> na M7	Aqui o ícone é de erro: o usuário tentou entrar e não conseguiu. A diferença de ícone entre M6 e M7 é o que faz as duas mensagens não parecerem a mesma coisa.
<code>txtSenha.Text = "";</code>	Limpa só a senha. O login provavelmente está certo — quem erra, erra a senha — e mandar redigitar o nome de usuário a cada tentativa maltrata quem trocou uma letra.
<code>txtSenha.Focus();</code>	Põe o cursor piscando dentro do campo de senha. Sem esta linha, o usuário fecha a <code>MessageBox</code> e precisa clicar no campo antes de digitar. É uma linha de código para economizar um clique de todo mundo, todos os dias.
<code>return;</code> (dentro do segundo <code>if</code>)	O mesmo motivo do primeiro: sem ele, o programa segue e registra <code>null</code> na sessão. Dois <code>return</code> no meio do método

Linha	Por que ela está aí
	substituem um <code>else</code> grande e deixam o caminho de sucesso sem indentação nenhuma — quem lê o método de cima para baixo vê primeiro as recusas, depois o que acontece quando dá certo.
<pre>Sessao.UsuarioLogado = achado;</pre>	<code>Sessao</code> é uma classe <code>static</code>: você não escreve <code>new Sessao()</code> em lugar nenhum, porque existe uma sessão só para o programa inteiro. Esta linha é a única forma de a <code>frmPrincipal</code> descobrir quem entrou — ela lê <code>Sessao.UsuarioLogado</code> para escrever "Conectado como: ..." e para impedir que você exclua a sua própria conta.
a mesma linha, olhando a posição	Tem que vir antes do <code>ShowDialog</code>. O <code>Load</code> da tela principal roda dentro do <code>ShowDialog</code>, e a primeira coisa que ele faz é perguntar se há alguém conectado.
<pre>this.Hide();</pre>	Esconde a janela de login sem fechá-la. Ela precisa continuar existindo: no <code>Program.cs</code> está escrito <code>Application.Run(new frmLogin())</code>, ou seja, o programa vive enquanto este formulário viver. Fechá-lo aqui derrubaria o programa antes de a principal aparecer.

Linha	Por que ela está aí
<code>frmPrincipal tela = new frmPrincipal();</code>	<code>new</code> cria a janela na memória. Neste instante ela ainda não apareceu: existe, está montada, e ninguém a viu. É o mesmo <code>new</code> de qualquer objeto — um formulário também é um objeto.
<code>tela.ShowDialog();</code>	Abre a janela e para a execução nesta linha . A linha seguinte só vai rodar quando a tela principal for fechada. Com <code>Show()</code> no lugar, a execução seguiria adiante no mesmo instante e cairia direto no <code>this.Close()</code> logo abaixo.
<code>this.Close();</code>	Roda quando a principal já foi fechada. Fecha o login, que estava escondido, e com ele termina o <code>Application.Run</code> — o programa encerra de vez. Sem esta linha, a tela de login voltaria a aparecer, ainda com o login e a senha digitados.

O erro mais comum

1. **Esquecer o `return` depois da mensagem M7.** Sintoma exato: você digita a senha errada, aparece "Usuario ou senha incorretos." e, assim que você clica OK, aparece uma segunda mensagem — "Ninguém esta conectado." — e o programa fecha. A segunda mensagem vem do `frmPrincipal_Load`, que já vem pronto e é o cinto de segurança da tela principal. Se você viu essa dupla de mensagens, o problema não está na principal: está no `return` que faltou aqui.
2. **Esquecer a linha `Sessao.UsuarioLogado = achado;`**. Sintoma idêntico ao de cima — "Ninguém esta conectado." — só que agora com a senha **certa**. É a confusão mais cara desta noite, porque parece defeito do login e é defeito da sessão. Para separar os dois casos: se a mensagem M7 apareceu antes, falta `return`; se você entrou direto na "Ninguém esta conectado.", falta a linha da `Sessao`.
3. **Trocar `ShowDialog()` por `Show()`**. Sintoma: a tela principal pisca na tela e o programa inteiro fecha. Motivo: `Show` não espera; a execução continua na hora para `this.Close()`, e fechar o formulário do

`Application.Run` encerra tudo.

4. **Escrever `this.Close()` no lugar de `this.Hide()`.** Sintoma: varia de máquina para máquina — às vezes a principal abre sem o login por trás, às vezes o Visual Studio acusa erro ao chegar no `this.Close()` final, dizendo que a janela já foi descartada. Justamente por variar, este é o erro que mais custa tempo para achar. Se o comportamento do seu Entrar está estranho e você não consegue descrever o defeito em uma frase, confira esta linha primeiro.
5. **Trocar `||` por `&&` no primeiro `if`.** Sintoma: com o login preenchido e a senha vazia, aparece "Usuario ou senha incorretos." em vez de "Informe o usuario e a senha." — a mensagem errada do manual para a situação errada.

Como saber que funcionou

Antes de testar, garanta duas coisas: o `lblStatus` verde do TODO 1, e pelo menos uma conta na tabela. O `CriarBanco.sql` já gravou três — `ana`, `bruno` e `admin` — no fim do script. Confira no Workbench:

```
SELECT Id, Nome, Login, Senha FROM Usuario;
```

Se a tabela estiver vazia (você apagou as contas testando outra coisa) e o TODO 6 ainda não estiver pronto, grave uma de volta:

```
INSERT INTO Usuario (Nome, Login, Senha) VALUES ('Ana Souza', 'ana', '1234');
```

Agora, com F5:

- **Campos vazios.** Não digite nada e clique em Entrar. Tem que aparecer "Informe o usuario e a senha." com ícone de aviso, e você continua na tela de login. Nada foi ao banco.
- **Senha errada.** Digite `ana` e `9999`. Tem que aparecer "Usuario ou senha incorretos." com ícone de erro. Ao fechar a mensagem, o campo Usuario continua com `ana`, o campo Senha está vazio e o cursor está piscando dentro dele — digite sem clicar em lugar nenhum para conferir.
- **Entrada válida.** Digite `ana` e `1234`. A janela de login some, a tela principal abre e no alto dela aparece "Conectado como: Ana". Sai só o primeiro nome porque a `frmPrincipal` passa o `Nome` pelo `Regras.PrimeiroNome` antes de escrever — o nome gravado no banco é "Ana Souza". Esse "Ana" é a prova de que a `Sessao` foi preenchida: quem escreve esse texto é a `frmPrincipal`, lendo `Sessao.UsuarioLogado`. Se o TODO 12 ainda não estiver escrito, a lista fica vazia e o rodapé diz "TODO 12 ainda nao foi escrito." — isso é esperado e não é defeito seu.
- **Fechamento.** Feche a tela principal no X. O programa tem que **terminar inteiro**: no Visual Studio, a execução para sozinha. Se a tela de login reaparecer com os campos ainda preenchidos, está faltando o `this.Close()` final.
- **Com breakpoint.** Ponha um ponto de parada na linha `if (achado == null)` e rode as duas tentativas. Com a senha errada, passe o mouse sobre `achado`: aparece `null`. Com a senha certa, aparece `{Portaria.Usuario}` e você pode abrir a setinha para ver Id, Nome e Login vindos do banco. Esse é o teste que mostra, sem adivinhação, se o problema está neste método ou lá no TODO 8.

Para pensar

A linha `Sessao.UsuarioLogado = achado;` está escrita aqui, na tela, e não dentro do `Autenticar` — que é quem tem o objeto na mão primeiro. O que o `Autenticar` deixaria de conseguir fazer se essa linha morasse lá

dentro?

TODO 10 - Listar Todos

Arquivo	Dados/UsuarioDAO.cs
Indicador	I5 · I2
O que voce exercita	<code>switch</code> com <code>case</code> , dois rotulos para um bloco so e <code>default</code> ; <code>while (r.Read()); List<Usuario></code> com <code>Add</code> ; <code>new</code> de lista e de comando (o leitor nao usa <code>new</code> : ele vem do <code>ExecuteReader</code>); <code>try/finally</code> sem <code>catch</code> ; chamada de um metodo privado da propria classe
Depende de	TODO 7 (<code>MontarUsuario</code>) - e ele que vira cada linha lida em objeto. Por tras dele, a classe <code>Usuario</code> , cujas cinco propriedades o <code>MontarUsuario</code> preenche - dessas cinco, a unica que voce escreve e o <code>set</code> do <code>Nome</code> , no TODO 2; as outras quatro ja vem prontas. O <code>Conexao.Obter()</code> ja vem pronto.

O que este método faz

Ele recebe um numero - o parametro `ordem` - e devolve a lista completa das contas cadastradas, ja na ordem que esse numero pediu. `0` significa "por nome, de A a Z"; `1` significa "os mais recentes primeiro". Quem devolve e um `List<Usuario>`: um objeto por conta, com as cinco propriedades preenchidas.

Quem chama e o `CarregarLista()` da tela principal, numa linha so:

```
listaAtual = UsuarioDAO.ListarTodos(cboOrdem.SelectedIndex);
```

O `SelectedIndex` de um `ComboBox` e a posicao do item escolhido: o primeiro item vale 0, o segundo vale 1. No `cboOrdem` o primeiro item e "Por nome (A-Z)" e o segundo e "Mais recentes primeiro". Ou seja: o numero que chega aqui nao foi inventado por ninguem, ele e a posicao da linha que a pessoa escolheu no combo. Esse mesmo metodo e chamado de novo toda vez que o combo muda e toda vez que uma conta e excluida - a tela nunca conserta a lista na mao, ela pede a lista de novo.

Quando nao ha nenhuma conta no banco, este metodo devolve uma lista com zero itens. Ele nao devolve `null`.

Por que ele existe assim

Por que ele mora no DAO. Ele fala com o banco, e no projeto Portaria quem fala com o banco e o `UsuarioDAO`. A tela nao sabe que existe um `SELECT`; ela sabe pedir "a lista na ordem 0". Se amanha a coluna `DataCadastro` mudar de nome, o `frmPrincipal.cs` continua igual.

Por que a assinatura recebe `int` e nao `string`. Seria possivel escrever `ListarTodos(string criterio)` e chamar com `"nome"` ou `"data"`. Mas o que a tela tem de graca na mao e o `SelectedIndex`, que ja e um numero. Passar texto obrigaria a tela a traduzir a escolha antes de chamar - trabalho a mais, e um lugar a mais para errar de digitacao (`"Nome"` com N maiusculo nao e `"nome"`, e o erro so apareceria rodando). Com `int`, o contrato e curto: a posicao do item do combo e o codigo da ordem, e a traducao acontece dentro do `switch`, num lugar so.

Por que quem ordena e o banco. O `ORDER BY` faz o servidor entregar as linhas ja na ordem certa. A alternativa seria trazer tudo desordenado e ordenar em C# - o que significa escrever um algoritmo de ordenacao com lacos e trocas, algo que voce nao precisa fazer e que este sistema nao faz em lugar nenhum. Alem disso, o banco ordena antes de mandar os dados; um laco em C# so poderia comecar depois que tudo ja chegou.

Por que o `switch` vem antes de abrir a conexao. Escolher o texto do SQL nao depende do banco em nada. Fazer essa escolha primeiro deixa a conexao aberta pelo menor tempo possivel - abre, le, fecha. Conexao aberta e um recurso do servidor, nao um objeto qualquer.

Por que devolver lista vazia e nao `null`. Compare com o TODO 8: o `Autenticar` devolve `null` de proposito, porque "nao achei essa pessoa" e uma resposta que precisa ser diferente de "achei". Aqui nao ha esse problema: "nao ha ninguem cadastrado" tem uma representacao natural, que e a lista com zero itens. E quem chama percorre o resultado com um `for` - se voce devolvesse `null`, a linha `listaAtual.Count` explodiria com *NullReferenceException* antes de desenhar coisa alguma.

Por que `while` e nao `for`. Um `for` precisa saber quantas voltas vai dar. Voce nao sabe: pode haver 3 contas ou 300, e o leitor do MySQL nao tem `Count`. O `while (r.Read())` e o laco de "enquanto houver" - o mesmo formato da Aula 21, que repete ate a condicao ficar falsa em vez de contar. Repare que o `for` aparece sim neste sistema, mas do outro lado: no `CarregarLista()` da tela, onde a lista ja existe, ja tem `Count`, e a posicao `i` importa para numerar a linha.

O código

```

public static List<Usuario> ListarTodos(int ordem)
{
    List<Usuario> lista = new List<Usuario>();

    string sql;

    switch (ordem)
    {
        case 1:
            sql = @"SELECT Id, Nome, Login, Senha, DataCadastro
                    FROM Usuario
                    ORDER BY DataCadastro DESC, Id DESC";
            break;

        case 0:
        default:
            sql = @"SELECT Id, Nome, Login, Senha, DataCadastro
                    FROM Usuario
                    ORDER BY Nome";
            break;
    }

    MySqlConnection con = Conexao.Obter();

    try
    {
        con.Open();

        MySqlCommand cmd = new MySqlCommand(sql, con);
        MySqlDataReader r = cmd.ExecuteReader();

        while (r.Read())           // enquanto houver linha
        {
            lista.Add(MontarUsuario(r));
        }
    }
    finally { con.Close(); }

    return lista;
}

```

Linha a linha

Linha	Por que ela esta ai
<code>public static List<Usuario> ListarTodos(int ordem)</code>	<code>static</code> porque a tela chama escrevendo <code>UsuarioDAO.ListarTodos(...)</code> , sem criar objeto de DAO - o DAO nao guarda nada entre uma chamada e outra, ele so executa. O retorno e <code>List<Usuario></code> : varias contas, cada uma um objeto.
<code>List<Usuario> lista = new List<Usuario>();</code>	A lista nasce vazia na primeira linha do metodo, antes de qualquer coisa poder dar errado. Se o <code>SELECT</code> nao trouxer nenhuma linha, o <code>while</code> da zero voltas e e exatamente esta lista vazia que sai no <code>return</code> . Nao ha nenhum caminho por onde este metodo devolva <code>null</code> .
<code>string sql;</code>	Declarada antes do <code>switch</code> e sem valor . Ela precisa continuar existindo depois que o <code>switch</code> terminar, que e onde o <code>MySQLCommand</code> vai usar o texto. Se voce declarasse <code>string sql = ...</code> dentro de cada <code>case</code> , seriam duas variaveis diferentes, cada uma morrendo no seu <code>break</code> , e nenhuma delas existiria la embaixo.
<code>switch (ordem)</code>	Um valor, varios destinos. Um <code>if/else</code> faria o mesmo servico, mas o <code>switch</code> diz melhor a intencao: "conforme o numero que chegou, escolha um SQL". E o que voce vai colocar aqui e so a escolha do texto - nenhuma leitura de banco acontece dentro do <code>switch</code> .
<code>case 1:</code>	O 1 e a segunda linha do combo, "Mais recentes primeiro". O numero nao tem significado proprio; ele so vale porque e a posicao daquele item.
<code>ORDER BY DataCadastro DESC, Id DESC</code>	<code>DESC</code> e de <i>descending</i> , descendente: do maior para o menor, ou seja, da data mais nova para a mais velha. O <code>Id DESC</code> depois da virgula e o criterio de desempate, e ele nao e decoracao: a coluna <code>DataCadastro</code> e <code>DATETIME</code> , que guarda ate o segundo. Duas contas criadas no mesmo segundo empatam, e as tres contas de exemplo do <code>CriarBanco.sql</code> entram num <code>INSERT</code> unico - portanto empatam sempre. Sem o desempate, a ordem entre elas fica a criterio do servidor e pode mudar de uma execucao para outra. Com <code>Id DESC</code> , quem tem o <code>Id</code> maior aparece primeiro, e o <code>Id</code> sempre cresce.
<code>break;</code>	Fecha o caso. Em C# ele nao e opcional: sem <code>break</code> , o compilador acusa <i>CS0163 - o controle nao pode passar de um rotulo case para outro</i> .
<code>case 0: seguido de default:</code>	Dois rotulos, um bloco so. Isso vale em C# porque nao ha nenhum codigo entre os dois rotulos - eles apontam para o mesmo lugar. Escrever o <code>case 0:</code> sendo que o <code>default</code> ja pegaria o zero e uma escolha de leitura: quem le o metodo ve os dois codigos que existem de verdade, o 0 e o 1, e ve tambem para onde vai o resto.

Linha	Por que ela esta ai
default:	Duas razoes, e a primeira e do compilador. Sem o <code>default</code> , ele nao consegue provar que a variavel <code>sql</code> recebeu algum valor em todos os caminhos, e recusa compilar com <i>CS0165 - uso de variavel local nao atribuida 'sql'</i> . A segunda razao e de projeto: se um dia alguem acrescentar um terceiro item no ComboBox e esquecer de mexer aqui, o numero 2 cai no <code>default</code> e a lista sai por nome, em vez de sair quebrada.
@"SELECT ..."	A arroba antes das aspas faz a string ser literal : ela pode ocupar varias linhas e o que estiver escrito e o que vale, quebras de linha inclusive. Sem a arroba, o compilador acusa string nao terminada na primeira quebra, e voce teria que grudar pedacos com <code>+</code> . O SQL escrito em tres linhas alinhadas se le como se le no Workbench.
As cinco colunas, iguais nos dois <code>case</code>	<code>Id, Nome, Login, Senha, DataCadastro</code> . Sao as cinco que o <code>MontarUsuario</code> procura pelo nome. Se voce trazer quatro, o <code>r["Senha"]</code> nao acha a coluna e a leitura falha - e por isso que a consulta de conferencia do fim do <code>CriarBanco.sql</code> , que traz so quatro colunas, nao serve para copiar aqui.
Nenhum <code>@parametro</code> neste SQL	A regra 1 do DAO continua valendo: valor digitado por gente entra por parametro. So que aqui nao entra valor nenhum - o que muda entre os dois casos e o proprio <code>ORDER BY</code> , e nome de coluna nao pode ser parametro. Por isso os dois textos estao escritos por extenso, cada um dentro do seu <code>case</code> . Montar um so SQL grudando o nome da coluna com <code>+</code> seria exatamente o que a regra 1 proibe.
<code>MySqlConnection con = Conexao.Obter();</code>	O <code>Obter()</code> devolve a conexao fechada , ja com o endereco do banco. Quem abre e fecha e este metodo. E por isso que a senha do MySQL mora num arquivo so.
<code>try / con.Open();</code>	Abrir e o passo que pode falhar (servico parado, senha errada, banco inexistente). A partir do <code>Open</code> , tudo que vier precisa da garantia do <code>finally</code> .
<code>new MySqlCommand(sql, con)</code>	O comando precisa das duas coisas: o texto a executar e por qual conexao. Repare que o <code>sql</code> aqui e a variavel decidida la em cima - o <code>switch</code> ja fez o seu trabalho e nao aparece mais.
<code>cmd.ExecuteReader()</code>	<code>SELECT</code> devolve linhas , e quem devolve linhas e o <code>ExecuteReader</code> . O <code>ExecuteScalar</code> do TODO 4 traria so a primeira coluna da primeira linha; o <code>ExecuteNonQuery</code> do TODO 13 traria so a contagem. O leitor <code>r</code> comeca posicionado antes da primeira linha. Repare que ele nao e criado com <code>new</code> : quem o entrega e o proprio comando.
<code>while (r.Read())</code>	<code>Read()</code> faz duas coisas numa chamada so: avanca uma linha e devolve <code>true</code> se conseguiu. A primeira chamada pousa na primeira linha; quando acabam as linhas, ele devolve <code>false</code> e

Linha	Por que ela esta ai
	o laco termina sozinho. Nao existe <code>r.Count</code> para consultar antes.
<code>// enquanto houver linha</code>	O comentario esta no gabarito e vale a pena manter: ele traduz a condicao. <code>while (r.Read())</code> nao se le "enquanto ler", se le "enquanto ainda houver linha".
<code>lista.Add(MontarUsuario(r));</code>	A cada volta, o <code>MontarUsuario</code> le a linha em que o leitor esta parado e devolve um objeto novo , com <code>new</code> la dentro. Objeto novo a cada volta e o que impede a lista de terminar com o mesmo usuario repetido N vezes. E e aqui que se ve por que o TODO 7 existe: sem ele, estas cinco atribuicoes estariam duplicadas dentro deste <code>while</code> e tambem dentro do <code>Autenticar</code> .
<code>finally { con.Close(); }</code>	O <code>finally</code> roda de qualquer jeito: com sucesso, com erro no meio da leitura, com o <code>return</code> chegando. Sem ele, um erro no meio do laco deixaria a conexao aberta ate o programa fechar. Repare que aqui ha <code>try</code> e <code>finally</code> sem <code>catch</code> , e isso e permitido: quer dizer "eu nao trato o erro, mas garanto a limpeza". Quem trata e o <code>CarregarLista()</code> da tela, que envolve a chamada num <code>try/catch</code> e mostra o MessageBox. O DAO nao desenha tela.
<code>return lista;</code>	Esta fora do <code>try</code> , depois do <code>finally</code> . Quando a lista chega a quem pediu, a conexao ja esta fechada.

O erro mais comum

- **Esquecer o `default`**. O projeto nem compila. A mensagem e *CS0165 - uso de variavel local nao atribuida 'sql'*, e o sublinhado vermelho aparece la embaixo, na linha do `new MySqlCommand(sql, con)` - longe do `switch`, que e onde esta a causa. Solucao: acrescentar o `default:` colado no `case 0:`.
- **Usar `if (r.Read())` em vez de `while`**. Acontece com quem escreveu o TODO 8 antes e copiou o bloco. Compila e roda. O sintoma e mudo: a tela mostra **um** usuario so, sempre o primeiro da ordem escolhida, e o rodape diz "1 usuario(s) cadastrado(s)" mesmo com cinco contas no banco. Se voce vir isso, va direto nesta linha.
- **Declarar `string sql = ...` dentro dos `case`**. Erro de compilacao dizendo que `sql` nao existe no contexto atual, na linha do `MySqlCommand`. A declaracao tem que ficar antes do `switch`.
- **Trazer quatro colunas em vez de cinco**, copiando a consulta de conferencia do `CriarBanco.sql` (aquela que nao traz a `Senha`). Compila. Ao abrir a tela principal aparece o MessageBox "Nao consegui ler a lista" com uma mensagem em ingles reclamando que nao encontrou a coluna nos resultados. Volte no SQL e confira as cinco.
- **Esquecer o `break` do `case 1`**. *CS0163*, com a mensagem sobre passar de um rotulo case para outro. Nao compila.
- **Tentar ordenar em C#**. Se voce se pegar escrevendo um laco que compara nomes e troca posicoes na `List`, pare: essa e a tarefa do `ORDER BY`, e ele ja foi escrito.

Como saber que funcionou

1. Rode o `CriarBanco.sql` se ainda nao rodou, e depois cadastre pela tela de cadastro **uma conta nova**, por exemplo `Zilda Prado` / login `zilda`. Agora o banco tem quatro contas: as tres de exemplo (Ana Souza, Bruno Lima, Administrador Geral) mais a sua.
2. Entre com `ana` / `1234`. A tela principal abre ja com o combo em "Por nome (A-Z)", porque o `Load` coloca `SelectedIndex = 0`. A lista tem que sair assim, nesta ordem: **Administrador Geral, Ana Souza, Bruno Lima, Zilda Prado**. (Administrador vem antes de Ana porque na comparacao letra a letra o `d` vem antes do `n`.)
3. Troque o combo para "Mais recentes primeiro". A lista tem que virar: **Zilda Prado** em cima - ela e a unica com data diferente - e depois **Administrador Geral, Bruno Lima, Ana Souza**, que e o desempate por `Id` de tras para a frente, ja que as tres entraram no mesmo `INSERT` e tem a mesma data ate o segundo.
4. Nas duas ordens o rodape tem que dizer a mesma coisa:
`4 usuario(s) cadastrado(s) - ultimo: Zilda Prado`. Se o "ultimo" mudar quando voce troca o combo, o problema nao esta aqui, esta no `UltimoCadastrado` - ele deve varrer por `Id`, sem confiar na ordem que recebeu.
5. **No Workbench**, rode as duas consultas exatas e compare com a tela, linha por linha:

```
SELECT Id, Nome, Login, Senha, DataCadastro FROM Usuario ORDER BY Nome;  
SELECT Id, Nome, Login, Senha, DataCadastro FROM Usuario ORDER BY DataCadastro DESC, Id DESC;
```

Se o Workbench e a tela discordam, o defeito esta no C#. Se os dois concordam mas a ordem nao e a que voce esperava, o defeito esta no seu `ORDER BY` - e o que voce esperava e que precisa ser revisto.

1. **Com breakpoint:** ponha um na linha `lista.Add(MontarUsuario(r));` e rode com F5. Ele tem que parar uma vez para cada conta - quatro paradas, quatro F5. Se parar uma vez so, voce escreveu `if` no lugar de `while`. Se nao parar nenhuma, o `SELECT` nao trouxe linha nenhuma - confira o nome da tabela.

Para pensar

O `case 0:` e o `default:` levam ao mesmo bloco e produzem o mesmo SQL. Apagar a linha `case 0:` deixaria o metodo com uma linha a menos e com exatamente o mesmo comportamento em toda execucao possivel. Entao por que ela esta escrita ali?

TODO 11 - UltimoCadastrado

Arquivo	Regras/Regras.cs	
Indicador	I2 · I3 · I4	
O que voce exercita	<code>foreach</code> , <code>if</code> com <code>\</code>	<code>\</code> , comparacao de numeros inteiros, acesso por indice (<code>lista[0]</code>), devolver <code>null</code>
Depende de	Para COMPILAR: nada. Para VER o resultado na tela: TODO 10 (<code>ListarTodos</code>) e TODO 12 (<code>CarregarLista</code>). A propriedade <code>Id</code> do <code>Usuario</code> ja vem pronta no esqueleto - ela nao e um TODO.	

O que este método faz

Ele recebe uma lista de usuarios que ja esta na memoria e devolve UM usuario dessa lista: aquele que tem o maior `Id`. Como o `Id` e gerado pelo banco em ordem crescente a cada conta nova, o maior `Id` da lista e sempre a conta cadastrada por ultimo. O metodo nao abre conexao, nao mostra mensagem, nao ordena e nao muda nada dentro da lista - ele percorre, compara e responde.

Quem chama ele e o `CarregarLista` da tela principal (TODO 12), logo depois de desenhar a `ListBox`. Com a resposta na mao, a tela monta o rodape: `3 usuario(s) cadastrado(s) - ultimo: Maria Silva`. Quando a lista chega vazia, nao existe "ultimo" nenhum para devolver, e o metodo devolve `null` - a tela testa esse `null` e escreve `Nenhum usuario cadastrado.` no lugar do texto do rodape.

Por que ele existe assim

Por que ele mora no `Regras.cs` e nao no `UsuarioDAO.cs`. A pergunta "quem foi o ultimo?" tambem pode ser respondida com SQL: `SELECT ... ORDER BY Id DESC LIMIT 1`. So que, quando o `CarregarLista` precisa dessa resposta, a lista inteira ja veio do banco e ja esta na memoria, dentro do campo `listaAtual`. Ir ao banco de novo seria abrir conexao, mandar comando, ler resposta e fechar conexao - tudo isso para descobrir algo que ja esta na sua frente. Entao a resposta se calcula em cima do que ja se tem, e calcular em cima do que ja se tem e trabalho de regra, nao de acesso a dados.

Tem uma consequencia pratica nisso: como este metodo nao toca no MySQL, voce consegue conferir se ele esta certo com o servidor desligado. E a mesma razao pela qual o `ValidarLogin` (TODO 3) mora aqui. Aquela frase do topo do arquivo - "o `Regras.cs` nao abre banco e nao desenha tela" - e o que voce esta cumprindo agora.

Por que ele recebe a lista como parametro. O metodo nao sabe, e nao precisa saber, de onde a lista veio. Pode ter vindo do banco, pode ter sido montada a mao para um teste. Ele recebe `List<Usuario>` e devolve `Usuario`: entra dado, sai resposta. Se ele fosse buscar a lista sozinho, ele deixaria de ser conferivel sem banco - e voltaria a ser um metodo de DAO com nome de regra.

Por que ele devolve o `Usuario` inteiro e nao so o nome. Seria possivel devolver `string` com o nome, ja que e o nome que aparece no rodape hoje. Mas quem decide o que exibir e a tela, nao a regra. Devolvendo o objeto completo, a tela escolhe: hoje usa `ultimo.Nome`, e se amanha o rodape passar a mostrar a data do ultimo cadastro, ela usa `ultimo.DataCadastro` sem que uma linha deste metodo mude. Regra responde "quem"; tela decide "como mostrar".

Por que `static`. Não existe nenhum dado guardado dentro de um objeto `Regras` - não há nada para o objeto lembrar entre uma chamada e outra. Sendo `static`, a tela chama `Regras.UltimoCadastrado(listaAtual)` direto, sem `new Regras()`. Todos os métodos deste arquivo seguem a mesma escolha, pelo mesmo motivo.

Por que o critério é o `Id` e não a `DataCadastro`. Os dois crescem com o tempo, mas a `DataCadastro` pode EMPATAR: é um `DATETIME`, com precisão de segundo, e duas contas criadas no mesmo segundo têm exatamente o mesmo valor. Aparecendo empate, "o maior" deixa de ser um só, e você teria de inventar um critério de desempate. O `Id` nunca empata - o banco garante isso, porque ele é chave primária e `AUTO_INCREMENT`, e cada conta nova recebe um número maior que todos os anteriores. Repare que o próprio SQL do TODO 10 reconhece esse problema: quando ordena pelos mais recentes, ele escreve `ORDER BY DataCadastro DESC, Id DESC` - a segunda coluna está lá justamente para desempatar a primeira. Aqui você não precisa de desempate nenhum, porque começou pela coluna que não empata.

Por que percorrer, se a lista já vem ordenada. Ela vem ordenada, mas nem sempre pelo que interessa. A `ComboBox` da tela principal tem duas opções, e o TODO 10 responde a cada uma com um `ORDER BY` diferente: por nome de A a Z (opção 0) ou pelos mais recentes primeiro (opção 1). Se este método pegasse `lista[0]` e devolvesse, ele acertaria na opção 1 e erraria na opção 0 - e erraria calado, mostrando um nome plausível no rodapé. Percorrendo e comparando, a resposta é a mesma nas duas ordens, que é como tem que ser: quem foi o último a se cadastrar não muda porque você mexeu numa `ComboBox`.

O código

```
public static Usuario UltimoCadastrado(List<Usuario> lista)
{
    if (lista == null || lista.Count == 0)
    {
        return null;
    }

    Usuario ultimo = lista[0];

    foreach (Usuario u in lista)
    {
        if (u.Id > ultimo.Id)
        {
            ultimo = u;
        }
    }

    return ultimo;
}
```

Linha a linha

Linha	Por que ela esta ai			
<code>public static Usuario UltimoCadastrado(List<Usuario> lista)</code>	O tipo de retorno e <code>Usuario</code> : sai um objeto da lista, nao um numero nem um texto. <code>static</code> porque nao ha nada para um objeto <code>Regras</code> guardar. O parametro e a lista inteira, porque so vendo todos da para saber qual e o maior.			
<code>`if (lista == null \</code>	\	lista.Count == 0)`	Sao dois desastres diferentes na mesma guarda. <code>lista == null</code> e "nao me mandaram lista nenhuma"; <code>lista.Count == 0</code> e "me mandaram uma lista, mas nao tem ninguem dentro". Os dois impedem o <code>lista[0]</code> de mais abaixo de funcionar.	
a ordem dentro do \	\	,	O \	\
				lista.Count` - que, numa lista nula,

Linha	Por que ela esta ai			
				quebraria o programa. Invertendo as duas partes, a guarda deixa de proteger e passa a causar o erro que deveria evitar.
<pre>return null;</pre>	Nao existe resposta certa para "quem foi o ultimo de uma lista vazia". <code>null</code> significa exatamente "nao ha resposta", e o <code>CarregarLista</code> sabe ler isso: ele faz <code>if (ultimo == null)</code> e escreve "Nenhum usuario cadastrado.". Devolver um <code>new Usuario()</code> em vez de <code>null</code> faria o rodape imprimir <code>ultimo:</code> seguido de nada - e a tela nem teria como perceber que aquilo e um usuario que nao existe.			

Linha	Por que ela esta ai
<pre>Usuario ultimo = lista[0];</pre>	O candidato inicial. Ele precisa ser alguem REAL da lista, e nao um <code>Usuario</code> recém-criado, para que a resposta final seja sempre alguem que esta mesmo cadastrado. E o primeiro lugar a quebrar sem a guarda de cima: numa lista vazia, <code>lista[0]</code> dispara <code>ArgumentOutOfRangeException</code> - e numa lista nula, ele nem chega a procurar a posicao 0.
<pre>lista[0]</pre>	O mesmo colchete que voce usa em <code>listaAtual[i]</code> no <code>TODO 12</code> - e o acesso por posicao da <code>List<T></code>, e a primeira posicao e a 0, nao a 1.
<pre>foreach (Usuario u in lista)</pre>	Aqui a <code>POSICAO</code> nao importa, so o conteudo: voce quer olhar todo mundo, e tanto faz quem esta em que lugar. Por isso <code>foreach</code>, que

Linha	Por que ela esta ai
	entrega um usuario por vez sem voce cuidar de contador. Compare com o <code>for</code> do <code>TODO</code> 12: la a posicao importa (o numero <code>i + 1</code> aparece na linha e o indice serve para o botao Excluir), e por isso la o laco e outro.
o primeiro item ser comparado com ele mesmo	Na primeira volta, <code>u</code> e o proprio <code>lista[0]</code>, que ja esta em <code>ultimo . u.Id > ultimo.Id</code> da falso e nada acontece. E uma volta a toa, sem efeito nenhum - trocar isso por um laco que pula o primeiro item so acrescentaria complicacao.
<code>if (u.Id > ultimo.Id)</code>	O coracao do padrao "maior item": <code>ultimo</code> guarda o campeao ate agora, e cada usuario e comparado contra ele. Compare <code>Id</code> com <code>Id</code> - numero com numero. Comparar

Linha	Por que ela esta ai
	<pre>u.Id > ultimo.Nome.Length</pre> , ou trocar <code>Id</code> por <code>Nome</code>, responde outra pergunta.
<pre>> e nao >=</pre>	Com <code>></code>, em caso de empate o primeiro encontrado permanece. Com <code>Id</code> isso nunca acontece, porque ele nao se repete - mas o <code>></code> deixa escrito qual seria a decisao se acontecesse, e e o habito que voce quer levar para os campos que empatam.
<pre>ultimo = u;</pre>	Novo campeao. O que muda aqui e para onde <code>ultimo</code> aponta - o objeto nao e copiado nem alterado. A lista continua exatamente como chegou; este metodo nao mexe em nada de quem o chamou.
<pre>return ultimo; FORA do foreach</pre>	So depois de olhar TODOS e que se sabe quem e o maior. Um <code>return</code> dentro do laco encerraria o metodo na

Linha	Por que ela esta ai
	primeira volta, e a resposta seria sempre o primeiro item - o erro voltaria a depender da ordem da ComboBox.

O erro mais comum

1. Devolver `lista[0]` (ou o ultimo item) sem percorrer. E o engano que mais aparece, porque ele quase funciona. Voce testa com a ComboBox em "mais recentes primeiro", o nome do rodape bate, e voce da o metodo por pronto. Sintoma: troque a ComboBox para a ordem por nome e o rodape passa a mostrar outra pessoa - normalmente alguem cujo nome comeca com A. Um rodape que muda de nome quando voce mexe na ordenacao esta dizendo que o metodo nao esta comparando nada.

2. Esquecer a guarda da lista vazia. Este e o erro silencioso, e por um motivo curioso: no funcionamento normal do sistema a lista NUNCA chega vazia aqui. Voce so entra na tela principal depois de o login dar certo, e quem acabou de entrar esta na lista - ou seja, voce pode terminar o projeto inteiro sem esse defeito dar as caras uma vez sequer. Ele aparece quando a lista vem vazia por outro caminho: com o programa aberto e conectado, alguem apaga as linhas da tabela pelo Workbench e voce troca a ComboBox, forçando o `CarregarLista` a rodar de novo. Sintoma: a caixa "Nao consegui ler a lista." com a mensagem `Index was out of range` (ou `ArgumentOutOfRangeException`) - o texto aponta o dedo para o banco, e o problema esta aqui: `lista[0]` foi buscar a posicao 0 de uma lista que nao tem posicao nenhuma. A guarda existe porque o metodo tem que responder certo para qualquer lista que lhe entreguem, e nao apenas para as que a tela costuma entregar.

3. Começar com `Usuario ultimo = null;` e so depois comparar. Sintoma: `NullReferenceException` na primeira volta do laco, porque `ultimo.Id` esta pedindo o `Id` de um objeto que nao existe. E por isso que o candidato inicial e `lista[0]`, um usuario de verdade, e nao o vazio.

4. Colocar o `return` dentro do `foreach`. Sintoma parecido com o do erro 1 - o rodape volta a depender da ordenacao - mas a causa e outra: o laco existe, roda uma vez e o metodo termina antes de comparar o resto.

5. Trocar `>` por `<`. Sintoma: o rodape mostra sempre o PRIMEIRO usuario que voce cadastrou, aquele de `Id` 1, em qualquer ordenacao. Se o nome do rodape esta constante e e o do usuario mais antigo, olhe o sinal.

Como saber que funcionou

Deixe a tela principal aberta, com a lista carregada, e faca o teste em tres passos.

1. A resposta nao pode depender da ordenacao. Anote o nome que aparece depois de `ultimo:` no rodape. Troque a ComboBox de ordem, e depois volte para a primeira opcao. O numero de usuarios e o nome do rodape tem que continuar exatamente os mesmos nas duas ordens - so a ordem das linhas da ListBox muda. Se o nome mudar junto, seu metodo esta devolvendo um item pela posicao, nao pelo maior `Id`.

1. Confira contra o banco. No Workbench, rode:

```
sql SELECT Id, Nome FROM Usuario ORDER BY Id DESC LIMIT 1;
```

O `Nome` que voltar tem que ser, letra por letra, o mesmo que esta no rodape.

1. **Cadastre alguém e veja o rodape trocar.** Clique em Sair. Repare que o programa inteiro termina: fechar a tela principal fecha a tela de login junto, porque o login so estava escondido esperando ela. Rode de novo (F5), clique em Criar conta, crie uma conta com um nome facil de achar (por exemplo `Zulmira Teste`), entre no sistema e olhe o rodape: ele tem que mostrar `ultimo: Zulmira Teste` - inclusive na ordenacao por nome, onde essa pessoa aparece la embaixo na ListBox. O contador de usuarios tambem tem que ter subido em um.

Sobre a lista vazia: nao adianta procurar esse caso na tela. O `CriarBanco.sql` ja deixa tres contas na tabela, e mesmo que a tabela estivesse vazia voce nao chegaria a tela principal - sem nenhuma conta cadastrada, nenhum login e aceito. Confira o caminho do `null` de outro jeito: marque um breakpoint na linha `Usuario ultimo = Regras.UltimoCadastrado(listaAtual);` do TODO 12 e confirme que, com a lista cheia, o que volta e um objeto e nao `null`. A outra metade voce confere lendo o proprio codigo: com `lista.Count == 0`, o metodo devolve `null` antes de encostar em `lista[0]`.

Para pensar

Este metodo devolve o `Usuario` inteiro, mas o unico pedaco que a tela usa hoje e o `Nome`. Se ele devolvesse direto uma `string` com o nome, o `CarregarLista` ficaria uma linha mais curto - entao por que a apostila insiste em devolver o objeto? E, se ele devolvesse `string`, como a tela avisaria a diferenca entre "a lista esta vazia" e "o ultimo cadastrado se chama nada"?

TODO 12 - CarregarLista

Arquivo	Telas/frmPrincipal.cs
Indicador	I3 · I5
O que voce exercita	<code>for</code> com indice, indexador de <code>List<T></code> (<code>listaAtual[i]</code>), <code>PadLeft</code> / <code>PadRight</code> , <code>ToString</code> com formato de data, teste de <code>null</code> , <code>try</code> / <code>catch</code> , <code>ListBox.Items</code>
Depende de	TODO 10 (<code>UsuarioDAO.ListarTodos</code>) e TODO 11 (<code>Regras.UltimoCadastrado</code>). Indiretamente tambem do TODO 7 (<code>MontarUsuario</code>), que e quem transforma cada linha do banco em objeto, e do TODO 2 (a propriedade <code>Nome</code>), que e quem garante que o nome chegue aqui sem espacos nas pontas.

O que este método faz

`CarregarLista` e o metodo que desenha a tela principal. Ele nao recebe parametro nenhum e nao devolve nada. Sozinho, ele olha em que posicao esta o ComboBox de ordem, pede ao banco a lista de usuarios naquela ordem, guarda essa lista no campo `listaAtual`, apaga o que estiver na ListBox e escreve linha por linha - cada linha numerada, com o nome, o login entre parenteses e a data de cadastro. No fim, escreve no rodape quantos usuarios existem e quem foi o ultimo a se cadastrar.

Nao existe botao "Atualizar" nesta tela. Quem chama este metodo sao tres lugares: o `frmPrincipal_Load`, que ja vem pronto e termina com `cboOrdem.SelectedIndex = 0;` (essa linha muda o ComboBox, e mudar o ComboBox dispara o evento); o `cboOrdem_SelectedIndexChanged`, que tambem ja vem pronto e so tem uma linha dentro, a chamada a `CarregarLista()`; e o TODO 14, o botao Excluir, que chama `CarregarLista()` depois de apagar alguem para a tela mostrar a situacao nova. Ou seja: toda vez que a tela precisa ser redesenhada, e este metodo que roda.

Por que ele existe assim

Este metodo mora em `frmPrincipal.cs` porque ele e o unico do projeto que precisa saber ao mesmo tempo de tres coisas de tela: o `cboOrdem`, o `lstUsuarios` e o `lblRodape`. O `UsuarioDAO` sabe falar com o banco mas nao enxerga nenhum controle da tela; o `Regras` nao abre banco nem desenha tela, so recebe valores e devolve resposta. Formatar `" 1 Maria Silva (maria) 02/09/2026"` - com o numero encostado a direita e as colunas de largura fixa - e decisao de apresentacao, e apresentacao mora na tela. Se voce colocasse essa formatacao dentro do DAO, o dia em que a lista precisasse aparecer noutro lugar (num relatorio, por exemplo) voce teria que reescrever o DAO inteiro.

A assinatura `private void CarregarLista()` tem duas escolhas dentro dela, e as duas sao de proposito.

`private` porque nenhuma outra tela chama este metodo. Ele so e util para quem tem o `lstUsuarios` na frente, e isso e so o `frmPrincipal`. Deixar `public` seria abrir uma porta que ninguem usa.

Sem parametro e sem retorno porque ele nao e um calculo, e um redesenho. Um metodo de calculo recebe valores e devolve resposta - `Regras.ValidarLogin` faz isso. Um metodo de redesenho le o estado da tela e escreve na tela; o resultado dele nao e um valor, e a tela mudada. Repare no que isso poupa: como ele mesmo le `cboOrdem.SelectedIndex`, quem chama nao precisa saber nada sobre ordem. O `cboOrdem_SelectedIndexChanged` tem uma linha so. No TODO 14, a chamada tambem e uma linha so. Se

`CarregarLista` recebesse um `int ordem`, esses tres lugares teriam que ir ler o ComboBox antes de chamar - tres copias da mesma leitura, tres lugares para esquecer de mudar depois.

Falta o principal: por que `listaAtual` e um campo da classe e nao uma variavel dentro do metodo. Porque a ListBox guarda **texto**, nao objetos. Depois que voce faz `lstUsuarios.Items.Add(linha)`, aquela linha virou uma string qualquer; o `Id` do usuario nao esta mais la. Quando o usuario clicar na terceira linha e mandar excluir, o TODO 14 vai receber apenas o numero 2 (`SelectedIndex`) e vai precisar descobrir de quem e aquela linha. Ele descobre em `listaAtual[2]` - porque a lista foi desenhada na mesma ordem em que esta guardada. O campo `listaAtual` e a memoria da tela: e ele que liga a linha que o usuario ve ao objeto que o programa tem. Por isso ele e declarado la em cima, fora de qualquer metodo, e sobrevive entre um clique e outro.

O código

```
private void CarregarLista()
{
    try
    {
        listaAtual = UsuarioDAO.ListarTodos(cboOrdem.SelectedIndex);

        lstUsuarios.Items.Clear();

        for (int i = 0; i < listaAtual.Count; i++)
        {
            Usuario u = listaAtual[i];

            string linha = (i + 1).ToString().PadLeft(2) + " "
                + u.Nome.PadRight(24)
                + "(" + u.Login + ")".PadRight(16)
                + u.DataCadastro.ToString("dd/MM/yyyy");

            lstUsuarios.Items.Add(linha);
        }

        Usuario ultimo = Regras.UltimoCadastrado(listaAtual);

        if (ultimo == null)
        {
            lblRodape.Text = "Nenhum usuario cadastrado.";
        }
        else
        {
            lblRodape.Text = listaAtual.Count + " usuario(s) cadastrado(s)"
                + " - ultimo: " + ultimo.Nome;
        }
    }
    catch (Exception ex)
    {
        MessageBox.Show("Nao consegui ler a lista.\n\n" + ex.Message,
            "Erro", MessageBoxButtons.OK, MessageBoxIcon.Error);
    }
}
```

Linha a linha

Linha	Por que ela esta ai
<code>try</code> logo na primeira linha, cobrindo tudo	A leitura do banco pode falhar por motivos que nao sao culpa do seu codigo: o servico do MySQL desligado, a senha da conexao trocada, o cabo de rede fora. Quando isso acontece dentro de um evento do Windows Forms e ninguem captura, o programa fecha na cara do usuario sem dizer por que. O <code>try</code> transforma esse fechamento numa mensagem. Ele comeca antes da chamada ao DAO e vai ate o rodape porque nao ha nada aqui dentro que faca sentido executar se a leitura falhou.
<code>listaAtual = UsuarioDAO.ListarTodos(...)</code>	Sem <code>List<Usuario></code> na frente. Isso e uma atribuicao ao campo que ja existe la em cima da classe. Se voce escrever <code>List<Usuario> listaAtual = ...</code> , o C# cria uma variavel nova, local, que morre no fim do metodo - e o campo da classe continua com a lista vazia que foi criada na declaracao. O projeto compila, a lista aparece na tela e o botao Excluir quebra. Este e o erro mais caro deste TODO.
<code>cboOrdem.SelectedIndex</code> como argumento	<code>SelectedIndex</code> e a posicao do item escolhido no ComboBox: 0 para "Por nome (A-Z)", 1 para "Mais recentes primeiro". O TODO 10 espera exatamente esse numero e usa um <code>switch</code> para escolher a clausula <code>ORDER BY</code> . A tela nao monta SQL nenhum - ela so informa qual foi a escolha.
<code>lstUsuarios.Items.Clear();</code> antes do <code>for</code>	A ListBox nao se limpa sozinha. Sem esta linha, o segundo <code>CarregarLista</code> acrescenta as linhas embaixo das antigas: voce troca a ordem uma vez e a lista dobra de tamanho. E pior do que feio: a ListBox passa a ter mais linhas do que <code>listaAtual</code> tem itens, a numeracao para de bater com o indice, e o botao Excluir passa a apagar a pessoa errada - ou a estourar.
<code>for (int i = 0; i < listaAtual.Count; i++)</code>	O <code>foreach</code> te da o objeto e esconde a posicao. Aqui a posicao e a informacao mais importante do metodo: ela vira o numero que aparece na tela (<code>i + 1</code>) e e a mesma que o TODO 14 vai usar para achar o usuario. <code>i < listaAtual.Count</code> e nao <code><=</code> porque a primeira posicao e 0: numa lista de 3, as posicoes validas sao 0, 1 e 2. Com <code><=</code> voce pediria <code>listaAtual[3]</code> , que nao existe.
<code>Usuario u = listaAtual[i];</code>	Puxa o objeto da posicao <code>i</code> para uma variavel curta, so para nao repetir <code>listaAtual[i].Nome</code> , <code>listaAtual[i].Login</code> , <code>listaAtual[i].DataCadastro</code> tres vezes. <code>u</code> nao e uma copia do usuario - e o mesmo objeto; a variavel so aponta para ele.
<code>(i + 1).ToString()</code>	O <code>for</code> conta a partir do zero, gente conta a partir do um. Ninguem quer ver "0" na primeira linha da lista. Os parenteses em volta de <code>i + 1</code> sao obrigatorios, e o motivo e mecanico: o ponto do <code>.ToString()</code> gruda no que esta imediatamente antes dele. Sem os parenteses, o C# le

Linha	Por que ela esta ai
	<code>i + 1.ToString().PadLeft(2)</code> , ou seja, e o <code>1</code> sozinho que vira texto e ganha o <code>PadLeft</code> - vira <code>" 1"</code> - e so depois o <code>i</code> e concatenado na frente. A primeira linha comecaria com <code>"0 1"</code>, a segunda com <code>"1 1"</code>, a terceira com <code>"2 1"</code>. Compila, roda e sai errado; por isso os parenteses.
<code>.PadLeft(2)</code>	Empurra o numero para a direita dentro de duas casas: <code>" 1"</code>, <code>" 9"</code>, <code>"10"</code>. Assim a coluna do numero fica alinhada pela direita, como numeracao de lista deve ficar, e a coluna do nome comeca sempre na mesma altura mesmo quando a lista passa de nove pessoas.
<code>+ " "</code>	Dois espacos de folga entre o numero e o nome. E o unico espacamento escrito na mao aqui; todo o resto e feito por <code>PadRight</code>.
<code>u.Nome.PadRight(24)</code>	<code>PadRight</code> completa o texto com espacos a direita ate ele ter 24 caracteres. E o que cria a coluna: "Ana" tem 3 letras e recebe 21 espacos (<code>"Ana "</code>), "Maria Aparecida Souza" tem 21 e recebe 3 (<code>"Maria Aparecida Souza "</code>) - as duas com 24, entao o que vem depois comeca no mesmo lugar nas duas linhas. Um aviso: <code>PadRight</code> nunca corta. Um nome com 30 caracteres continua com 30 e empurra o resto da linha; a lista sai torta so naquela linha, e nao ha erro nenhum.
<code>("(" + u.Login + " ").PadRight(16)</code>	Os parenteses de fora nao sao enfeite. Sem eles - <code>("(" + u.Login + " ").PadRight(16)</code> - o <code>.PadRight</code> se aplica so ao pedaco <code>"")"</code>, porque ele gruda no que esta imediatamente antes dele. O resultado seria o parentese de fechar seguido de 15 espacos, sempre, independente do tamanho do login: a coluna da data sairia deslocada em cada linha. Com os parenteses, o <code>PadRight</code> recebe o texto ja montado <code>"(maria)"</code> e completa esse ate 16.
<code>u.DataCadastro.ToString("dd/MM/yyyy")</code>	<code>DataCadastro</code> e <code>DateTime</code>, nao texto. Sem formato, o <code>ToString</code> te devolve data e hora completas, com segundos, e a coluna estoura. <code>"dd/MM/yyyy"</code> pede dia, mes e ano, cada um com o numero de casas que voce escreveu. O <code>MM</code> e maiusculo de proposito: <code>mm</code> minusculo significa minutos. Como esta e a ultima coluna, ela nao leva <code>Pad</code> nenhum - nao ha nada depois dela para alinhar.
As quatro partes concatenadas numa <code>string</code> linha so	Poderiam ser quatro <code>Items.Add</code> seguidos, e sairiam quatro linhas na tela em vez de uma. A <code>ListBox</code> trata cada <code>Add</code> como um item independente - e cada item e uma linha, e e um <code>SelectedIndex</code>. A linha da tela tem que ser uma string, senao o indice da <code>ListBox</code> deixa de corresponder ao indice de <code>listaAtual</code>.

Linha	Por que ela esta ai
<code>lstUsuarios.Items.Add(linha);</code> dentro do <code>for</code>	Dentro, nao depois. Se estiver fora das chaves, so a ultima linha montada entra na lista - o <code>for</code> roda inteiro sobrescrevendo a variavel <code>linha</code> e no fim voce adiciona uma coisa so.
<code>Usuario ultimo = Regras.UltimoCadastrado(listaAtual);</code>	Chama o TODO 11 passando a mesma lista que acabou de ser desenhada. Repare que o rodape nao pergunta ao banco quem foi o ultimo: a informacao ja esta na memoria, e ir ao banco de novo seria uma segunda viagem para descobrir algo que voce ja tem. O TODO 11 percorre a lista guardando o de maior <code>Id</code> , e funciona mesmo quando a lista chega ordenada por nome.
<code>if (ultimo == null)</code>	O TODO 11 devolve <code>null</code> quando a lista esta vazia, e <code>null</code> quer dizer "nao ha ninguem", nao "ha alguem sem nome". Se ele devolvesse um <code>Usuario</code> vazio, o rodape escreveria <code>"ultimo: "</code> com o espaco em branco depois - uma mentira educada. Devolvendo <code>null</code> , ele obriga quem chama a decidir o que dizer nesse caso. Quem decide o texto e a tela, porque texto de tela nao mora em <code>Regras</code> . Sem este <code>if</code> , o <code>ultimo.Nome</code> do <code>else</code> explode com "Object reference not set to an instance of an object" na primeira vez que a tabela estiver vazia.
<code>listaAtual.Count + " usuario(s) cadastrado(s)"</code>	<code>Count</code> e um <code>int</code> e esta sendo somado com texto. Nesse caso o <code>+</code> nao soma: ele converte o numero para texto e junta. <code>2 + " usuario(s)"</code> da <code>"2 usuario(s)"</code> .
<code>" - ultimo: " + ultimo.Nome</code>	O nome completo, nao o <code>Regras.PrimeiroNome</code> . O <code>PrimeiroNome</code> e usado la em cima, no <code>lblConectado</code> , porque saudacao e informal; o rodape e informacao de cadastro, e informacao de cadastro se le por inteiro.
<code>catch (Exception ex)</code>	<code>Exception</code> e o tipo mais geral: pega qualquer erro que apareca la dentro. Voce nao sabe de antemao se vai ser erro de conexao, de permissao ou de tabela inexistente, e para o usuario da portaria a diferenca nao importa - importa a tela nao morrer.
<code>"Nao consegui ler a lista.\n\n" + ex.Message</code>	Duas informacoes na mesma caixa: a sua frase, que o usuario entende, e o <code>ex.Message</code> , que e o texto que o MySQL mandou. E o <code>ex.Message</code> que te diz, na hora de consertar, se o problema foi "Unable to connect to any of the specified MySQL hosts" ou "Table 'portariadb.usuario' doesn't exist". O <code>\n\n</code> pula duas linhas para separar visualmente as duas coisas.
Nao ha <code>finally</code> aqui	Quem abre a conexao e o <code>UsuarioDAO</code> , e e la dentro que existe o <code>finally { con.Close(); }</code> fechando ela acontece o que acontecer. Aqui nao ha nada aberto para fechar: esta tela

Linha	Por que ela esta ai
	so recebeu uma lista pronta. <code>finally</code> existe para devolver recurso, nao por costume.

O erro mais comum

Sao cinco, e os tres primeiros aparecem em quase toda turma.

1. Declarar o tipo na frente de `listaAtual`. Escrever `List<Usuario> listaAtual = UsuarioDAO.ListarTodos(...)` em vez de `listaAtual = UsuarioDAO.ListarTodos(...)`. Sintoma: nada de errado aparece agora - a lista desenha certinho, o rodape conta certo. O erro so surge quando voce terminar o TODO 14: voce seleciona uma linha, clica em Excluir e o programa quebra na linha `Usuario escolhido = listaAtual[i];` com "Index was out of range. Must be non-negative and less than the size of the collection.". Motivo: o campo da classe continuou com a lista vazia da declaracao, porque voce preencheu uma variavel local que morreu no fim do metodo. Como sair: apague `List<Usuario>` do comeco da linha. So isso.

2. Esquecer o `lstUsuarios.Items.Clear();` Sintoma imediato e visivel: abra a tela, troque o ComboBox de "Por nome (A-Z)" para "Mais recentes primeiro" e a lista dobra de tamanho, com a numeracao recomecando do 1 no meio. Troque de novo e triplica. Como sair: a linha vai antes do `for`, nunca dentro dele - dentro do `for` ela apagaria tudo a cada volta e no fim so a ultima pessoa apareceria.

3. Trocar o `for` por `foreach`. Sintoma: nao compila, porque nao existe `i` para escrever `(i + 1)`. Alguns resolvem criando um contador manual antes do `foreach`. Compila, e desenha certo - mas voce escreveu tres linhas para nao usar a construcao que ja te dava a posicao pronta, e continua sem o indice na hora de conferir se a linha 3 da tela e o item 2 da lista. Use o `for`.

4. Perder os parenteses do login. Digitar `("" + u.Login + ").PadRight(16)`. Sintoma: compila e roda, mas a coluna da data fica em zigue-zague - cada linha comeca a data num lugar diferente, conforme o tamanho do login. Como sair: compare com o gabarito e confira que o parentese que abre esta antes das aspas do `"("` e o que fecha esta depois do `)"`, antes do ponto do `PadRight`.

5. Escrever `"dd/mm/yyyy"` com m minusculo. Sintoma: no lugar do mes aparece um numero que nao tem nada a ver com a data. `mm` minusculo significa **minutos**, entao o que sai ali e o minuto em que aquela conta foi cadastrada - e como cada conta foi cadastrada num momento diferente, o numero do meio muda de linha para linha sem nenhuma logica. Quem se cadastrou depois do minuto 12 aparece com um "mes" que nem existe. O mes e `MM`.

E, antes de qualquer teste: apague a linha `lblRodape.Text = "TODO 12 ainda nao foi escrito.";` que veio no esqueleto. Se ela ficar, ela roda depois do seu codigo e sobrescreve o rodape que voce montou.

Como saber que funcionou

O teste tem quatro partes, nesta ordem.

A lista desenha. Rode o projeto (F5), entre com um login valido e olhe a tela principal. As linhas tem que aparecer numeradas a partir de 1, e as colunas tem que ficar em pe uma embaixo da outra: o nome comecando sempre na mesma altura, o `(login)` sempre na mesma altura, a data sempre na mesma altura. Esse alinhamento so acontece porque a ListBox esta com a fonte Consolas, em que toda letra ocupa a mesma largura - se voce trocar a fonte para Segoe UI no designer, o `PadRight` continua funcionando e a coluna sai torta do mesmo jeito, porque um "i" e mais estreito que um "m". Confira tambem se as colunas batem com o cabecalho em negrito logo acima da lista, o `lblCabecalho`, que ja vem escrito no designer: o `#`, o `NOME` e o `(USUARIO)`

batem exatamente com o que o seu código escreve. So a data cai um caractere a direita da palavra `CADASTRO` - isso e o texto fixo do cabeçalho, não defeito do seu código.

A troca de ordem redesenha, não acumula. Troque o ComboBox para "Mais recentes primeiro" e volte para "Por nome (A-Z)", umas três vezes. A quantidade de linhas tem que ser sempre a mesma; so a sequência muda. Se dobrou, falta o `Clear()`.

O rodapé confere com o banco. Anote o que o rodapé diz e abra o MySQL Workbench:

```
SELECT COUNT(*) FROM Usuario;  
SELECT Id, Nome FROM Usuario ORDER BY Id DESC LIMIT 1;
```

A primeira consulta tem que dar o mesmo número que o rodapé mostra. A segunda tem que devolver o mesmo nome que aparece depois de "ultimo:". Repare que são coisas diferentes: a lista está ordenada por nome, mas o "ultimo" é o de maior `Id`, então ele normalmente **não** é a última linha da tela. Se no seu teste ele coincidir com a última linha, cadastre mais um usuário com nome começando por A e olhe de novo.

O `catch` funciona. Este teste tem duas etapas, e a ordem importa: com o MySQL desligado você nem chega na tela principal, porque o login também lê o banco. Então faça assim. Primeiro, com o serviço do MySQL ligado, rode o projeto e entre normalmente. Com a tela principal já aberta, vá no Windows: tecla Windows + R, `services.msc`, ache o serviço MySQL80 e clique em Parar. Volte para a tela principal e troque o ComboBox de ordem - trocar a ordem chama o `CarregarLista` de novo, e agora ele não consegue ler. Tem que aparecer a caixa "Não consegui ler a lista." com o texto do erro do MySQL embaixo, e o programa tem que continuar de pé, com o botão Sair funcionando. Se em vez disso o Visual Studio parar num erro vermelho apontando a linha do `ListarTodos`, o seu `try` não está cobrindo a chamada ao DAO. Ligue o serviço de volta depois.

Para ver o ramo do `null` - "Nenhum usuário cadastrado." - você precisaria de uma tabela `Usuario` vazia, e uma tabela vazia te impede de fazer login. O jeito honesto de conferir esse caminho é no depurador: ponha um breakpoint na linha do `if (ultimo == null)`, rode até ele parar e confira no painel de variáveis que `ultimo` está preenchido; e no `Regras.UltimoCadastrado`, que você escreveu no TODO 11, confirme que a primeira guarda devolve `null` para lista vazia.

Para pensar

`CarregarLista` não recebe parâmetro nenhum: ele mesmo vai ler `cboOrdem.SelectedIndex` lá dentro. Se a assinatura fosse `private void CarregarLista(int ordem)`, o que precisaria mudar nos três lugares que chamam este método - o `frmPrincipal_Load`, o `cboOrdem_SelectedIndexChanged` e o botão Excluir do TODO 14 - e de quem passaria a ser a responsabilidade de saber em que ordem a lista deve aparecer?

TODO 13 - ExcluirConta

Arquivo	Dados/UsuarioDAO.cs
Indicador	I5
O que voce exercita	<code>DELETE</code> com <code>WHERE</code> , parametro nomeado <code>@id</code> , <code>ExecuteNonQuery</code> , <code>try / finally</code> , comparacao devolvida direto como <code>bool</code>
Depende de	Nada que voce tenha escrito. A <code>Conexao.Obter()</code> ja vem pronta no projeto. Quem vai chamar este metodo e o TODO 14, escrito depois dele

O que este método faz

Ele recebe um numero inteiro - o `Id` de uma conta - e apaga do banco a linha da tabela `Usuario` que tem esse `Id`. Nada mais. Ele nao pergunta se voce tem certeza, nao verifica se a conta e a sua, nao mostra mensagem nenhuma na tela. Recebe um numero, manda um comando ao MySQL, e devolve `true` ou `false` dizendo se alguma linha foi mesmo apagada.

Quem chama ele e o botao **Excluir o selecionado** da tela principal, no TODO 14. Quando voce chegar la, vai ver que a tela faz um monte de coisa antes: confere se ha uma linha selecionada, confere se a conta escolhida nao e a do proprio usuario conectado, e abre uma caixa Sim/Nao pedindo confirmacao. So depois de tudo isso ela chama `UsuarioDAO.ExcluirConta(escolhido.Id)`. Este metodo aqui e a ultima etapa da corrente, a que efetivamente toca no banco.

Por que ele existe assim

Por que ele mora no UsuarioDAO e nao no formulario. Existe uma divisao no projeto inteiro que vale a pena voce enxergar agora, porque ela cai na arguicao: a **tela decide**, o **DAO executa**. Perguntar "excluir a conta de Bruno Lima?" e decisao; ela mora na tela. Impedir que voce apague a si mesmo e decisao; mora na tela. Mandar `DELETE` para o MySQL nao e decisao, e execucao; mora aqui. Se voce misturar os dois, acaba com um `MessageBox` dentro do DAO - e o DAO nao tem como saber se existe alguem olhando para uma tela. Amanha o mesmo metodo pode ser chamado de outro lugar.

Por que a assinatura recebe `int id` e nao o objeto `Usuario` inteiro. A tela tem o objeto completo na mao (`Usuario escolhido = listaAtual[i];`) e seria tentador passar o objeto. Mas o `DELETE` so precisa de uma coisa: qual linha. Passar o objeto inteiro daria a entender que o `Nome`, o `Login` e a `Senha` importam para apagar - e nao importam. A assinatura de um metodo e um contrato: ela deve pedir exatamente o que ele usa, nem mais nem menos. Quem le `ExcluirConta(int id)` sabe na hora que a exclusao e pelo `Id`.

Por que pelo `Id` e nao pelo `Login`. O `Login` tambem serve, porque a tabela tem `UNIQUE (Login)` e por isso nao existem dois iguais. Mas o `Id` e a **chave primaria**: e o numero que o banco gerou para aquela linha e que nunca muda. Um login um dia pode ser corrigido ("brunolima" virou "bruno.lima"); o `Id` nao. Apagar pela chave primaria e a forma que continua certa depois de qualquer mudanca.

Por que devolve `bool` e nao `void`. O `ExecuteNonQuery` ja te entrega de graca a contagem de linhas afetadas. Devolver essa informacao custa uma linha e responde a uma pergunta real: *apagou mesmo?* Pode acontecer de a resposta ser `false` com o `WHERE` totalmente correto - por exemplo, se aquele `Id` ja tiver sido apagado por outra pessoa, em outra maquina, entre o momento em que a lista foi desenhada na tela e o

momento em que voce clicou. Seja honesto ao ler o TODO 14: la a tela chama

`UsuarioDAO.ExcluirConta(escolhido.Id);` **sem guardar a resposta**. O metodo devolve `bool` mesmo assim, porque a informacao existe e jogar fora e escolha de quem chama, nao do DAO.

Por que o esqueleto e igual ao dos outros quatro metodos que abrem conexao. Abrir conexao, montar SQL, criar comando, pendurar parametro, executar, fechar no `finally`. Voce ja escreveu isso quatro vezes ate aqui, nos TODO 4, 5, 8 e 10. A unica coisa nova neste TODO e **qual metodo executa**: nao e `ExecuteScalar` (que devolve um valor so) nem `ExecuteReader` (que devolve linhas), e sim `ExecuteNonQuery` - "execute algo que **nao e** uma consulta". `INSERT`, `UPDATE` e `DELETE` nao devolvem tabela nenhuma; devolvem um numero: quantas linhas voce mexeu.

O código

```
public static bool ExcluirConta(int id)
{
    MySqlConnection con = Conexao.Obter();

    try
    {
        con.Open();

        string sql = @"DELETE FROM Usuario
                      WHERE Id = @id";

        MySqlCommand cmd = new MySqlCommand(sql, con);
        cmd.Parameters.AddWithValue("@id", id);

        int linhas = cmd.ExecuteNonQuery();

        return linhas > 0;
    }
    finally { con.Close(); }
}
```

Linha a linha

Linha	Por que ela esta ai
<code>public static bool ExcluirConta(int id)</code>	<code>static</code> porque em lugar nenhum do projeto se escreve <code>new UsuarioDAO()</code> . A classe e uma caixa de ferramentas, nao um objeto com estado: voce chama <code>UsuarioDAO.ExcluirConta(7)</code> direto pelo nome da classe, igual aos outros quatro metodos publicos daqui - <code>LoginExiste</code> , <code>CriarConta</code> , <code>Autenticar</code> e <code>ListarTodos</code> . (O <code>MontarUsuario</code> tambem e <code>static</code> , mas e <code>private</code> : so o proprio DAO chama ele.)
<code>int id</code>	O unico dado que o <code>DELETE</code> precisa. Como e um numero inteiro, quem chama nao consegue mandar texto por engano - o compilador recusa antes de rodar.
<code>MySqlConnection con = Conexao.Obter();</code>	Fica FORA do <code>try</code>, e isso nao e estilo, e obrigacao. Uma variavel declarada dentro do <code>{ }</code> do <code>try</code> so existe dentro daquele bloco; o <code>finally</code> e outro bloco e nao enxergaria o <code>con</code> . Se voce mover esta linha para dentro do <code>try</code> , o projeto para de compilar com <code>CS0103 - o nome 'con' nao existe no contexto atual</code> .
<code>Conexao.Obter()</code>	Devolve uma conexao fechada , ja com o endereco do <code>portariadb</code> dentro. O endereco esta escrito num lugar so, no <code>Conexao.cs</code> . E por isso que nenhum metodo do DAO repete <code>server=localhost;...</code> : se a senha do MySQL mudar, muda la e o sistema inteiro acompanha.
<code>try</code>	Repare que nao ha <code>catch</code> aqui . O <code>try</code> deste metodo existe por causa do <code>finally</code> , nao por causa de um <code>catch</code> . Quem avisa o usuario que deu errado e a tela (o TODO 14 tem o <code>catch</code> com o <code>MessageBox</code>). O DAO so garante que a conexao nao fique aberta.
<code>con.Open();</code>	A conexao chegou fechada, entao a primeira coisa dentro do <code>try</code> e abrir. Esquecer esta linha nao da erro de compilacao - da erro so na hora de executar.
<code>string sql = @"DELETE FROM Usuario</code>	A arroba antes das aspas faz uma string verbatim : ela pode atravessar varias linhas sem <code>+</code> e sem <code>\n</code> . E o que deixa o SQL com a cara de SQL, com <code>DELETE</code> e <code>WHERE</code> alinhados um debaixo do outro. Os espacos que sobram dentro da string nao incomodam o MySQL - para ele, um espaco ou dez sao a mesma coisa.
<code>WHERE Id = @id";</code>	A linha mais importante do metodo. Sem ela o comando vira <code>DELETE FROM Usuario</code> puro, que apaga todas as linhas da tabela. Com ela, e como <code>Id</code> e a chave primaria, o comando atinge no maximo uma linha. Nao existe "desfazer" depois de um <code>DELETE</code> .
<code>@id</code> e nao <code>+ id</code>	Aqui vale a mesma regra dos outros metodos: valor entra por parametro nomeado, nunca colado com <code>+</code> . Sendo honesto com voce: um <code>int</code> realmente nao consegue carregar um

Linha	Por que ela esta ai
	comando escondido, entao neste caso especifico o <code>+</code> nao abria uma brecha de seguranca. A razao de usar <code>@id</code> assim mesmo e outra - a regra vale para todo metodo do DAO, sem excecao, e uma regra que tem excecao e uma regra que voce esquece justamente no dia em que o parametro deixa de ser numero.
<code>MySQLCommand cmd = new MySQLCommand(sql, con);</code>	O comando precisa de duas coisas: o texto do que fazer e por qual conexao sair. E a mesma construcao dos TODO 4, 5, 8 e 10 - nada de novo.
<code>cmd.Parameters.AddWithValue("@id", id);</code>	Preenche o buraco. O nome entre aspas tem que ser identico ao que esta no SQL, arroba inclusive. O <code>"@id"</code> (texto) e o <code>id</code> (o parametro do metodo) sao coisas diferentes que por acaso se parecem: o primeiro e o nome do buraco, o segundo e o valor que entra nele.
<code>int linhas = cmd.ExecuteNonQuery();</code>	<code>DELETE</code> nao devolve tabela (nao ha o que ler com um <code>MySqlDataReader</code>) nem um valor calculado (nao ha o que pegar com <code>ExecuteScalar</code>). Ele devolve a contagem de linhas afetadas . Guardar essa contagem numa variavel chamada <code>linhas</code> e o que permite responder a pergunta seguinte.
<code>return linhas > 0;</code>	<code>linhas > 0</code> ja e um <code>bool</code> - a comparacao produz verdadeiro ou falso sozinha. Escrever <code>if (linhas > 0) return true; else return false;</code> funciona, mas sao quatro linhas para dizer o que uma diz. Repare tambem que o <code>return</code> esta dentro do <code>try</code> .
<code>finally { con.Close(); }</code>	O <code>finally</code> roda de qualquer jeito : depois do <code>return</code> acima, e tambem se o MySQL recusar o comando no meio do caminho. E exatamente por isso que ele existe. Se voce puser o <code>con.Close()</code> como ultima linha do <code>try</code> , ele nunca executa - o <code>return</code> sai antes - e a cada exclusao voce deixa uma conexao aberta pendurada no servidor.

O erro mais comum

1. Esquecer o `WHERE`. E o erro caro deste TODO. O sintoma e assustador: voce clica em Excluir o selecionado uma vez, a lista volta **vazia**, o rodape diz "Nenhum usuario cadastrado.", e nenhum login funciona mais - nem o `admin`. Nao ha como desfazer. Para sair: rode o `CriarBanco.sql` de novo no Workbench (ele apaga e recria o banco com as tres contas de exemplo), conserte o SQL, e so entao teste outra vez. Antes de rodar qualquer `DELETE`, procure o `WHERE` com o dedo na tela.

2. Escrever `WHERE Id = id` (sem a arroba). Este e o irmao gmeo do erro 1, e engana ainda mais: **nao aparece erro nenhum, e a tabela esvazia do mesmo jeito**. Sem a arroba, o MySQL nao le `id` como "o valor que veio do C#" - le como **nome de coluna**. E existe uma coluna com esse nome na tabela: a `Id` (para o MySQL, nome de coluna nao distingue maiuscula de minuscula, entao `id` e `Id` sao a mesma coisa). O comando passa a dizer "apague as linhas em que a coluna Id e igual a ela mesma" - o que e verdade em **todas**

as linhas. O parametro que voce pendurou com o `AddWithValue` fica sobrando, sem ninguem usar, e ninguem reclama. A arroba nao e enfeite: e ela que separa "aqui entra um valor" de "aqui esta o nome de uma coluna". Se cair nessa, o conserto e o mesmo do erro 1: `CriarBanco.sql` de novo.

3. Usar `ExecuteScalar` no lugar de `ExecuteNonQuery`. Este e traicoeiro porque **a conta some da lista mesmo assim**. Sozinho, `int linhas = cmd.ExecuteScalar();` nem compila - o `ExecuteScalar` devolve `object`. Mas se voce copiar do TODO 4 o `Convert.ToInt32(...)` em volta, compila. O `ExecuteScalar` executa o `DELETE` normalmente, so que, como o comando nao produz valor nenhum, ele devolve `null`; o `Convert.ToInt32` transforma esse `null` em zero e o metodo devolve `false` para uma exclusao que deu certo. Como o TODO 14 ignora a resposta, voce so descobre no dia da arguicao. `DELETE` pede `ExecuteNonQuery`, sempre.

4. Esquecer o `con.Open()`. Nao da erro de compilacao. Da erro na execucao: uma `InvalidOperationException`, dizendo que a conexao nao esta aberta - que chega na tela dentro da caixa "Nao consegui excluir.". Ponha um breakpoint na linha do `ExecuteNonQuery` e confira se o `con.Open()` esta acima dela.

5. Declarar o `con` dentro do `try`. O projeto nao compila:

CS0103 - o nome 'con' nao existe no contexto atual, apontando para a linha do `finally`. A declaracao tem que ficar antes do `try`.

Como saber que funcionou

Primeiro, teste o SQL sozinho, no Workbench. Antes de o codigo C# existir, voce ja pode conferir a forma do comando. No Workbench, conectado no `portariadb`:

```
SELECT Id, Nome, Login FROM Usuario ORDER BY Id;
```

Anote os Ids que aparecem. Escolha um que **nao** seja o do `admin` - digamos que o `Bruno Lima` seja o `Id 2` - e rode:

```
DELETE FROM Usuario WHERE Id = 2;
```

O Workbench responde na barra de saida `1 row(s) affected`. Esse `1` e exatamente o numero que o `ExecuteNonQuery` vai devolver para a sua variavel `linhas`. Rode o `SELECT` de novo: Bruno sumiu, os outros continuam. Se em vez de `1 row(s) affected` aparecer `3 row(s) affected`, voce escreveu o comando sem `WHERE`.

Agora reponha o banco antes de continuar. Voce acabou de apagar o Bruno de verdade, e o proximo teste precisa dele de volta: rode o `CriarBanco.sql` no Workbench outra vez. Lembre que ele apaga o banco inteiro antes de recriar - as tres contas de exemplo voltam como estavam no primeiro dia, e qualquer conta que voce tenha cadastrado a mais some junto.

Depois, com o TODO 14 escrito, teste pelo sistema. Rode o projeto, entre com `ana` / `1234`, clique na linha do `Bruno Lima` na lista, clique em **Excluir o selecionado** e responda Sim na confirmacao. Tres coisas tem que acontecer ao mesmo tempo: a linha some da ListBox, o contador do rodape cai de `3 usuario(s) cadastrado(s)` para `2 usuario(s) cadastrado(s)` (o rodape mostra ainda, depois do traco, o nome do ultimo cadastrado), e o Workbench confirma:

```
SELECT Id, Nome, Login FROM Usuario ORDER BY Id;
```


Se quiser ver o numero com os proprios olhos, ponha um breakpoint na linha

`int linhas = cmd.ExecuteNonQuery();`, rode, faca a exclusao, e quando o programa parar aperte F10 uma vez e passe o mouse sobre `linhas`. Tem que aparecer `1`. Se aparecer `0`, o `Id` que chegou nao existe na tabela; se aparecer `3`, corra conferir o `WHERE`.

Depois do teste, recomponha o banco mais uma vez. Rode o `CriarBanco.sql` para as tres contas de exemplo voltarem - e deixe o sistema assim para a proxima aula. Rode sabendo que ele apaga o banco inteiro antes de recriar, e nao no meio de um teste que voce ainda quer conferir.

Para pensar

O metodo devolve `true` ou `false`, mas o botao Excluir o selecionado chama ele sem guardar essa resposta - o `bool` vai direto para o lixo. Em que situacao concreta, com o `WHERE` totalmente correto, esse retorno seria `false`, e o que a tela poderia mostrar de diferente se estivesse olhando para ele?

TODO 14 - btnExcluir_Click

Arquivo	Telas/frmPrincipal.cs
Indicador	I3 · I4 · I6
O que voce exercita	SelectedIndex, acesso por indice na List, comparacao de int, DialogResult, return antecipado, try/catch
Depende de	TODO 12 (CarregarLista, que enche o campo listaAtual) e TODO 13 (UsuarioDAO.ExcluirConta). Para testar de verdade voce tambem precisa do TODO 9, porque so se chega nesta tela pelo login.

O que este método faz

Este metodo e o que roda quando voce clica no botao "Excluir o selecionado". Ele nao recebe nada de util pelos parametros e nao devolve nada: tudo o que ele precisa saber ele vai buscar na propria tela. Primeiro ele pergunta a ListBox qual linha esta marcada. Com esse numero de linha ele acha o objeto `Usuario` correspondente dentro de `listaAtual`, o campo que o TODO 12 deixou preenchido. Depois ele passa por tres portoes antes de deixar qualquer coisa ser apagada: uma linha precisa estar selecionada, a conta escolhida nao pode ser a conta de quem esta conectado, e a pessoa precisa responder "Sim" na caixa de confirmacao. So depois dos tres ele manda o DAO apagar e manda a tela se redesenhar.

Ninguem chama este metodo por escrito. Quem chama e o Windows, no instante do clique. Isso muda a forma de pensar o metodo: ele nao tem um "chamador" que confere as coisas antes, nao ha ninguem entre o dedo do usuario e a primeira linha aqui dentro. Todas as recusos tem que estar dentro dele.

Por que ele existe assim

Por que este codigo mora na tela e nao no DAO. `UsuarioDAO.ExcluirConta` sabe apagar uma linha da tabela pelo Id, e so. Ele nao sabe que existe uma ListBox, nao sabe quem esta logado, nao sabe que existe uma caixa de confirmacao. Isso e proposital: o dia em que este sistema ganhar uma segunda tela que tambem apaga contas, o DAO continua servindo sem mudar uma virgula. As perguntas "quem voce selecionou?" e "voce tem certeza?" sao perguntas de tela, e por isso elas ficam na tela. A regra pratica que vale para o projeto inteiro: se a resposta depende de um controle visual (`lstUsuarios`, um botao, uma caixa de mensagem), o codigo e da tela; se depende so de dados, e do DAO ou de `Regras`.

Por que a assinatura e `(object sender, EventArgs e)`. Voce nao escolheu essa assinatura e nao pode muda-la. Ela e a forma que o Windows Forms exige de todo metodo ligado a um evento de clique — o `sender` seria o botao que foi clicado e o `e` traria detalhes do evento, e neste projeto voce nao usa nenhum dos dois. A ligacao entre o botao e este metodo ja vem feita no arquivo do designer; voce so preenche o corpo.

Por que a protecao contra o -1 e um `if` e nao um `try/catch`. Este metodo usa as duas ferramentas, em lugares diferentes, e a diferenca entre elas e um dos pontos do capitulo. O `if (i < 0)` protege de uma coisa que voce sabe que vai acontecer: mais cedo ou mais tarde alguem clica em Excluir sem ter escolhido uma linha. Isso nao e execucao, e uso normal do programa, e uso normal se trata com `if`. Ja o `try/catch` no fim protege do que voce nao consegue prever: o MySQL desligado, o cabo de rede solto, a senha do banco trocada. Envolver `listaAtual[i]` num `try/catch` para nao pensar no -1 funcionaria, mas ensinaria a coisa errada: `catch` nao e substituto de conferencia.

Por que o `try` cobre so duas linhas. Olhe onde ele comeca: depois de todas as decisoes. Ler `SelectedIndex`, pegar um item da lista e mostrar uma `MessageBox` nao dependem de banco nenhum, e nao podem falhar por causa do banco. As unicas linhas que saem do programa e vao falar com o MySQL sao `ExcluirConta` e o `CarregarLista` que vem logo atras. Um `try` que envolve o metodo inteiro esconde onde o risco esta; um `try` curto e uma documentacao de onde ele mora.

Por que a comparacao e por `Id`. `Nome` se repete (duas pessoas podem se chamar Ana Souza) e `Login` e texto, sujeito a maiuscula, minuscula e espaco. `Id` e o numero que o banco gerou, unico por definicao, e comparar dois `int` com `==` nao tem armadilha nenhuma.

O código

```

private void btnExcluir_Click(object sender, EventArgs e)
{
    int i = lstUsuarios.SelectedIndex;

    // M8
    if (i < 0)
    {
        MessageBox.Show("Escolha um usuario na lista primeiro.",
            "Ninguem selecionado", MessageBoxButtons.OK, MessageBoxIcon.Warning);
        return;
    }

    Usuario escolhido = listaAtual[i];

    // M9 - excluir a propria conta enquanto esta logado deixaria
    // a sessao apontando para alguem que nao existe mais.
    if (escolhido.Id == Sessao.IdUsuarioLogado())
    {
        MessageBox.Show("Voce nao pode excluir a sua propria conta "
            + "enquanto esta conectado.",
            "Nao permitido", MessageBoxButtons.OK, MessageBoxIcon.Warning);
        return;
    }

    DialogResult resposta = MessageBox.Show(
        "Excluir a conta de " + escolhido.Nome + "?",
        "Confirmar exclusao",
        MessageBoxButtons.YesNo, MessageBoxIcon.Question);

    if (resposta != DialogResult.Yes)
    {
        return;
    }

    try
    {
        UsuarioDAO.ExcluirConta(escolhido.Id);
        CarregarLista();
    }
    catch (Exception ex)
    {
        MessageBox.Show("Nao consegui excluir.\n\n" + ex.Message,
            "Erro", MessageBoxButtons.OK, MessageBoxIcon.Error);
    }
}

```

Linha a linha

Linha	Por que ela esta ai
<pre>int i = lstUsuarios.SelectedIndex;</pre>	<code>SelectedIndex</code> e a propriedade da <code>ListBox</code> que diz qual linha esta marcada , contando a partir do zero: a primeira linha da tela e o indice 0. Guardar o valor numa variavel em vez de perguntar a <code>ListBox</code> tres vezes tem duas razoes. A primeira e que o valor fica congelado: voce confere e usa exatamente o mesmo numero. A segunda e nome: <code>i</code> e de proposito a mesma letra do <code>for</code> do TODO 12, para voce enxergar que e o mesmo indice viajando de um metodo para o outro.
<pre>// M8</pre>	O comentario nao explica o codigo, ele numera a mensagem . M8 e o nome dessa recusa no manual do sistema que voce entrega junto com o projeto. Quando o professor perguntar "onde nasce a M8?", a resposta e esta linha.
<pre>if (i < 0)</pre>	Quando nenhuma linha esta selecionada, <code>SelectedIndex</code> não vale zero: vale <code>-1</code> . Zero ja e o nome da primeira linha, entao a <code>ListBox</code> precisava de outro valor para dizer "nada aqui", e escolheu o -1. Esta e a unica conferencia que impede a linha seguinte de explodir. Escrever <code>i < 0</code> em vez de <code>i == -1</code> e habito de defesa: qualquer negativo cai fora, nao so o -1.
<pre>MessageBox.Show(... MessageBoxButtons.OK, MessageBoxIcon.Warning);</pre>	Aviso de uma via: um botao OK, porque nao ha nada a decidir. O icone <code>Warning</code> (triangulo amarelo) diz "voce esqueceu de algo", diferente do <code>Error</code> (X vermelho) que fica reservado para falha de verdade, la embaixo no <code>catch</code> . O segundo texto, <code>"Ninguem selecionado"</code> , e o titulo da janelinha, nao o corpo.
<pre>return;</pre>	Sem ele, o codigo continuaria e chegaria em <code>listaAtual[i]</code> com <code>i</code> valendo -1 — que e exatamente o acidente que o <code>if</code> estava tentando evitar. Cada recusa deste metodo termina em <code>return</code> : o metodo tem uma unica saida boa, no fim, e varias saidas de recusa pelo caminho.
<pre>Usuario escolhido = listaAtual[i];</pre>	Os colchetes pedem a <code>List</code> o item que esta naquela posicao . Aqui e onde o TODO 12 se paga: como aquele <code>for</code> desenhou a <code>ListBox</code> percorrendo <code>listaAtual</code> do inicio ao fim, sem pular e sem trocar de ordem, a linha numero <code>i</code> da tela e o objeto numero <code>i</code> da lista. Nao existe busca, nao existe comparacao de texto — a posicao ja e a resposta.
<pre>Usuario escolhido (e nao int idEscolhido)</pre>	Voce guarda o objeto inteiro, nao so o Id, porque vai precisar de duas coisas dele: o <code>Id</code> para comparar com a sessao e para mandar ao DAO, e o <code>Nome</code> para escrever na confirmacao. Um objeto so, lido uma vez da lista.
<pre>if (escolhido.Id == Sessao.IdUsuarioLogado())</pre>	<code>Sessao</code> e a classe <code>static</code> que guarda quem entrou no sistema — ela ja vem pronta, voce so consulta. <code>IdUsuarioLogado()</code> devolve o Id de quem esta conectado (e devolve 0 se nao houver ninguem, o que aqui nunca acontece

Linha	Por que ela esta ai
	porque o <code>Load</code> da tela ja fechou a janela nesse caso). Se os dois numeros forem iguais, a conta selecionada e a sua.
// M9 - ... deixaria a sessao apontando para alguem que nao existe mais.	A razao da recusa, escrita no codigo. Nao e frescura de seguranga: <code>Sessao.UsuarioLogado</code> e um objeto que ficou na memoria. Apagar a linha do banco nao apaga esse objeto — a tela continuaria mostrando "Conectado como: Fulano" com o Fulano ja inexistente, e qualquer coisa feita dali para frente estaria em nome de um fantasma.
"Voce nao pode excluir a sua propria conta " + "enquanto esta conectado."	Duas partes coladas com <code>+</code> so porque o texto nao cabia numa linha de codigo. Na tela sai uma frase unica. Repare no espaco antes da aspas de fechamento da primeira parte: sem ele, sairia <code>contaenquanto</code> .
DialogResult resposta = MessageBox.Show(...)	Aqui o <code>MessageBox.Show</code> nao esta so avisando: ele esta perguntando , e por isso o retorno dele importa. <code>DialogResult</code> e o tipo desse retorno — um valor que diz qual botao foi apertado. Quando a caixa tem so OK, ninguem guarda o retorno porque nao ha o que decidir; quando ela tem Sim e Nao, jogar o retorno fora e apagar sem perguntar.
"Excluir a conta de " + escolhido.Nome + "?"	A pergunta diz o nome de quem vai sumir. Uma confirmacao generica ("Confirma a exclusao?") nao protege de nada, porque o erro que a confirmacao existe para pegar e justamente ter clicado na linha errada.
MessageBoxButtons.YesNo, MessageBoxIcon.Question	Dois botoes, porque ha uma escolha de verdade, e o icone de interrogacao, que e o que o Windows usa para pergunta. Trocar por <code>OKCancel</code> funcionaria, mas "OK" e uma palavra fraca para uma acao que nao tem volta.
if (resposta != DialogResult.Yes)	Leia como "se a resposta foi qualquer coisa diferente de Sim, va embora". Escrever <code>== DialogResult.No</code> daria no mesmo hoje, com dois botoes; escrever <code>!= Yes</code> continua correto no dia em que a caixa ganhar um terceiro botao ou um X que fecha. So o Sim explicito autoriza.
try { ... }	Comeca aqui, e nao no topo do metodo, porque so daqui para baixo o programa sai para conversar com o MySQL. Se o banco estiver fora do ar, e nesta parte que a coisa desanda.
UsuarioDAO.ExcluirConta(escolhido.Id);	A tela nao sabe escrever SQL e nao precisa saber. Ela entrega um numero e confia. Do outro lado, o TODO 13 monta o <code>DELETE ... WHERE Id = @id</code> — com <code>WHERE</code> , e com o Id entrando por parametro nomeado, nunca colado com <code>+</code> no texto do comando.
CarregarLista();	Sem esta chamada, a linha apagada continua na tela e, pior, continua dentro de <code>listaAtual</code> . Como este metodo acha o usuario pela posicao, uma lista desatualizada faz o proximo clique mirar na pessoa errada. Chamar <code>CarregarLista</code> refaz a leitura no banco, redesenha a ListBox e reconstrói

Linha	Por que ela esta ai
	<code>listaAtual</code> — as tres coisas de uma vez, porque as tres tem que andar juntas.
<code>catch (Exception ex)</code>	Pega o que der errado e transforma em recado. Sem ele, uma falha de banco fecha o programa na cara do usuario, com uma tela de erro em ingles. <code>ex</code> e o objeto da falha; <code>ex.Message</code> e a frase que o MySQL mandou.
<code>"Nao consegui excluir.\n\n" + ex.Message</code>	A sua frase primeiro, a do sistema depois. <code>\n</code> e a quebra de linha; dois deles deixam uma linha em branco entre as duas partes. O usuario le a sua; voce, no dia do problema, le a do MySQL.
<code>MessageBoxIcon.Error</code>	Aqui sim: X vermelho. As recusas M8 e M9 sao avisos de uso; esta e falha de verdade, e o icone tem que separar as duas coisas.

O erro mais comum

- **Usar o indice antes de conferir o -1.** O aluno escreve `Usuario escolhido = listaAtual[i];` logo na segunda linha e deixa o `if` para depois. Sintoma: enquanto voce clica sempre numa linha, tudo funciona. Na primeira vez que alguem clica em Excluir sem selecionar nada, o Visual Studio para em cima dessa linha com uma mensagem de "index out of range" — indice fora do intervalo. Saida: a conferencia vem **antes** do uso, sempre. Ponha um breakpoint na primeira linha, clique em Excluir sem selecionar nada e, quando a execucao parar, ande uma linha (F10) para a atribuicao acontecer: ver o -1 dentro de `i` com os proprios olhos vale mais que decorar a regra.
- **`listaAtual` virou variavel local no TODO 12.** Se la voce escreveu `List<Usuario> listaAtual = UsuarioDAO.ListarTodos(...)` com o tipo na frente, criou uma lista nova que morre no fim daquele metodo, e o campo da classe continua vazio. Sintoma cruel: a tela mostra os usuarios normalmente (o `for` usou a lista local), mas ao clicar em Excluir com uma linha selecionada da erro de indice fora do intervalo — porque `listaAtual.Count` aqui e zero. Saida: no TODO 12 a linha comeca em `listaAtual =`, sem tipo nenhum antes.
- **Esquecer o `return` depois da mensagem.** O codigo mostra a M8 e continua andando. Sintoma: aparece a mensagem "Escolha um usuario na lista primeiro." e, ao clicar OK nela, o programa quebra em seguida. Saida: toda recusa deste metodo termina em `return;` dentro das chaves do `if`.
- **Nao chamar `CarregarLista()` depois de excluir.** Sintoma: voce confirma a exclusao, nada acontece na tela, o nome continua ali. Parece que o botao nao funcionou. Confira no Workbench: a linha sumiu do banco — era so a tela que estava velha. Saida: a chamada vai logo depois do `ExcluirConta`, dentro do mesmo `try`.
- **Dar duplo clique no botao dentro do designer.** O evento `Click` deste botao **ja esta ligado** a este metodo. Um duplo clique no botao pela tela de desenho cria um segundo metodo, chamado `btnExcluir_Click_1`, e voce acaba escrevendo o codigo certo no metodo errado. Sintoma: o botao continua mostrando a mensagem "TODO 14 ainda nao foi escrito." mesmo depois de voce ter escrito tudo. Saida: apague o metodo extra e escreva dentro do `btnExcluir_Click` que ja existe no arquivo.
- **Comparar `escolhido.Nome` com o nome do logado** em vez de comparar os Ids. Sintoma: nao aparece erro nenhum, o programa roda; ele so recusa a conta errada no dia em que houver dois cadastros com o mesmo nome. Erro que passa no teste e falha na vida real.

Como saber que funcionou

Faca os quatro testes, nesta ordem. Voce precisa de pelo menos tres contas cadastradas — se nao tiver, cadastre pela tela de cadastro antes.

1. **A recusa M8.** Entre no sistema, chegue na lista e clique em "Excluir o selecionado" **sem clicar em nenhuma linha antes**. Tem que aparecer a caixa "Escolha um usuario na lista primeiro." com o triangulo amarelo. Clique OK: o programa continua vivo, nada foi apagado.

1. **A recusa M9.** Clique na linha da conta com que **voce** entrou e clique em Excluir. Atencao na hora de achar essa linha: o "Conectado como:" do alto da tela mostra so o **primeiro nome** (quem corta o resto e o `Regras.PrimeiroNome`, que ja vem pronto), enquanto a ListBox mostra o nome completo — procure a linha que comeca por esse primeiro nome. Tem que aparecer "Voce nao pode excluir a sua propria conta enquanto esta conectado." e nada mais.

1. **A desistencia.** Clique numa linha de outra pessoa, clique em Excluir e responda **Nao** na pergunta. A lista tem que ficar exatamente como estava. Confira no MySQL Workbench que a conta continua la:

```
sql SELECT Id, Nome, Login FROM Usuario ORDER BY Nome;
```

1. **A exclusao de verdade.** Antes de comecar, rode a consulta acima e anote quantas linhas voltaram. Agora selecione a conta de outra pessoa, clique em Excluir e responda **Sim**. Tres coisas tem que acontecer juntas: a linha some da ListBox, a numeracao das linhas restantes se reorganiza sem buracos (1, 2, 3... e nao 1, 3, 4), e o rodape passa a mostrar um usuario a menos. Rode a consulta de novo: uma linha a menos, e o nome apagado nao aparece.

Um teste extra, para o indice. Depois de excluir, troque a ordem no ComboBox (de "Por nome (A-Z)" para "Mais recentes primeiro") e exclua outra conta. Se a pessoa apagada for a que voce selecionou, o casamento entre a linha da tela e o item de `listaAtual` esta certo nas duas ordens. Se sumir a pessoa errada, o problema nao esta aqui: esta no TODO 12, que desenhou a tela sem seguir a ordem da lista.

Para pensar

`UsuarioDAO.ExcluirConta` devolve `true` ou `false` — true quando apagou mesmo, false quando nenhuma linha foi atingida — e este metodo joga esse retorno fora sem olhar. Em que situacao concreta esse `false` chegaria, e o que o usuario veria na tela hoje quando isso acontecesse?

PARTE 3 — A caça ao defeito

Um método deste projeto já vem escrito, e ele está com defeito. Não é engano: é o exercício de depuração da UC, e é o que fecha o indicador **I4**.

O sintoma

Entre no sistema com a conta de **Ana Souza**. No alto da tela principal, onde deveria aparecer

Conectado como: Ana

aparece

Conectado como: Souza

O defeito mora em `Regras.PrimeiroNome`.

O que ele não é

Não quebra o programa	Nenhuma exceção, nenhuma tela travada, nenhuma mensagem vermelha
Não é erro de compilação	O projeto compila com ele vivo. A Lista de Erros não vai te ajudar
Acerta às vezes	Com um nome de uma palavra só , a resposta sai certa

Este é o tipo de defeito que o compilador nunca acha: ele encontra o que está **mal escrito**, nunca o que está **mal pensado**.

Como caçar

1. Abra `Regras/Regras.cs` e ache o `PrimeiroNome`
2. Clique na margem cinza, à esquerda da linha do `Substring`. Aparece uma ****bola vermelha**** — é o breakpoint

1. Rode o programa e entre com uma conta
2. A execução **para** naquela linha e a linha fica amarela
3. Abra a janela **Locais** (Depurar > Janelas > Locais). Ela mostra o valor de cada variável **naquele instante**

1. Olhe o valor de `espaco`. Depois aperte **F10** para andar uma linha e veja o que o método devolve

A pergunta que resolve: o `Substring` tem duas formas. Uma recebe onde começar; a outra recebe onde começar e quantas letras levar. Qual delas está sendo usada, e qual deveria ser?

O Registro de Depuração

Você entrega um documento com **duas partes**:

1. Os três erros de conexão (da Parte 0, com o botão do TODO 1). Para cada um: o que você fez para provocar, e a mensagem que apareceu.

#	O que fazer	Mensagem que apareceu
1	Parar o serviço MySQL80 e clicar em Testar conexão	
2	Errar a senha no <code>Conexao.cs</code>	
3	Trocar <code>portariadb</code> por um nome que não existe	

2. O defeito: o que você viu de errado, onde pôs o breakpoint, **o valor que a janela Locais mostrou**, o que você mudou, e por que aquilo resolve.

Anexe **um print da execução pausada**, com a janela Locais visível.

É este documento que fecha o I4. Um sistema que compila prova metade; a outra metade é você mostrar que sabe usar o depurador.

PARTE 4 — O manual de utilização

O último entregável, e o que fecha o indicador **I6**. São **seis seções**, em PDF, com pelo menos uma imagem das telas nas seções 3, 4 e 5.

#	Seção	O que vai nela
1	Apresentação	Duas frases: para que serve o sistema
2	Requisitos e instalação	O que precisa estar instalado, e o <code>CriarBanco.sql</code>
3	Criar uma conta	Passos numerados, com print
4	Entrar no sistema	Print, e o que acontece se errar a senha
5	Ver e excluir usuários	Print, as duas ordens, a confirmação
6	Mensagens do sistema	A tabela das nove mensagens

A régua do manual

Escreva para a sua mãe, não para o professor.

Se a frase tem *método*, *instância*, *parâmetro*, *exceção* ou *classe*, ela está errada para este documento. Escreva "clique em **Entrar**", não "invoque o método de autenticação".

O teste: eu vou ler o seu manual fingindo nunca ter visto o sistema. Se, seguindo **só o que está escrito**, eu não conseguir criar uma conta e entrar, o indicador não atende.

A seção 6 você não inventa — você copia do seu código

As **nove mensagens** que você escreveu nos TODO 6, 9 e 14 estão numeradas de **M1 a M9** nos comentários do esqueleto. A tabela do manual tem exatamente essas nove linhas, **com o mesmo texto**.

#	Onde você escreveu	Quando aparece
M1	TODO 6	Algum campo em branco
M2	TODO 6	Login inválido (espaço, ou curto demais)
M3	TODO 6	Senha curta
M4	TODO 6	As duas senhas não são iguais
M5	TODO 6	Login já em uso
M6	TODO 9	Usuário ou senha em branco
M7	TODO 9	Usuário ou senha incorretos
M8	TODO 14	Nenhuma linha selecionada
M9	TODO 14	Tentou excluir a própria conta

Uma das perguntas da arguição é literalmente: "abra a linha M4 do seu manual e me mostre onde ela está no seu código." Se as duas não baterem, aparece.

O roteiro de teste final

Antes de entregar, passe por estes treze. **Se os treze passarem, o sistema está pronto.**

#	O que fazer	Deve acontecer
1	Clicar em Testar conexão	Faixa verde e mensagem de OK
2	Criar conta, fechar o programa, abrir e entrar	A conta continua lá
3	Cadastrar com um campo em branco	Recusa (M1)
4	Cadastrar com usuário <code>ab</code> , e depois com <code>ana bia</code>	Recusa nos dois (M2)
5	Cadastrar com senha <code>123</code>	Recusa (M3)
6	Digitar senhas diferentes	Recusa (M4)
7	Cadastrar com o usuário <code>ana</code>	Recusa (M5)
8	Entrar com a senha errada	Recusa e não quebra (M7)
9	Trocar a ordem no ComboBox	A lista muda de ordem
10	Clicar em Excluir sem escolher ninguém	Recusa (M8)
11	Tentar excluir a si mesmo	Recusa (M9)
12	Excluir outro usuário	Pergunta antes, e some da lista
13	Cadastrar um nome com acento	Aparece certo na lista

O que você entrega

#	O quê	Indicador
1	Termo de Configuração do Ambiente, com os 4 prints	I1
2	A pasta do projeto, sem <code>bin/</code> e sem <code>obj/</code>	I2 · I3 · I5
3	Registro de Depuração (3 erros de conexão + o defeito), com print	I4
4	Manual em PDF , 6 seções	I6
5	Arguição de 3 minutos	todos

Nome da pasta: `Portaria-SeuNome`

⚠ **Antes de entregar, o `Conexao.cs` volta para `SUA_SENHA_AQUI`.** Entregar a própria senha é falha da atitude **A2 — sigilo no tratamento de dados**, e ela é observada.

Se o MySQL não conectar

Mensagem	O que fazer
Não achei o servidor	O serviço MySQL80 está parado. <code>services.msc</code> e iniciar
Usuário ou senha incorretos	A senha do <code>Conexao.cs</code> não é a sua
O banco <code>portariadb</code> não existe	Falta rodar o <code>CriarBanco.sql</code>
A tabela <code>Usuario</code> não existe	O script rodou pela metade. Rode de novo